

Problem Statement

Context

A significant number of hotel bookings are called off due to cancellations or no-shows. The typical reasons for cancellations include change of plans, scheduling conflicts, etc. This is often made easier by the option to do so free of charge or preferably at a low cost, which is beneficial to hotel guests, but it is a less desirable and possibly revenue-diminishing factor for hotels to deal with. Such losses are particularly high on last-minute cancellations.

The new technologies involving online booking channels have dramatically changed customers' booking possibilities and behavior. This adds a further dimension to the challenge of how hotels handle cancellations, which are no longer limited to traditional booking and guest characteristics.

The cancellation of bookings impacts a hotel on various fronts:

1. Loss of resources (revenue) when the hotel cannot resell the room.
2. Additional costs of distribution channels by increasing commissions or paying for publicity to help sell these rooms.
3. Lowering prices last minute, so the hotel can resell a room, resulting in a reduction of the profit margin.
4. Human resources to make arrangements for the guests.

Objective

The increasing number of cancellations calls for a Machine Learning based solution that can help in predicting which booking is likely to be canceled. INN Hotels Group has a chain of hotels in Portugal, and they are facing problems with the high number of booking cancellations and have reached out to your firm for data-driven solutions. As a data scientist have to analyze the data provided to find which factors have a high influence on booking cancellations, build a predictive model that can predict which bookings are going to be canceled in advance, and help in formulating profitable policies for cancellations and refunds.

Data Description

The data contains the different attributes of customers' booking details. The detailed data dictionary is given below.

Data Dictionary

- Booking_ID: the unique identifier of each booking
- no_of_adults: Number of adults
- no_of_children: Number of Children

- no_of_weekend_nights: Number of weekend nights (Saturday or Sunday) the guest stayed or booked to stay at the hotel
- no_of_week_nights: Number of weeknights (Monday to Friday) the guest stayed or booked to stay at the hotel
- type_of_meal_plan: Type of meal plan booked by the customer:
 - Not Selected – No meal plan selected
 - Meal Plan 1 – Breakfast
 - Meal Plan 2 – Half board (breakfast and one other meal)
 - Meal Plan 3 – Full board (breakfast, lunch, and dinner)
- required_car_parking_space: Does the customer require a car parking space? (0 - No, 1- Yes)
- room_type_reserved: Type of room reserved by the customer. The values are ciphered (encoded) by INN Hotels Group
- lead_time: Number of days between the date of booking and the arrival date
- arrival_year: Year of arrival date
- arrival_month: Month of arrival date
- arrival_date: Date of the month
- market_segment_type: Market segment designation.
- repeated_guest: Is the customer a repeated guest? (0 - No, 1- Yes)
- no_of_previous_cancellations: Number of previous bookings that were canceled by the customer prior to the current booking
- no_of_previous_bookings_not_canceled: Number of previous bookings not canceled by the customer prior to the current booking
- avg_price_per_room: Average price per day of the reservation; prices of the rooms are dynamic. (in euros)
- no_of_special_requests: Total number of special requests made by the customer (e.g. high floor, view from the room, etc)
- booking_status: Flag indicating if the booking was canceled or not.

Loading Libraries

```
In [ ]: # this will help in making the Python code more structured automatically (good c
        # %load_ext nb_black

        # Library to suppress warnings or deprecation notes
        import warnings

        warnings.filterwarnings("ignore")

        # Libraries to help with reading and manipulating data

        import pandas as pd
        import numpy as np

        # Library to split data
        from sklearn.model_selection import train_test_split
```

```
# Libraries to help with data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Removes the limit for the number of displayed columns
pd.set_option("display.max_columns", None)
# Sets the limit for the number of displayed rows
pd.set_option("display.max_rows", 200)

# Libraries to build decision tree classifier
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

# To build linear regression_model
import statsmodels.api as sm

# To tune different models
from sklearn.model_selection import GridSearchCV

# To perform statistical analysis
import scipy.stats as stats

# To get different metric scores
from sklearn.metrics import (
    f1_score,
    accuracy_score,
    recall_score,
    precision_score,
    confusion_matrix,
    precision_recall_curve,
    roc_curve,
    roc_auc_score,
    classification_report,
    # plot_confusion_matrix,
    make_scorer,
)
```

Loading data

```
In [ ]: inn_hotels=pd.read_csv('INNHôtelsGroup.csv')
inn_hotels.head(10)
```

Out []:

	Booking_ID	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights
0	INN00001	2	0	1	2
1	INN00002	2	0	2	3
2	INN00003	1	0	2	1
3	INN00004	2	0	0	2
4	INN00005	2	0	1	1
5	INN00006	2	0	0	2
6	INN00007	2	0	1	3
7	INN00008	2	0	1	3
8	INN00009	3	0	0	4
9	INN00010	2	0	0	5



In []: `inn_hotels.tail(10)`

Out []:

	Booking_ID	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nig
36265	INN36266	2	0	1	
36266	INN36267	2	0	2	
36267	INN36268	2	0	1	
36268	INN36269	1	0	0	
36269	INN36270	2	2	0	
36270	INN36271	3	0	2	
36271	INN36272	2	0	1	
36272	INN36273	2	0	2	
36273	INN36274	2	0	0	
36274	INN36275	2	0	1	



Understand the shape of the dataset.

In []: `inn_hotels.shape`

Out []: (36275, 19)

- The dataset contains with 36275 rows and 19 columns.

Check the data types of the columns for the dataset.

In []: `inn_hotels.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 36275 entries, 0 to 36274
Data columns (total 19 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Booking_ID                               36275 non-null  object
1   no_of_adults                             36275 non-null  int64
2   no_of_children                           36275 non-null  int64
3   no_of_weekend_nights                     36275 non-null  int64
4   no_of_week_nights                       36275 non-null  int64
5   type_of_meal_plan                        36275 non-null  object
6   required_car_parking_space               36275 non-null  int64
7   room_type_reserved                       36275 non-null  object
8   lead_time                               36275 non-null  int64
9   arrival_year                             36275 non-null  int64
10  arrival_month                            36275 non-null  int64
11  arrival_date                             36275 non-null  int64
12  market_segment_type                     36275 non-null  object
13  repeated_guest                           36275 non-null  int64
14  no_of_previous_cancellations             36275 non-null  int64
15  no_of_previous_bookings_not_canceled     36275 non-null  int64
16  avg_price_per_room                       36275 non-null  float64
17  no_of_special_requests                   36275 non-null  int64
18  booking_status                           36275 non-null  object
dtypes: float64(1), int64(13), object(5)
memory usage: 5.3+ MB
```

- There are 14 numerical columns and 5 categorical columns in the dataset.
- `required_car_parking_space`, `repeated_guest`, `no_of_previous_cancellations`, `no_of_previous_bookings_not_canceled` these are the columns where most number of 0 values, so it means we can drop them for further case studies purpose.
- Here, the dependent variable **booking_status** is categorical in type.

In []: `#Converting booking status into numerical type`
`inn_hotels['booking_status']=inn_hotels['booking_status'].replace({'Canceled':1,`
`inn_hotels.head()`

Out[]:

	Booking_ID	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights
0	INN00001	2	0	1	2
1	INN00002	2	0	2	3
2	INN00003	1	0	2	1
3	INN00004	2	0	0	2
4	INN00005	2	0	1	1

```
In [ ]: # Missing values
inn_hotels.isnull().sum()
```

```
Out[ ]: 0
```

Booking_ID	0
no_of_adults	0
no_of_children	0
no_of_weekend_nights	0
no_of_week_nights	0
type_of_meal_plan	0
required_car_parking_space	0
room_type_reserved	0
lead_time	0
arrival_year	0
arrival_month	0
arrival_date	0
market_segment_type	0
repeated_guest	0
no_of_previous_cancellations	0
no_of_previous_bookings_not_canceled	0
avg_price_per_room	0
no_of_special_requests	0
booking_status	0

dtype: int64

- It is clearly visible that there is no missing values in the dataset.

```
In [ ]: # Checking duplicate values
inn_hotels.duplicated().sum()
```

```
Out[ ]: np.int64(0)
```

- There are no duplicate values either.

Statistical summary

```
In [ ]: inn_hotels.describe().T
```

Out[]:

	count	mean	std	min	25%
no_of_adults	36275.0	1.844962	0.518715	0.0	2.0
no_of_children	36275.0	0.105279	0.402648	0.0	0.0
no_of_weekend_nights	36275.0	0.810724	0.870644	0.0	0.0
no_of_week_nights	36275.0	2.204300	1.410905	0.0	1.0
required_car_parking_space	36275.0	0.030986	0.173281	0.0	0.0
lead_time	36275.0	85.232557	85.930817	0.0	17.0
arrival_year	36275.0	2017.820427	0.383836	2017.0	2018.0
arrival_month	36275.0	7.423653	3.069894	1.0	5.0
arrival_date	36275.0	15.596995	8.740447	1.0	8.0
repeated_guest	36275.0	0.025637	0.158053	0.0	0.0
no_of_previous_cancellations	36275.0	0.023349	0.368331	0.0	0.0
no_of_previous_bookings_not_canceled	36275.0	0.153411	1.754171	0.0	0.0
avg_price_per_room	36275.0	103.423539	35.089424	0.0	80.3
no_of_special_requests	36275.0	0.619655	0.786236	0.0	0.0
booking_status	36275.0	0.327636	0.469358	0.0	0.0



- The number of adults ranges from 0 to 4, which is usual.
- The maximum number of children is 10, which is a bit unusual and might require a check.
- The range of the number of weeks and weekend nights seems fine. Though 7 weekends might be a very long stay.
- At least 75% of the customers do not require car parking space.
- On an average the customers book 85 days in advance and there are few customers who book from 126 days up to 443 days in advance which seems to be an outlier.
- At least 75% of the customers are not repeating customers.
- At least 75% of the customers who didn't cancel any previous bookings but there are also some customers who canceled the booking more than 12 times.
- The average price per room is 103 euros and in some of the cases the price per room goes up to 540 euros which seems to be an outlier.

In []:

```
#Drop Booking ID from the dataframe
inn_hotels.drop(['Booking_ID'],axis=1,inplace=True)
```

Exploratory Data Analysis(EDA)

Note:In this case study, we will mainly focus on the model building aspects. We will only be looking at the key observations from EDA.

```
In [ ]: def histogram_boxplot(data, feature, figsize=(10, 7), kde=False, bins=None):
        """
        Boxplot and histogram combined

        data: dataframe
        feature: dataframe column
        figsize: size of figure (default (15,10))
        kde: whether to show the density curve (default False)
        bins: number of bins for histogram (default None)
        """

        f2, (ax_box2, ax_hist2) = plt.subplots(
            nrows=2, # Number of rows of the subplot grid= 2
            sharex=True, # x-axis will be shared among all subplots
            gridspec_kw={"height_ratios": (0.25, 0.75)},
            figsize=figsize,
        ) # creating the 2 subplots
        sns.boxplot(
            data=data, x=feature, ax=ax_box2, showmeans=True, color="violet"
        ) # boxplot will be created and a triangle will indicate the mean value of
        sns.histplot(
            data=data, x=feature, kde=kde, ax=ax_hist2, bins=bins
        ) if bins else sns.histplot(
            data=data, x=feature, kde=kde, ax=ax_hist2
        ) # For histogram
        ax_hist2.axvline(
            data[feature].mean(), color="green", linestyle="--"
        ) # Add mean to the histogram
        ax_hist2.axvline(
            data[feature].median(), color="black", linestyle="-"
        ) # Add median to the histogram
```

```
In [ ]: # function to create Labeled barplots

def labeled_barplot(data, feature, perc=False, n=None):
    """
    Barplot with percentage at the top

    data: dataframe
    feature: dataframe column
    perc: whether to display percentages instead of count (default is False)
    n: displays the top n category levels (default is None, i.e., display all le
    """

    total = len(data[feature]) # Length of the column
    count = data[feature].nunique()
    if n is None:
        plt.figure(figsize=(count + 2, 6))
    else:
        plt.figure(figsize=(n + 2, 6))

    plt.xticks(rotation=90, fontsize=10)
    ax = sns.countplot(
        data=data,
        x=feature,
        palette="Paired",
        order=data[feature].value_counts().index[:n],
    )
```



```

for p in ax.patches:
    if perc == True:
        label = "{:.1f}%".format(
            100 * p.get_height() / total
        ) # percentage of each class of the category
    else:
        label = p.get_height() # count of each level of the category

x = p.get_x() + p.get_width() / 2 # width of the plot
y = p.get_height() # height of the plot

ax.annotate(
    label,
    (x, y),
    ha="center",
    va="center",
    size=12,
    xytext=(0, 5),
    textcoords="offset points",
) # annotate the percentage

plt.show() # show the plot

```

```

In [ ]: def stacked_barplot(data, predictor, target):
        """
        Print the category counts and plot a stacked bar chart

        data: dataframe
        predictor: independent variable
        target: target variable
        """
        count = data[predictor].nunique()
        sorter = data[target].value_counts().index[-1]
        tab1 = pd.crosstab(data[predictor], data[target], margins=True).sort_values(
            by=sorter, ascending=False
        )
        print(tab1)
        print("-" * 120)
        tab = pd.crosstab(data[predictor], data[target], normalize="index").sort_val
            by=sorter, ascending=False
        )
        tab.plot(kind="bar", stacked=True, figsize=(count + 5, 5))
        plt.legend(
            loc="lower left", frameon=False,
        )
        plt.legend(loc="upper left", bbox_to_anchor=(1, 1))
        plt.show()

```

```

In [ ]: ### function to plot distributions wrt target

def distribution_plot_wrt_target(data, predictor, target):

    fig, axs = plt.subplots(2, 2, figsize=(10, 10))

    target_uniq = data[target].unique()

    axs[0, 0].set_title("Distribution of target for target=" + str(target_uniq[0]

```

```

sns.histplot(
    data=data[data[target] == target_uniq[0]],
    x=predictor,
    kde=True,
    ax=axes[0, 0],
    color="teal",
    stat="density",
)

axes[0, 1].set_title("Distribution of target for target=" + str(target_uniq[1]))
sns.histplot(
    data=data[data[target] == target_uniq[1]],
    x=predictor,
    kde=True,
    ax=axes[0, 1],
    color="orange",
    stat="density",
)

axes[1, 0].set_title("Boxplot w.r.t target")
sns.boxplot(data=data, x=target, y=predictor, ax=axes[1, 0], palette="gist_rainbow")

axes[1, 1].set_title("Boxplot (without outliers) w.r.t target")
sns.boxplot(
    data=data,
    x=target,
    y=predictor,
    ax=axes[1, 1],
    showfliers=False,
    palette="gist_rainbow",
)

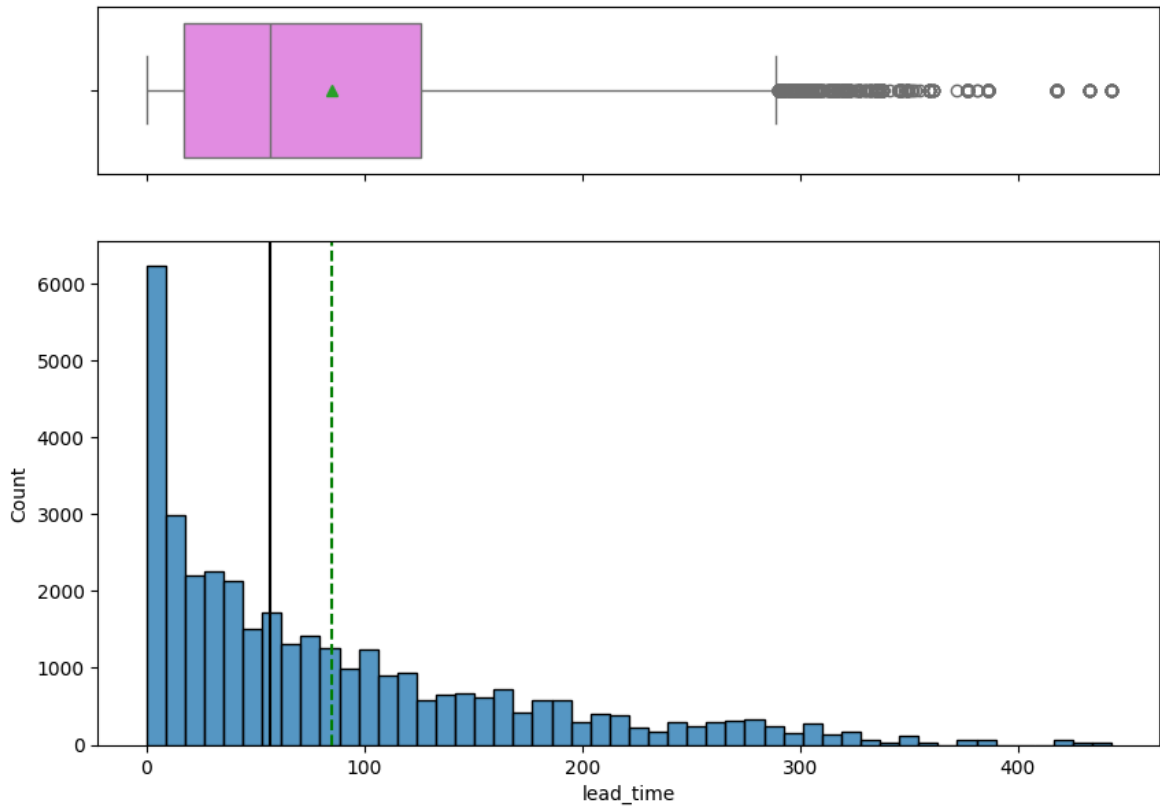
plt.tight_layout()
plt.show()

```

Univariate Analysis

Observations on lead time

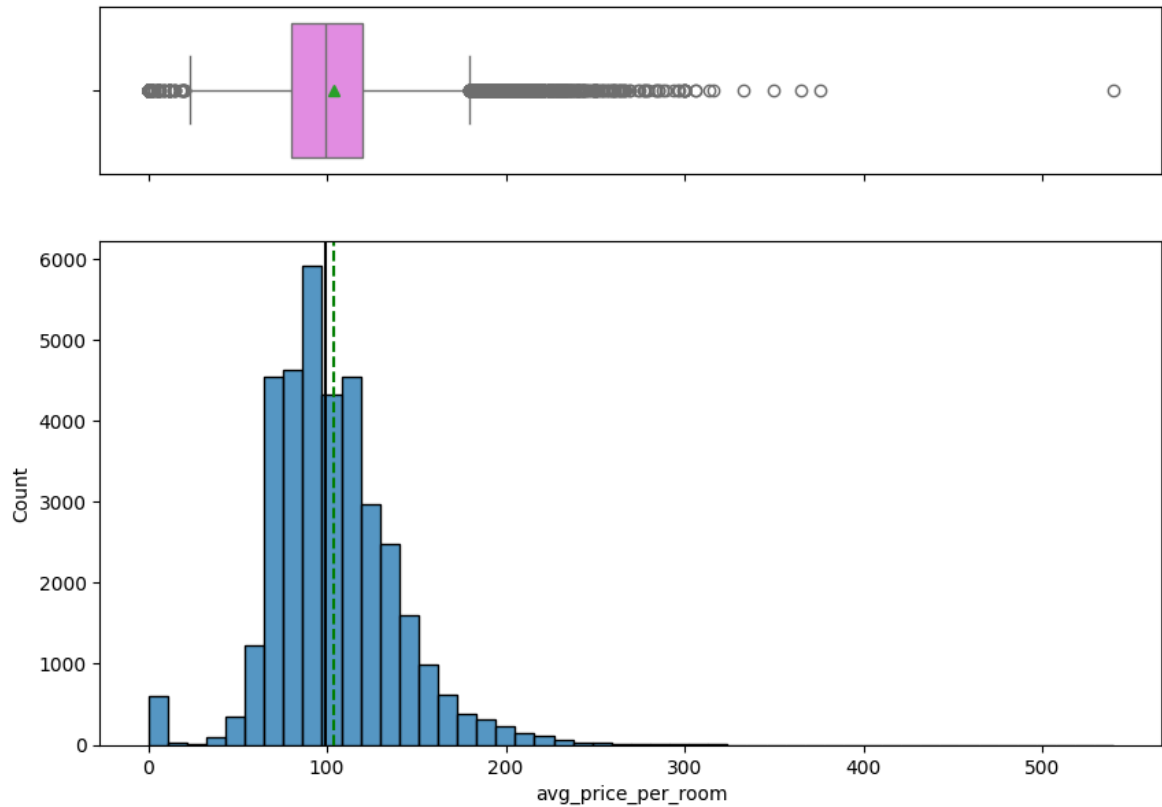
```
In [ ]: histogram_boxplot(inn_hotels, 'lead_time', bins=50)
```



- The distribution is Right Skewed with so many outliers.
- The majority of clients have made their booking on the same day of their arrival(At day 0).
- Some of the customers made reservations around 100-500 days in advance.

Observations on average price per room

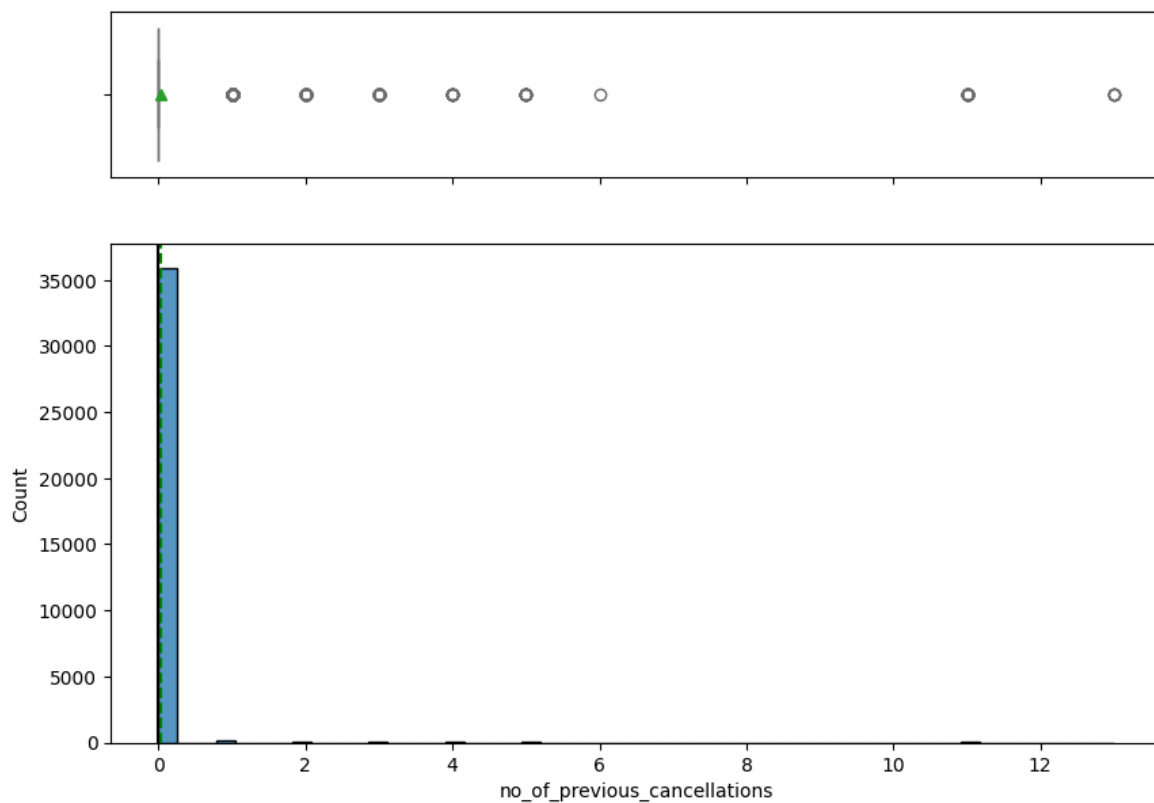
```
In [ ]: histogram_boxplot(inn_hotels, 'avg_price_per_room', bins=50)
```



- This is a Right Skewed distribution and there are some outliers in the both ends of this distribution.
- The average price per room is around ~103 euros.
- There are some rooms which price cost 540 euros as well as some rooms that have a price equal to 0. So, we have to further check it.

Observations on Number of Previous Booking Cancellations

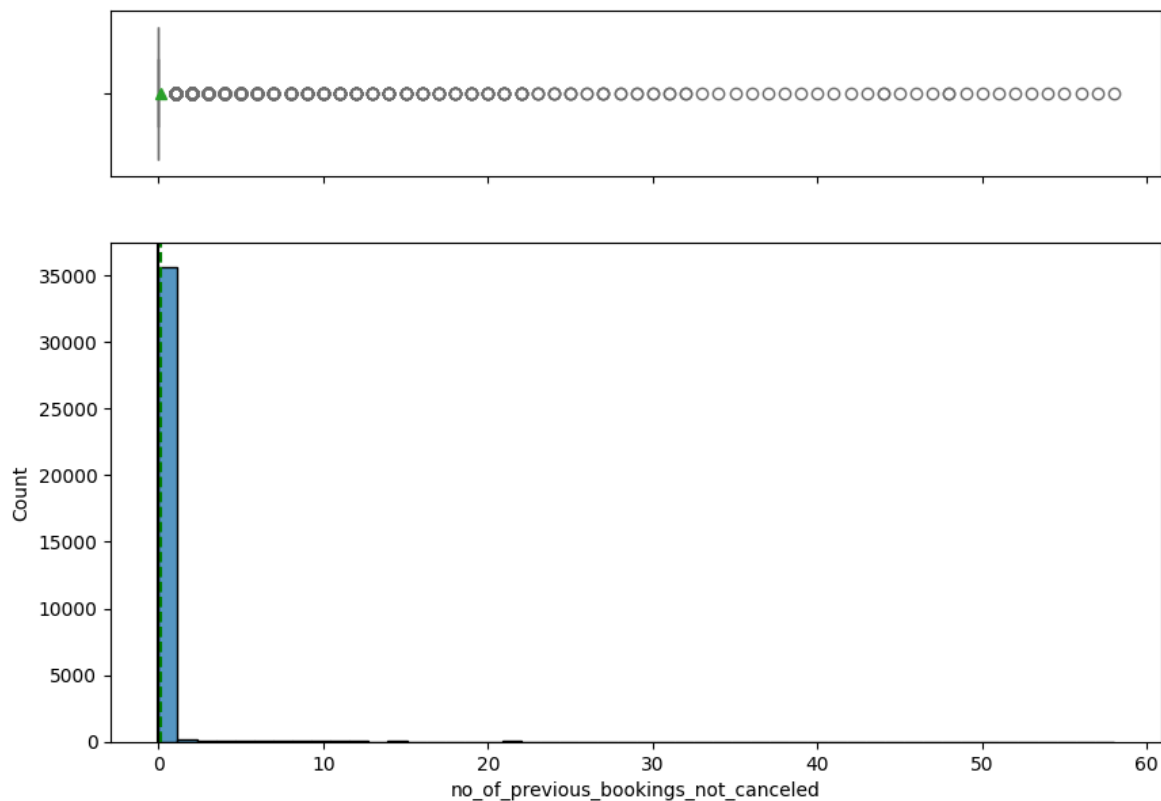
```
In [ ]: histogram_boxplot(inn_hotels,'no_of_previous_cancellations', bins=50)
```



- The customers who previously cancelled, have cancelled mostly one time.
- However, some customers cancelled the bookings more than 12 times.

Observations on Number of Previous booking Not cancelled

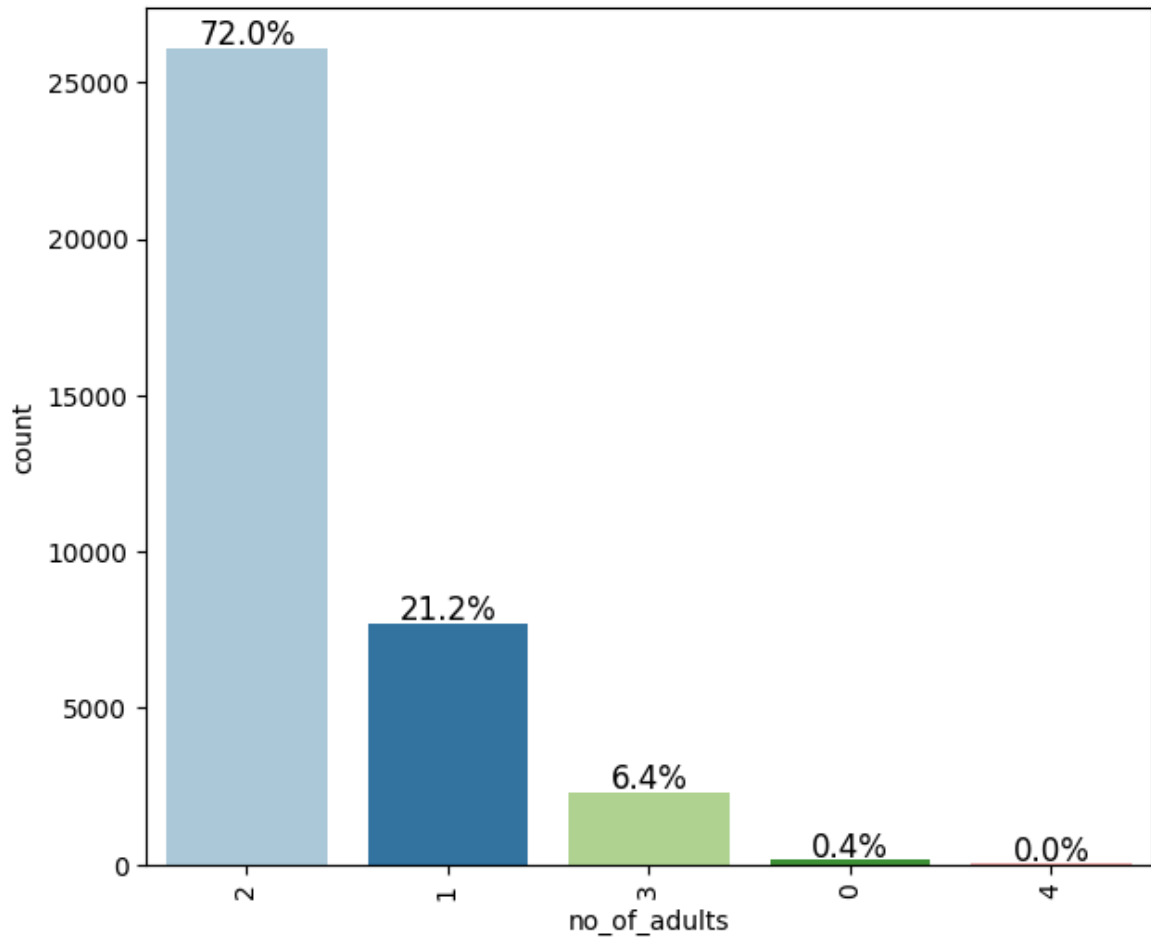
```
In [ ]: histogram_boxplot(inn_hotels, 'no_of_previous_bookings_not_canceled', bins=50)
```



- Very few customers have more than one booking not cancelled previously.
- Some customers have not cancelled their bookings around 60 times.

Observations on Number of Adults

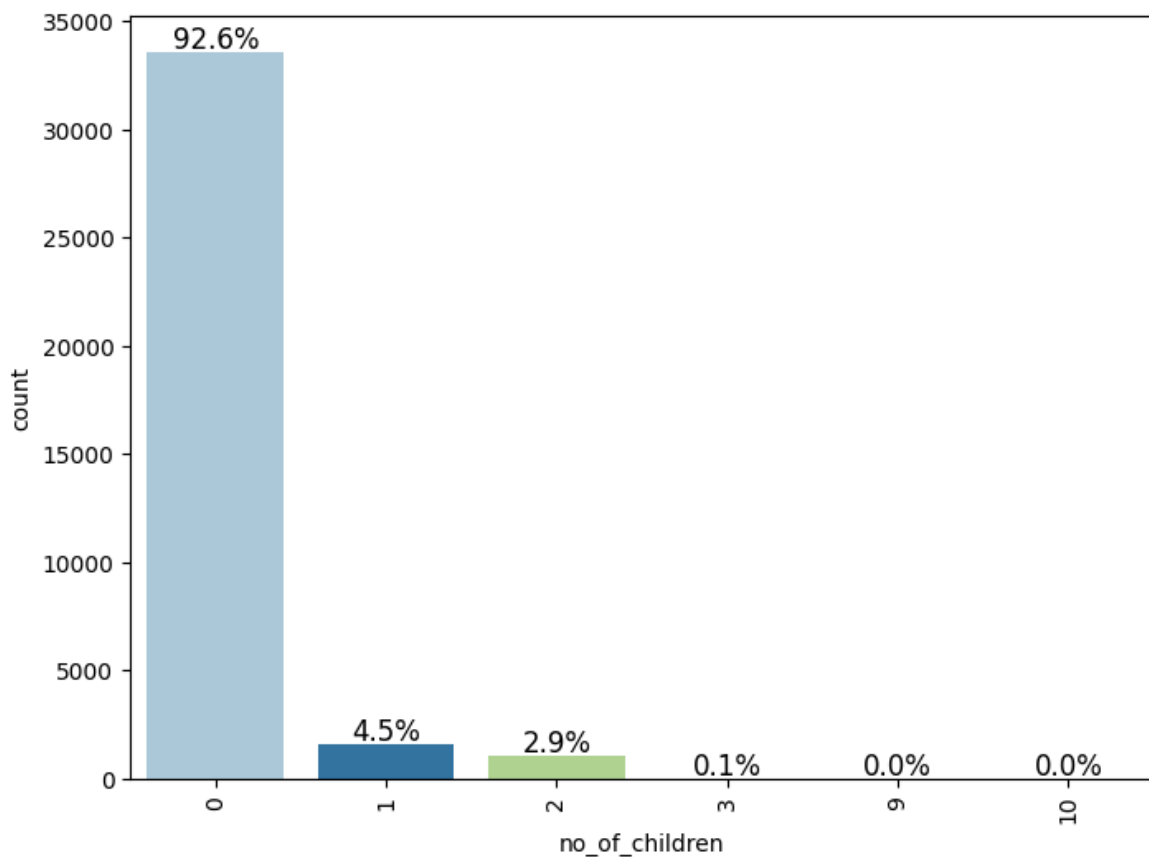
```
In [ ]: labeled_barplot(inn_hotels, 'no_of_adults', perc=True)
```



- 72% of the bookings were made for 2 adults. 21% of the bookings were made for one adult. That means most of the travellers who have chosen this hotel are couples and solo travellers.

Observations on Number of Children

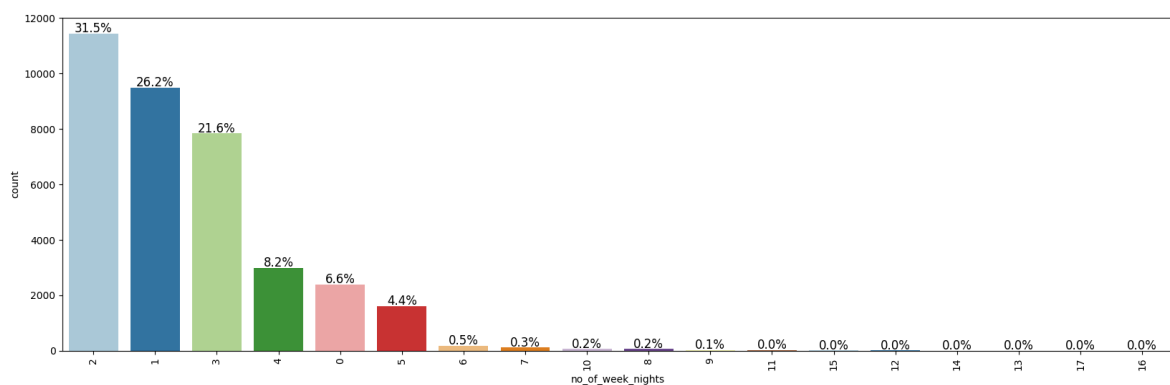
```
In [ ]: labeled_barplot(inn_hotels, 'no_of_children', perc=True)
```



- 93% of the customers didn't make reservations for children.
- There are some values in the data where the number of children is 9 or 10, which is likely unlikely.

Observations on Number of Week Nights

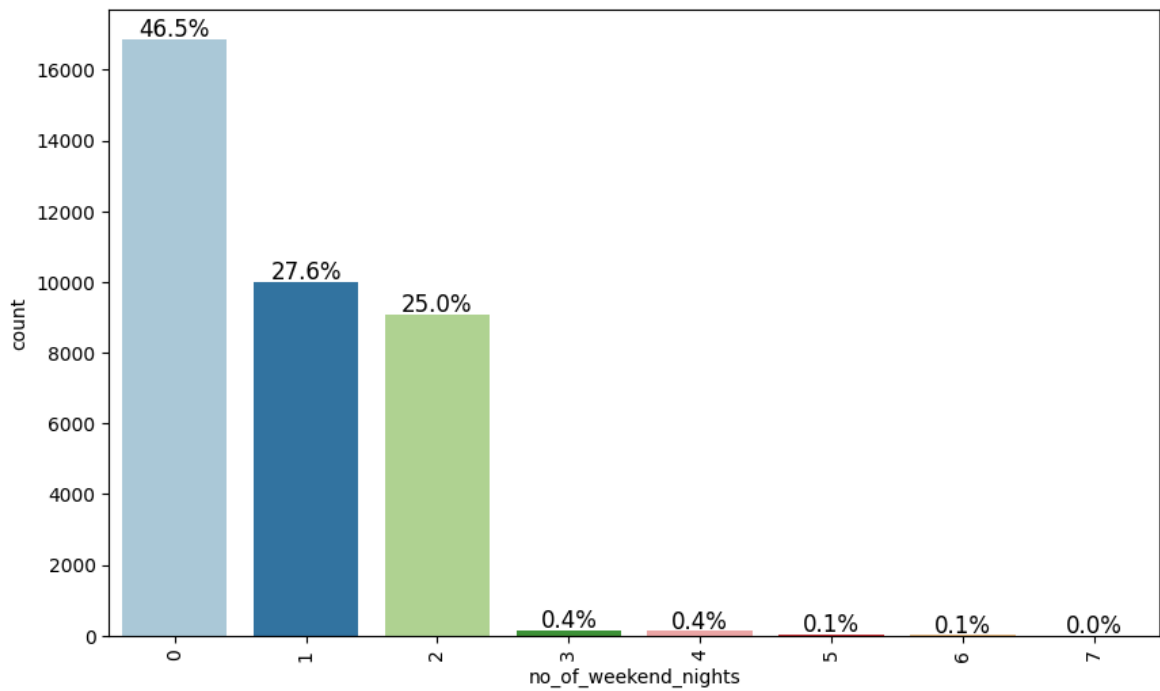
```
In [ ]: labeled_barplot(inn_hotels, 'no_of_week_nights', perc=True)
```



- 31% of the bookings are made for 2 nights and 26% of the bookings are made for 1 night and 22% of the bookings are made for 3 nights stay.

Observations on Number of Weekend Nights

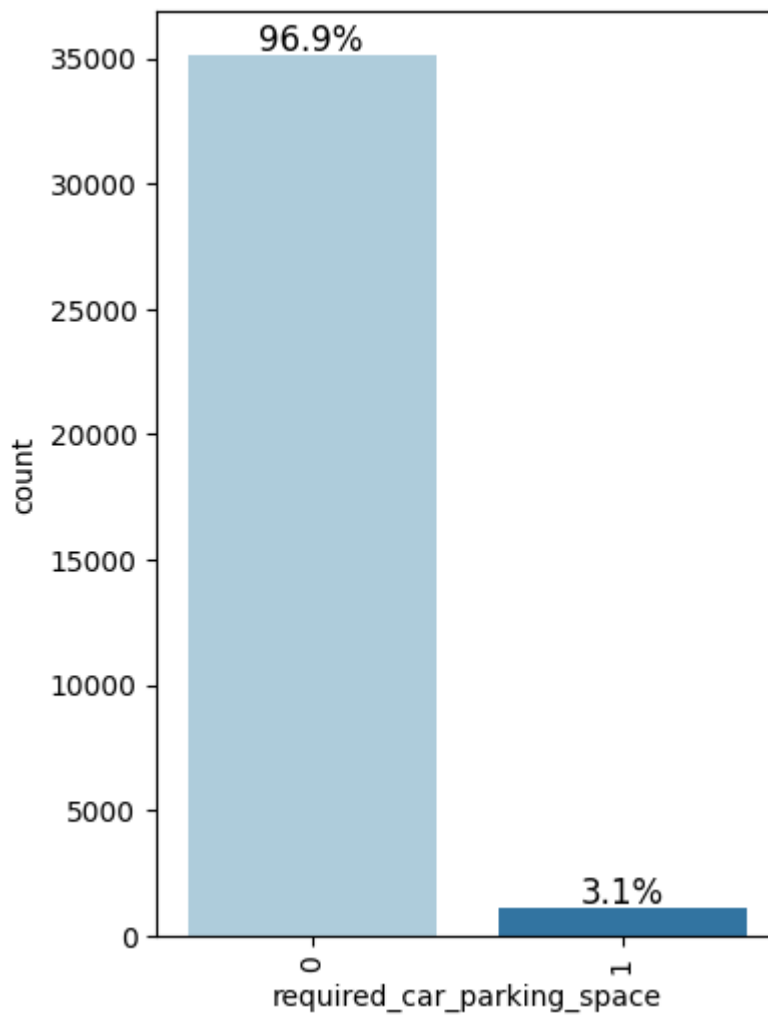
```
In [ ]: labeled_barplot(inn_hotels, 'no_of_weekend_nights', perc=True)
```



- Almost 47% of the customers do not plan to spend the weekend in the hotel.
- 28% and 25% of the customers have planned to stay at least 1 and 2 weekends in the hotel.

Observations on Required Car Parking Space

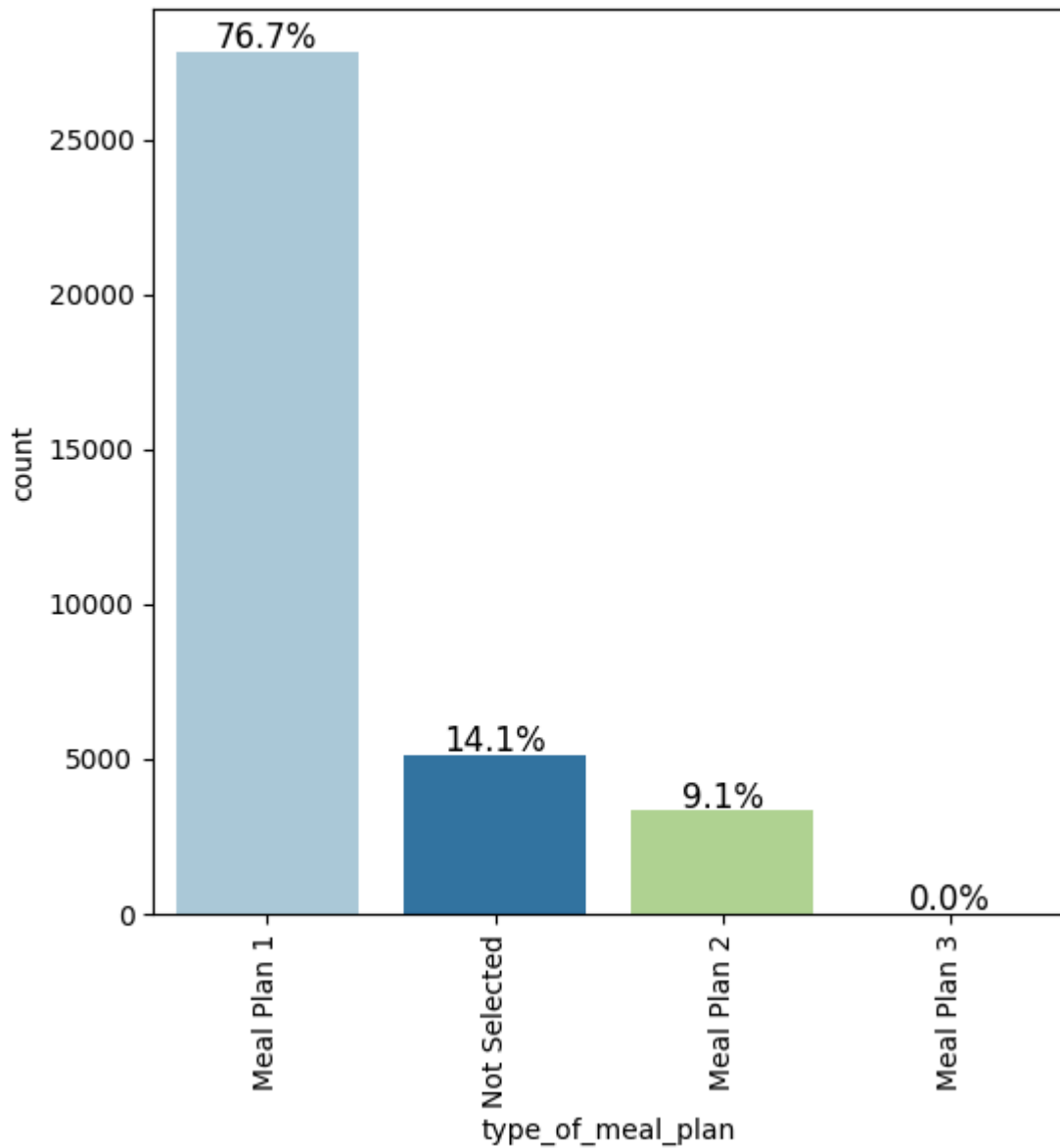
```
In [ ]: labeled_barplot(inn_hotels, 'required_car_parking_space', perc=True)
```

- 97% of the customers do not require a car parking space.

Observations on Type of Meal Plan

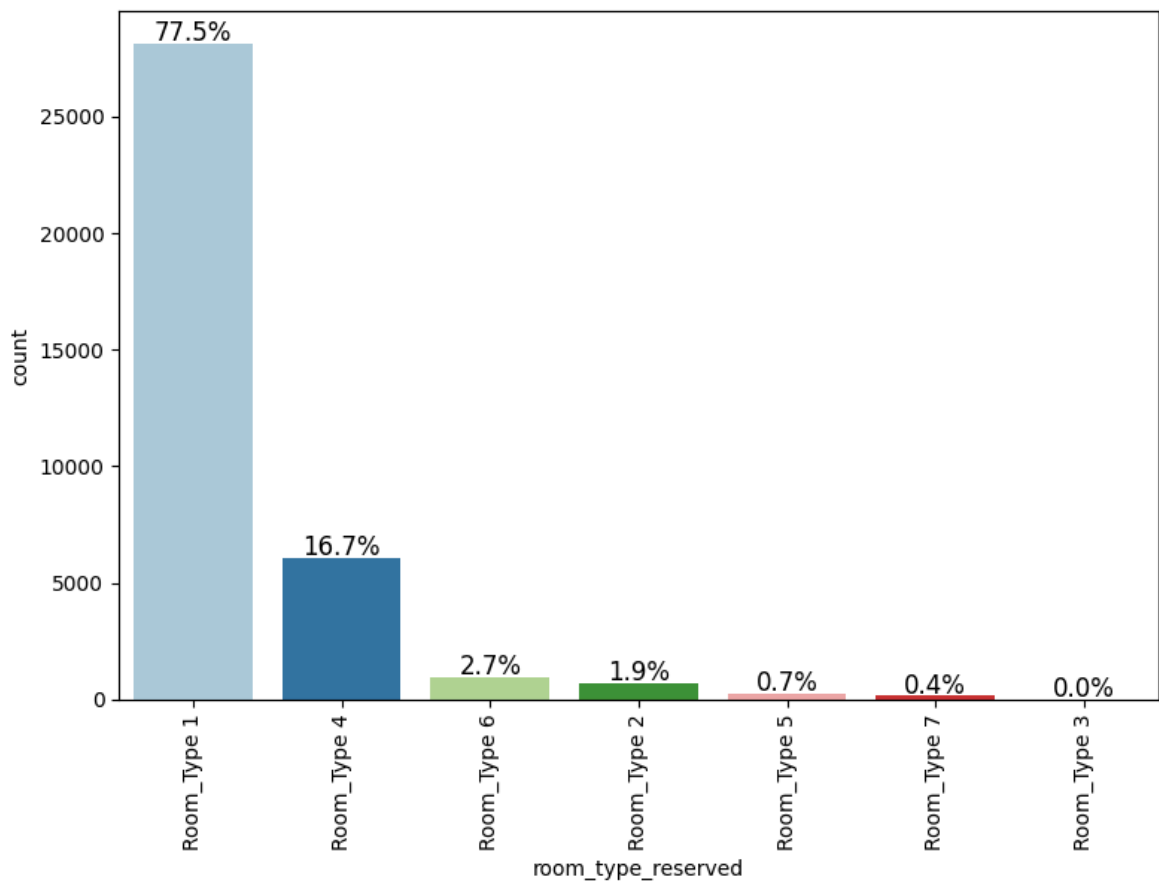
```
In [ ]: labeled_barplot(inn_hotels, 'type_of_meal_plan', perc=True)
```



- Most of the customers prefer meal plan 1(breakfast).
- There are 14% of the customers who didn't select any meal plan.

Observations on Room Type Reserved

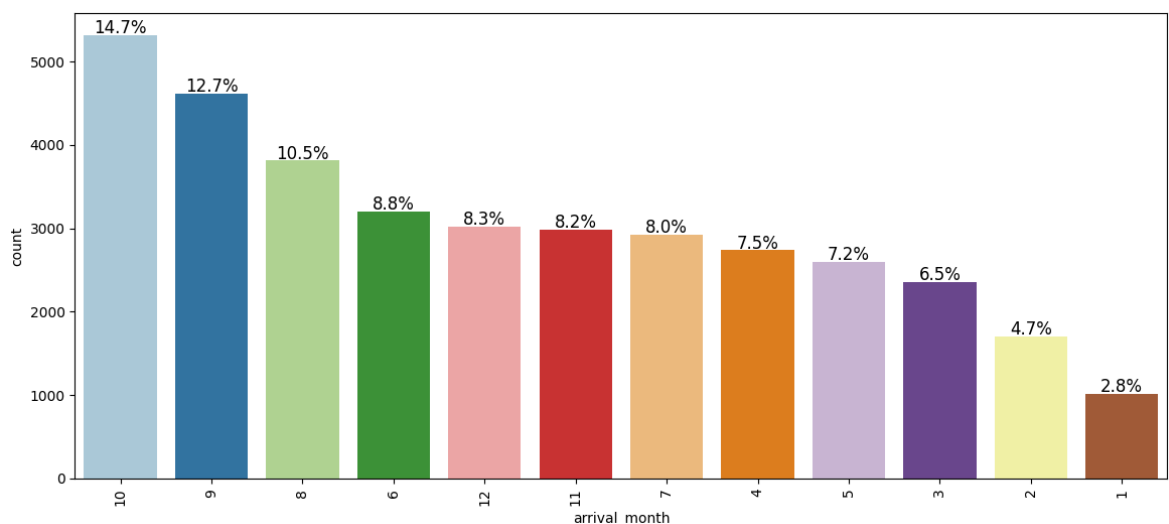
```
In [ ]: labeled_barplot(inn_hotels, 'room_type_reserved', perc=True)
```



- Almost 78% of the customers booked Room_Type 1 followed by 17% of the customers booking Room_Type 4.

Observations on Arrival Month

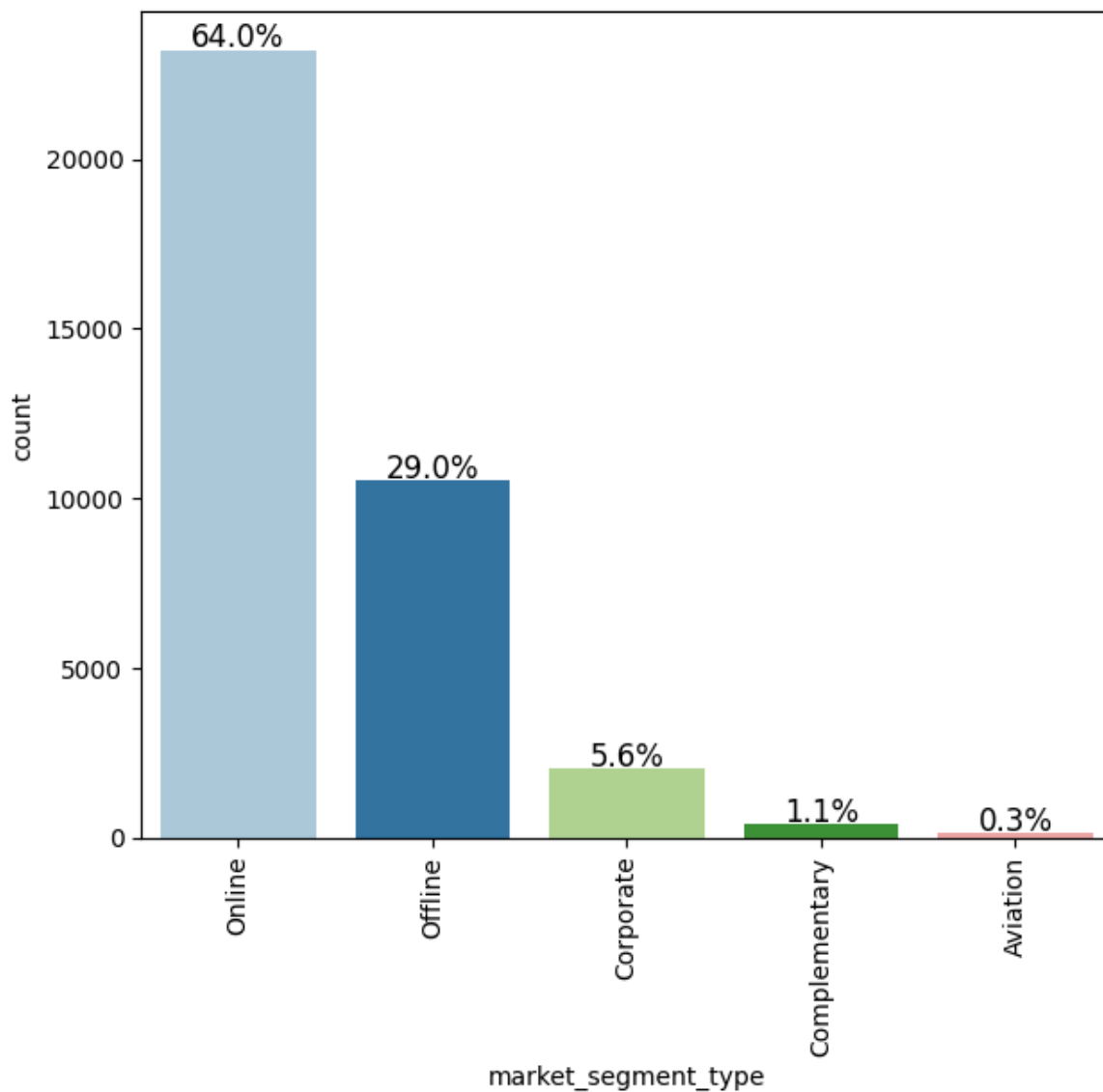
```
In [ ]: labeled_barplot(inn_hotels,'arrival_month', perc=True)
```



- October is the busiest month for the hotel followed by September.
- There are almost 17% of the bookings in October month followed by 13% of the bookings in the September month.

Observations on Market Segment Type

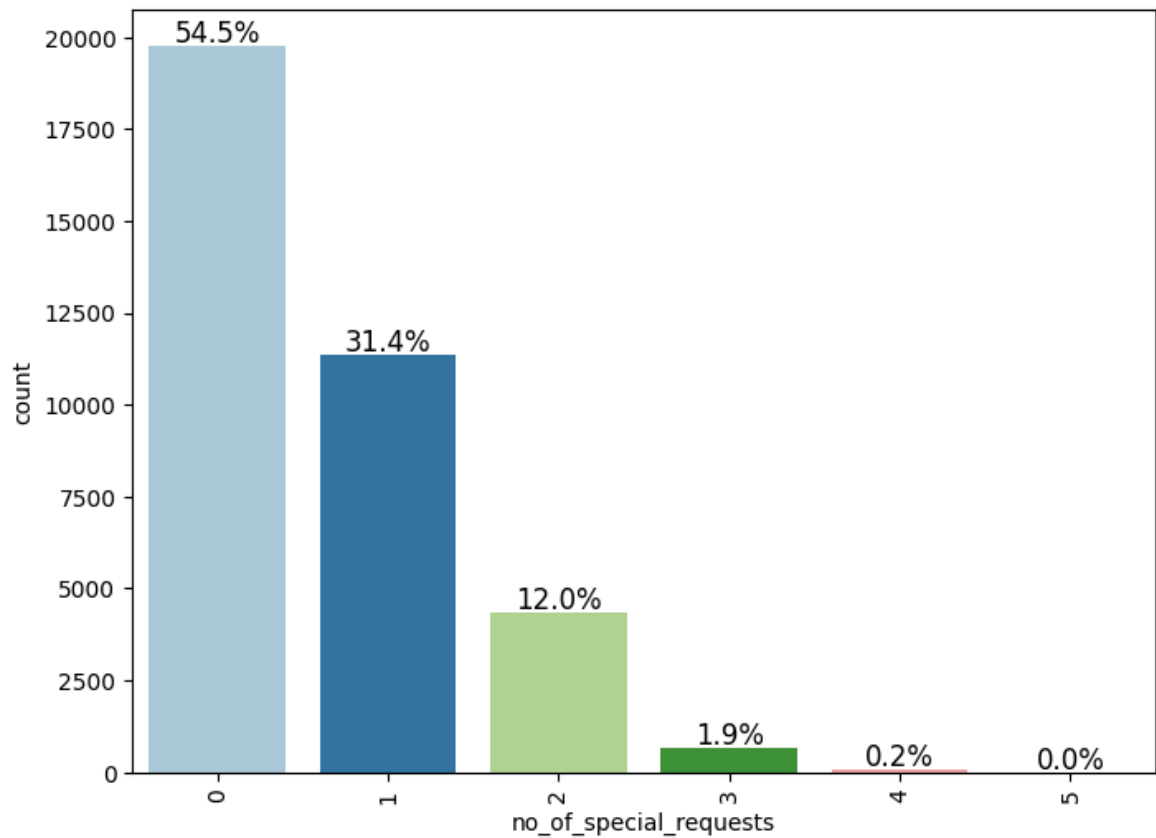
```
In [ ]: labeled_barplot(inn_hotels, 'market_segment_type', perc=True)
```



- 64% of the customers booked the hotel online and 29% of the hotel bookings were made offline.

Observations on Number of Special Request

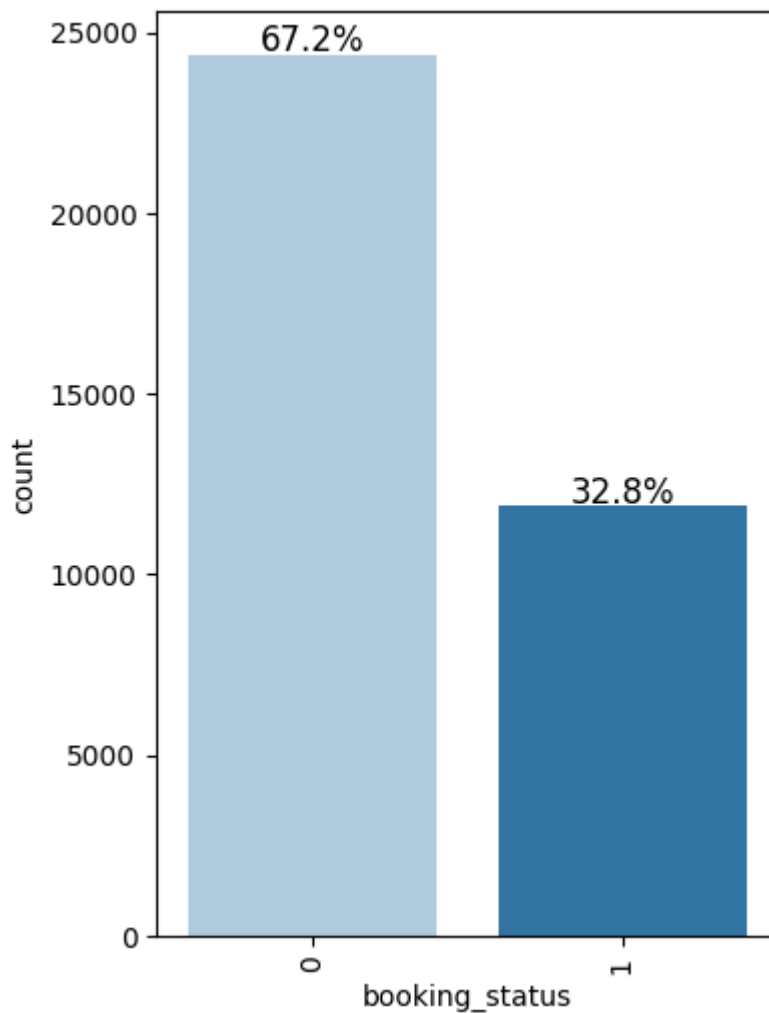
```
In [ ]: labeled_barplot(inn_hotels, 'no_of_special_requests', perc=True)
```



- Almost 55% of the customers generally do not make any requests while booking a hotel room.
- There are 31% of the customers having at least 1 special request.

Observations on Booking Status

```
In [ ]: labeled_barplot(inn_hotels, 'booking_status', perc=True)
```

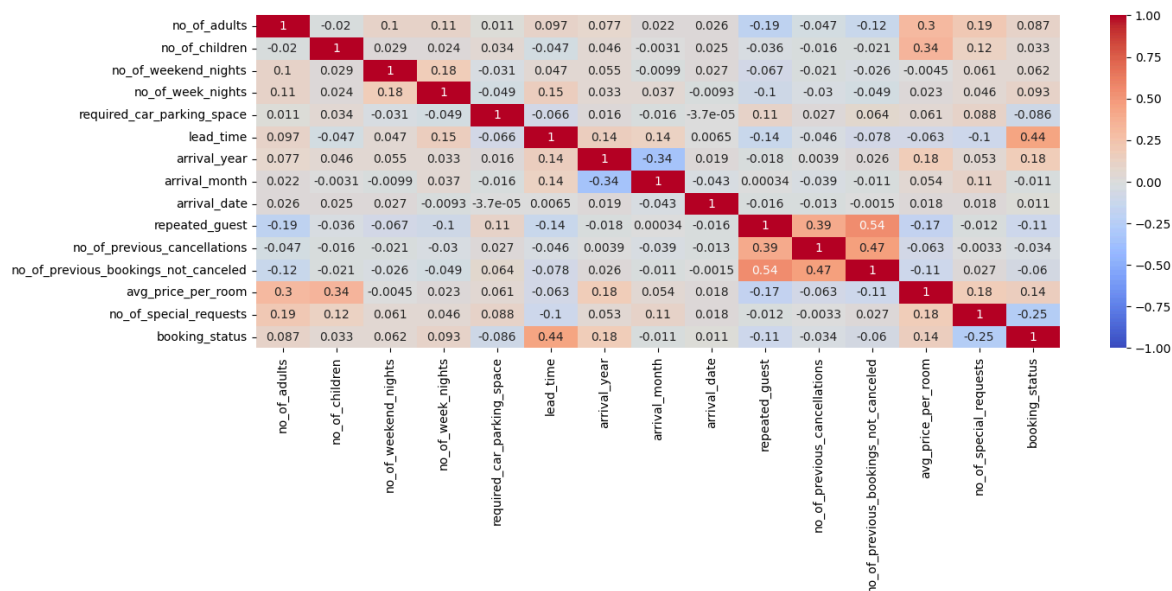


- There is a big chance that 67% of the bookings are not canceled by the customers.
- 33% of the bookings were cancelled by the customers.

Bivariate Analysis

Observation on correlation Matrix on Numerical Variables

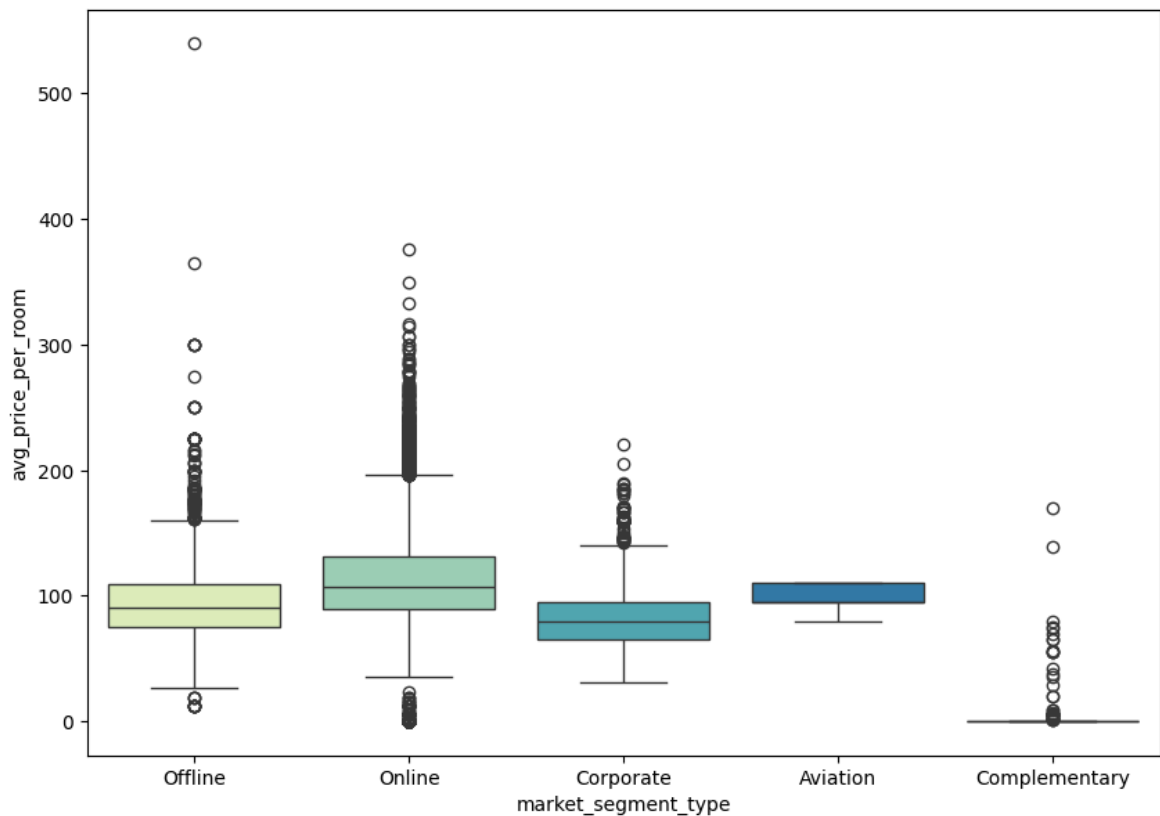
```
In [ ]: plt.figure(figsize=(15,5))
sns.heatmap(inn_hotels.corr(numeric_only=True), annot=True, cmap='coolwarm', vmin=-
plt.show()
```



- There is a moderate correlation between the number of previous bookings not cancelled with repeated guest. It means the customers who previously came to this hotel are satisfied with the facilities of the hotel.
- There's a positive correlation between the number of customers (adults and children) and the average price per room. This makes sense as the number of customers will require more rooms and the cost will increase.
- There is a negative correlation between average room price and repeated guests. The hotel might be giving some loyalty benefits to the customers.
- There is a positive correlation between lead time and booking status. It means higher the lead time, higher are the chances of cancellation. We will analyze it further.
- There's a negative correlation between the number of special requests from the customer and the booking status. Indicating if a customer has made some special requests, the chances of cancellation might decrease. We will analyze it further.

Bivariate Analysis on Average price per room and Market segment type

```
In [ ]: plt.figure(figsize=(10,7))
sns.boxplot(x='market_segment_type',y='avg_price_per_room',data=inn_hotels,palette='magma')
plt.show()
```

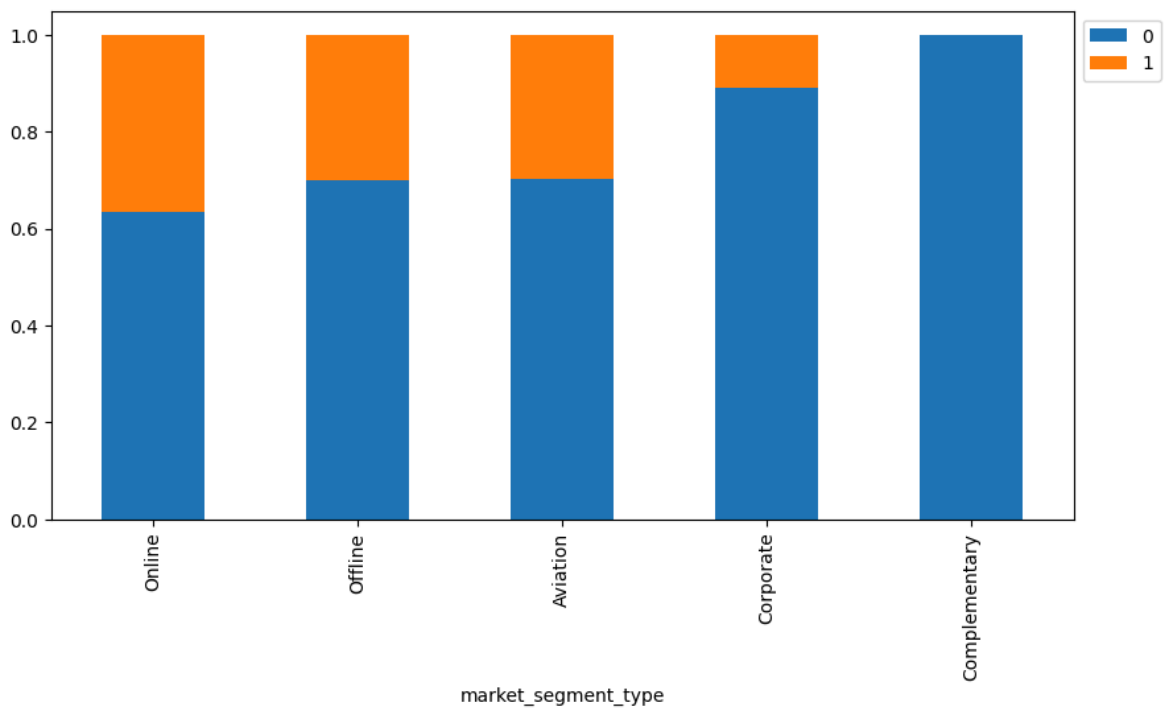


- Rooms booked online have high variation in prices.
- The offline and corporate room prices are almost similar.
- The Complementary market segment gets the rooms at very low prices, which makes sense.

Bivariate Analysis on Booking Status and Market segment type

```
In [ ]: stacked_barplot(inn_hotels, 'market_segment_type', 'booking_status')
```

booking_status	0	1	All
market_segment_type			
All	24390	11885	36275
Online	14739	8475	23214
Offline	7375	3153	10528
Corporate	1797	220	2017
Aviation	88	37	125
Complementary	391	0	391

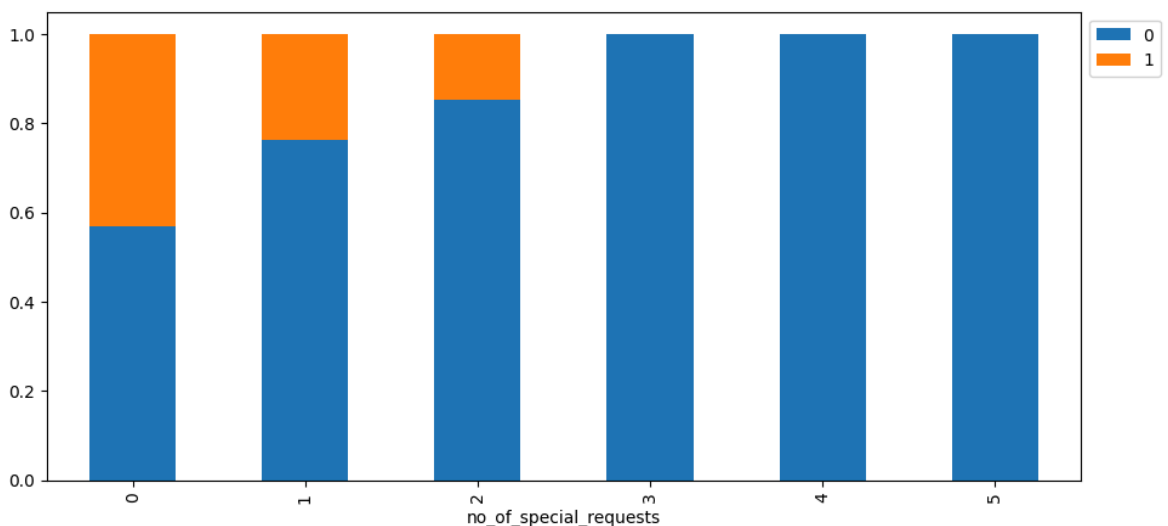


- About 38% of the online bookings were cancelled.
- Bookings made offline have less chances to get cancelled.
- The Corporate segment shows very low cancellations.

Bivariate Analysis on Booking Status and Number of Special Requests

In []: `stacked_barplot(inn_hotels, 'no_of_special_requests', 'booking_status')`

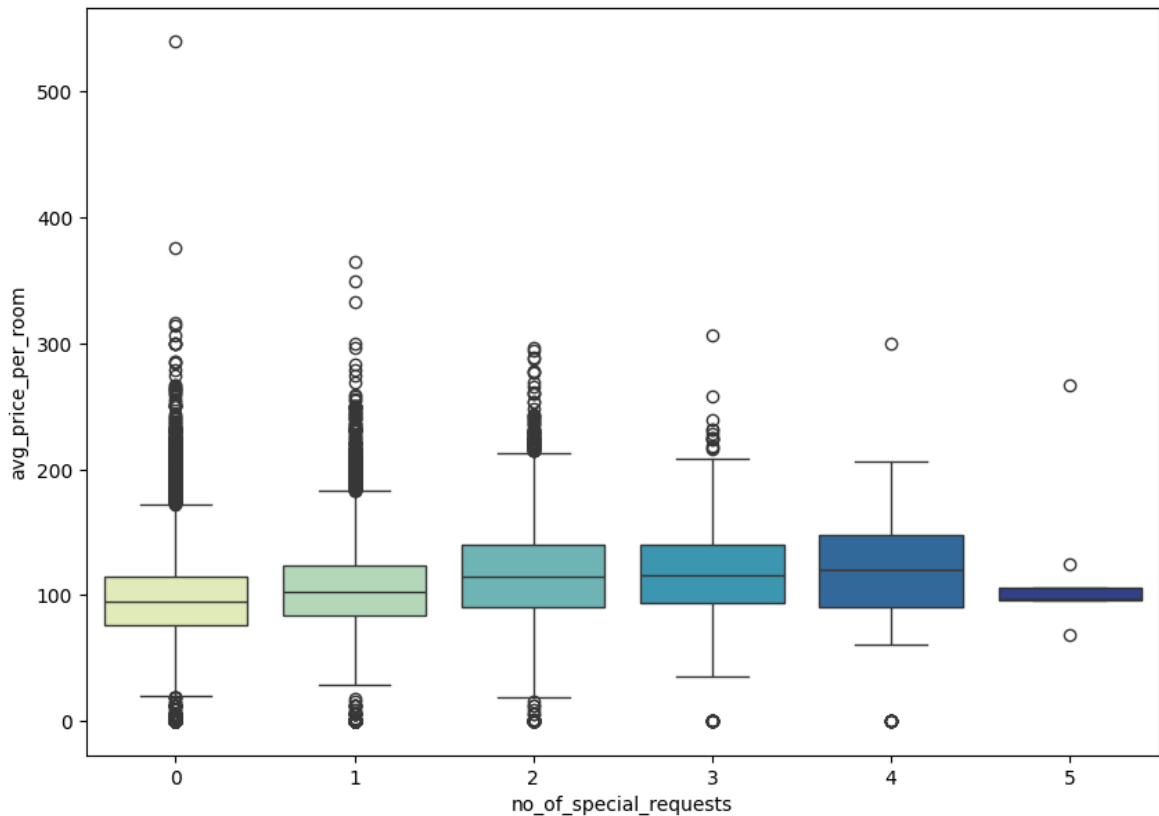
booking_status	0	1	All
no_of_special_requests			
All	24390	11885	36275
0	11232	8545	19777
1	8670	2703	11373
2	3727	637	4364
3	675	0	675
4	78	0	78
5	8	0	8



- If a customer has made more than 2 requests there's a very high chance that the booking will not be cancelled.

Bivariate Analysis on Average Price per room and Number of Special Requests

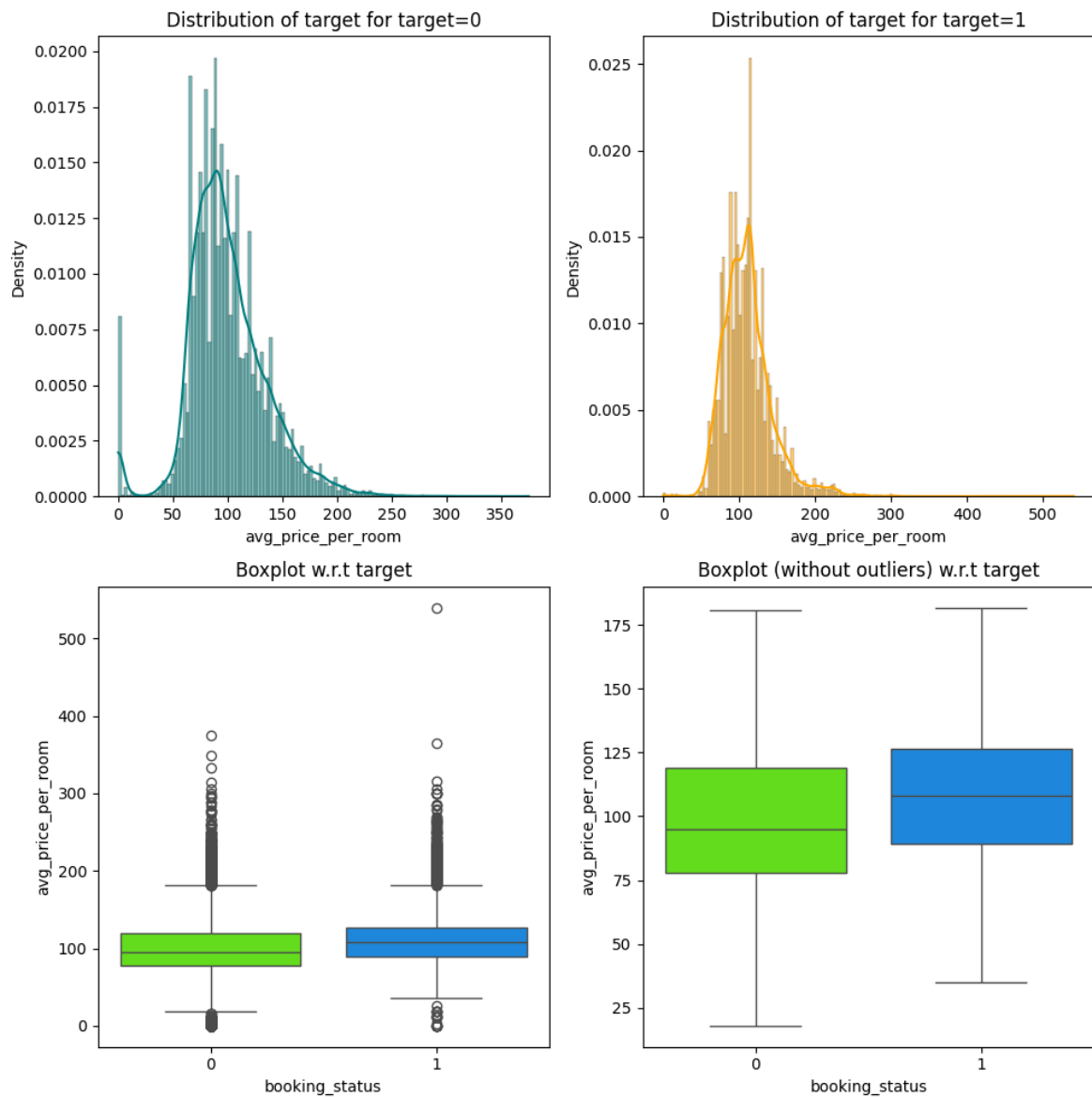
```
In [ ]: plt.figure(figsize=(10,7))
sns.boxplot(x='no_of_special_requests',y='avg_price_per_room',data=inn_hotels,pa
plt.show()
```



- The average price per room where some special requests were made by the customers are slightly higher than the rooms where customers didn't make any requests.

Bivariate Analysis on Average price per room and Booking status

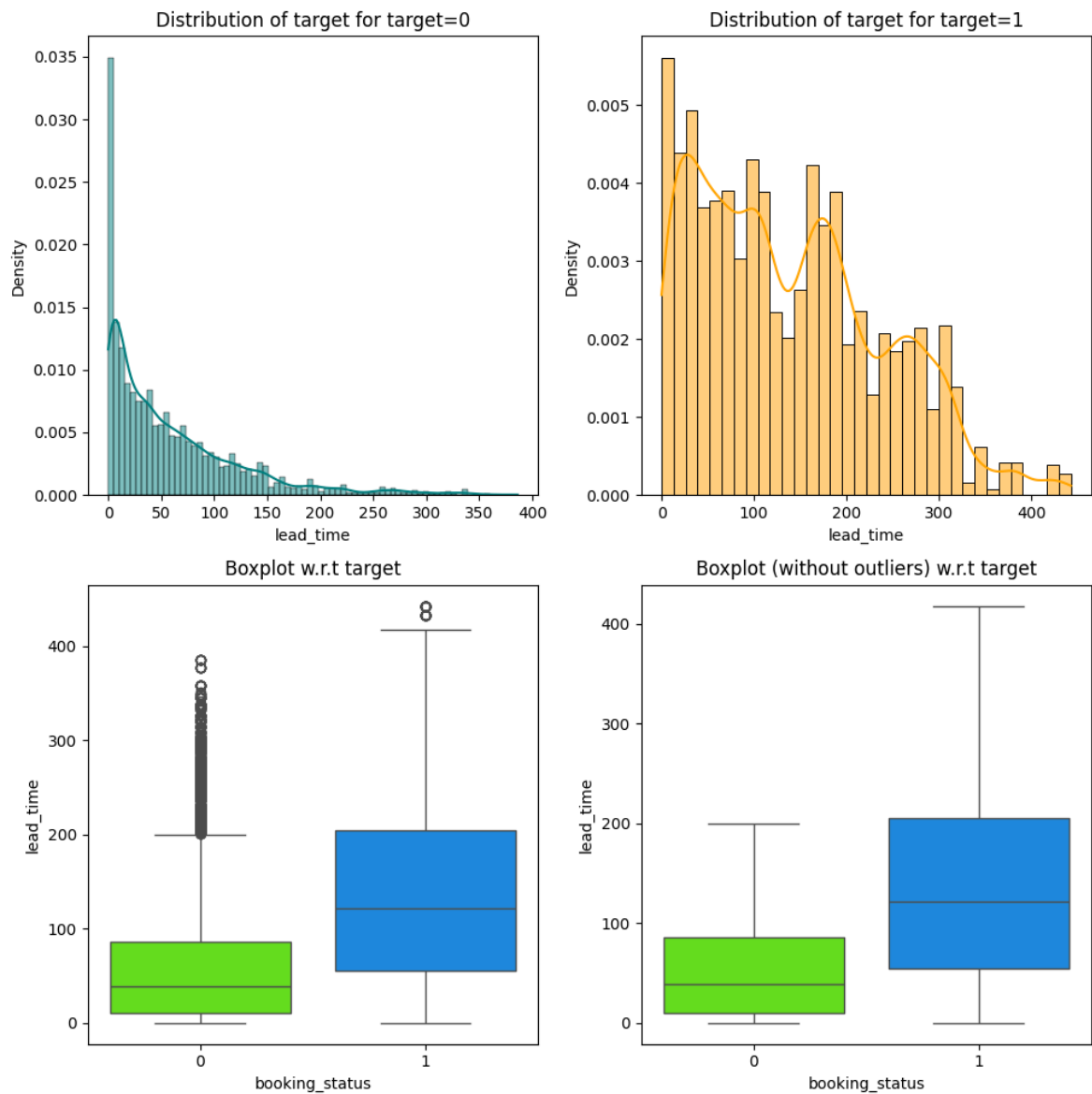
```
In [ ]: distribution_plot_wrt_target(inn_hotels,'avg_price_per_room','booking_status')
```



- Both of the distribution plots(0 and 1) show a right skewed distribution.
- The prices for the cancelled booking(booking_status 1) are slightly higher than the prices of booking which were not cancelled(booking_status 0).

Bivariate Analysis on Lead time and Booking status

```
In [ ]: distribution_plot_wrt_target(inn_hotels,'lead_time','booking_status')
```

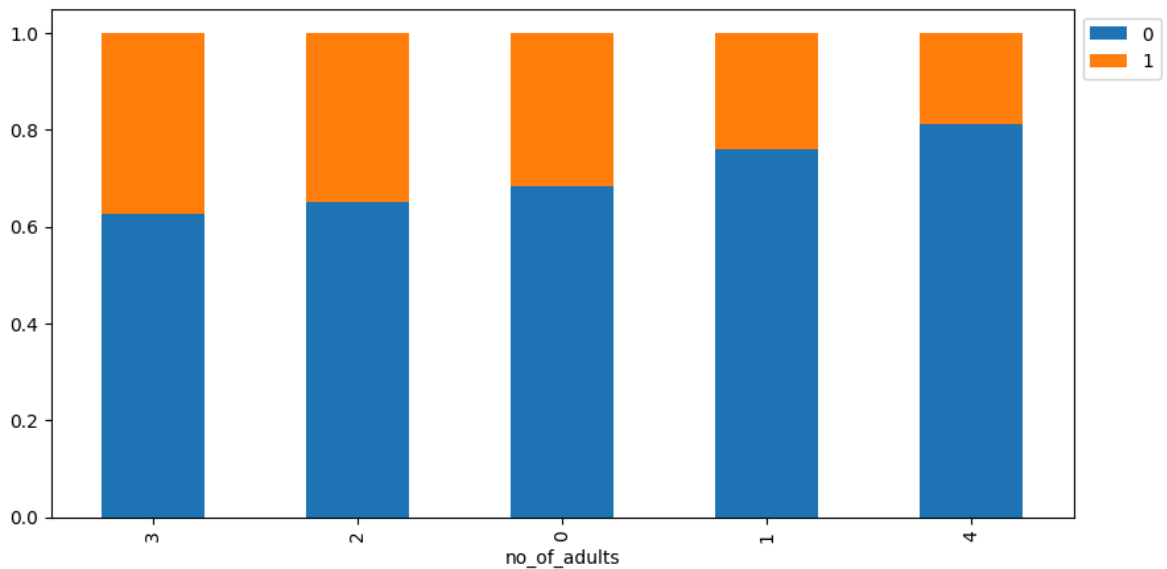


- Both of these distributions(0 and 1) are positively skewed.
- There's a big difference in the median value of lead time for bookings that were cancelled and bookings that were not cancelled. That means higher the lead time, higher are the chances of a booking being cancelled.

Bivariate Analysis on Number of adults and Booking status

```
In [ ]: stacked_barplot(inn_hotels,'no_of_adults','booking_status')
```

booking_status	0	1	All
no_of_adults			
All	24390	11885	36275
2	16989	9119	26108
1	5839	1856	7695
3	1454	863	2317
0	95	44	139
4	13	3	16

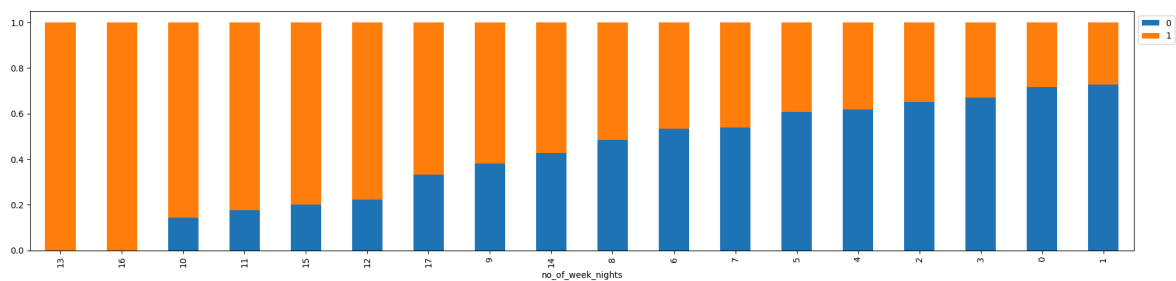


- Almost 40% of the bookings were cancelled by the group of 3 adults.
- Most of the bookings are made by the 2 adults and 36% of the bookings were cancelled.
- There are also the customers most likely teenagers(not adults) booking the hotel and the chances of the bookings getting cancelled is around 30%.
- solo travellers have less tendency to cancel the bookings most of the times.

Bivariate Analysis on Number of week nights and Booking status

```
In [ ]: stacked_barplot(inn_hotels, 'no_of_week_nights', 'booking_status')
```

booking_status	0	1	All
no_of_week_nights			
All	24390	11885	36275
2	7447	3997	11444
3	5265	2574	7839
1	6916	2572	9488
4	1847	1143	2990
0	1708	679	2387
5	982	632	1614
6	101	88	189
10	9	53	62
7	61	52	113
8	30	32	62
9	13	21	34
11	3	14	17
15	2	8	10
12	2	7	9
13	0	5	5
14	3	4	7
16	0	2	2
17	1	2	3

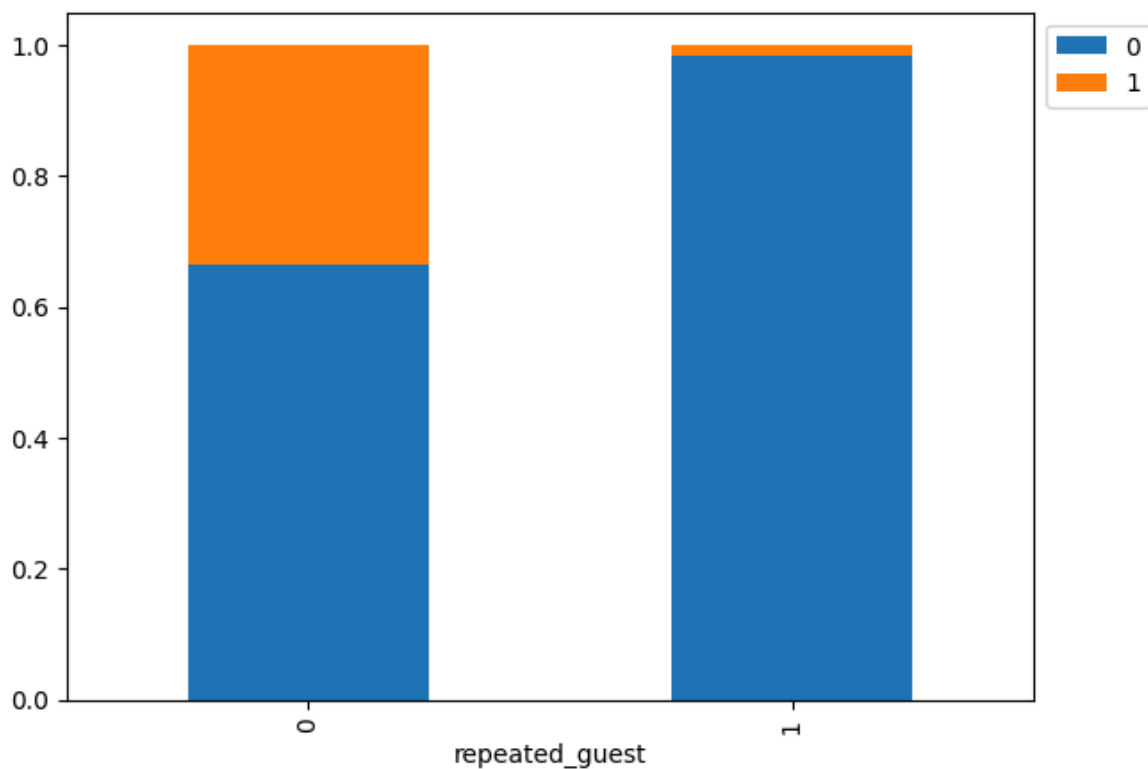


- Customers who booked the hotel rooms for more than 7 nights have high chances to get cancelled.

Bivariate Analysis on Repeated Guests and Booking status

```
In [ ]: stacked_barplot(inn_hotels, 'repeated_guest', 'booking_status')
```

booking_status	0	1	All
repeated_guest			
All	24390	11885	36275
0	23476	11869	35345
1	914	16	930

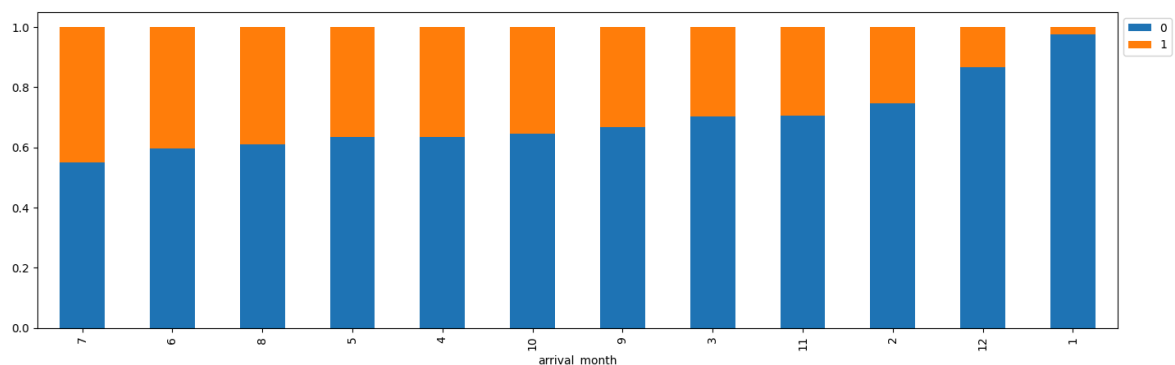


- There are very few repeat customers but the cancellation among them is very less.

Bivariate Analysis on Booking Status and Month

```
In [ ]: stacked_barplot(inn_hotels, 'arrival_month', 'booking_status')
```

booking_status	0	1	All
arrival_month			
All	24390	11885	36275
10	3437	1880	5317
9	3073	1538	4611
8	2325	1488	3813
7	1606	1314	2920
6	1912	1291	3203
4	1741	995	2736
5	1650	948	2598
11	2105	875	2980
3	1658	700	2358
2	1274	430	1704
12	2619	402	3021
1	990	24	1014



- We see that even though the highest number of bookings were made in September and October- around 40% of these bookings got cancelled.
- Least bookings were cancelled in December and January. It means customers might have travelled to celebrate Christmas and New Year.

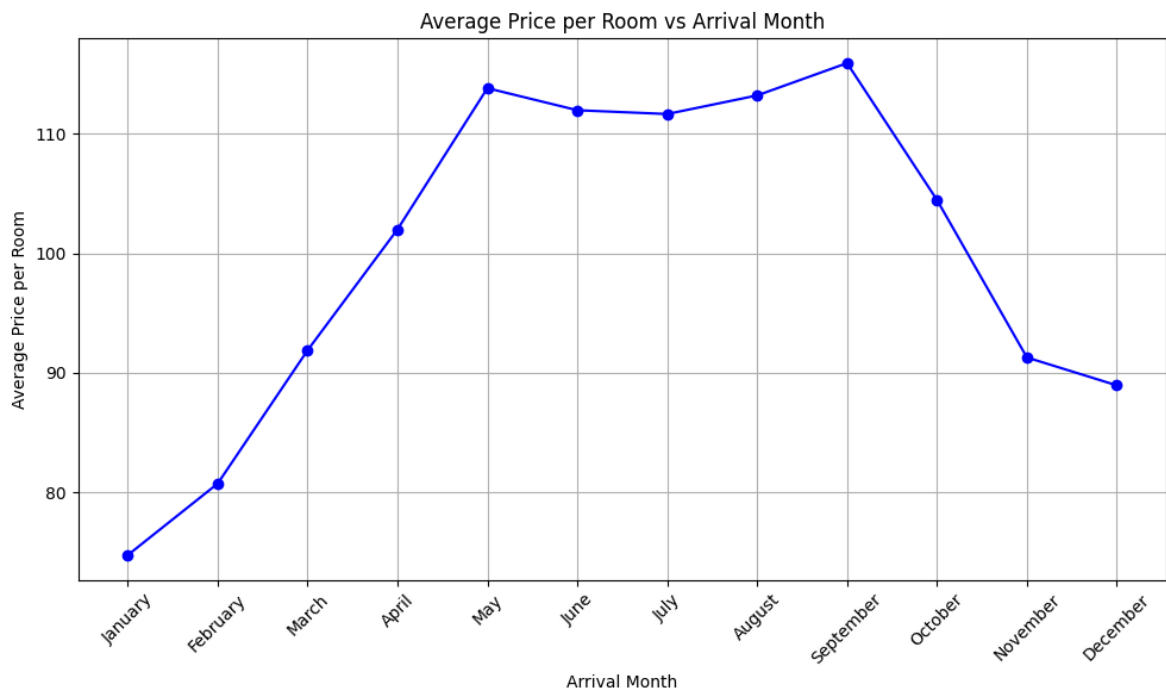
Prices of the Rooms across various months of the year

```
In [ ]: month_order = ['January', 'February', 'March', 'April', 'May', 'June',
                      'July', 'August', 'September', 'October', 'November', 'December']

# Group by arrival month and calculate average price per room
avg_price_by_month = inn_hotels.groupby('arrival_month')['avg_price_per_room'].mean()

# The arrival_month column is numerical (1-12), so reindexing with month names is needed
# We need to map the numerical month to month names or sort by the numerical month
# Let's sort by the numerical month to maintain the chronological order.
avg_price_by_month = inn_hotels.groupby('arrival_month')['avg_price_per_room'].mean()

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(avg_price_by_month.index, avg_price_by_month.values, marker='o', linestyle='solid')
plt.title('Average Price per Room vs Arrival Month')
plt.xlabel('Arrival Month')
plt.ylabel('Average Price per Room')
plt.xticks(list(avg_price_by_month.index), month_order, rotation=45) # Set x-ticks
plt.grid(True)
plt.tight_layout()
plt.show()
```



- The prices of rooms are highest in the month of May to the September month.

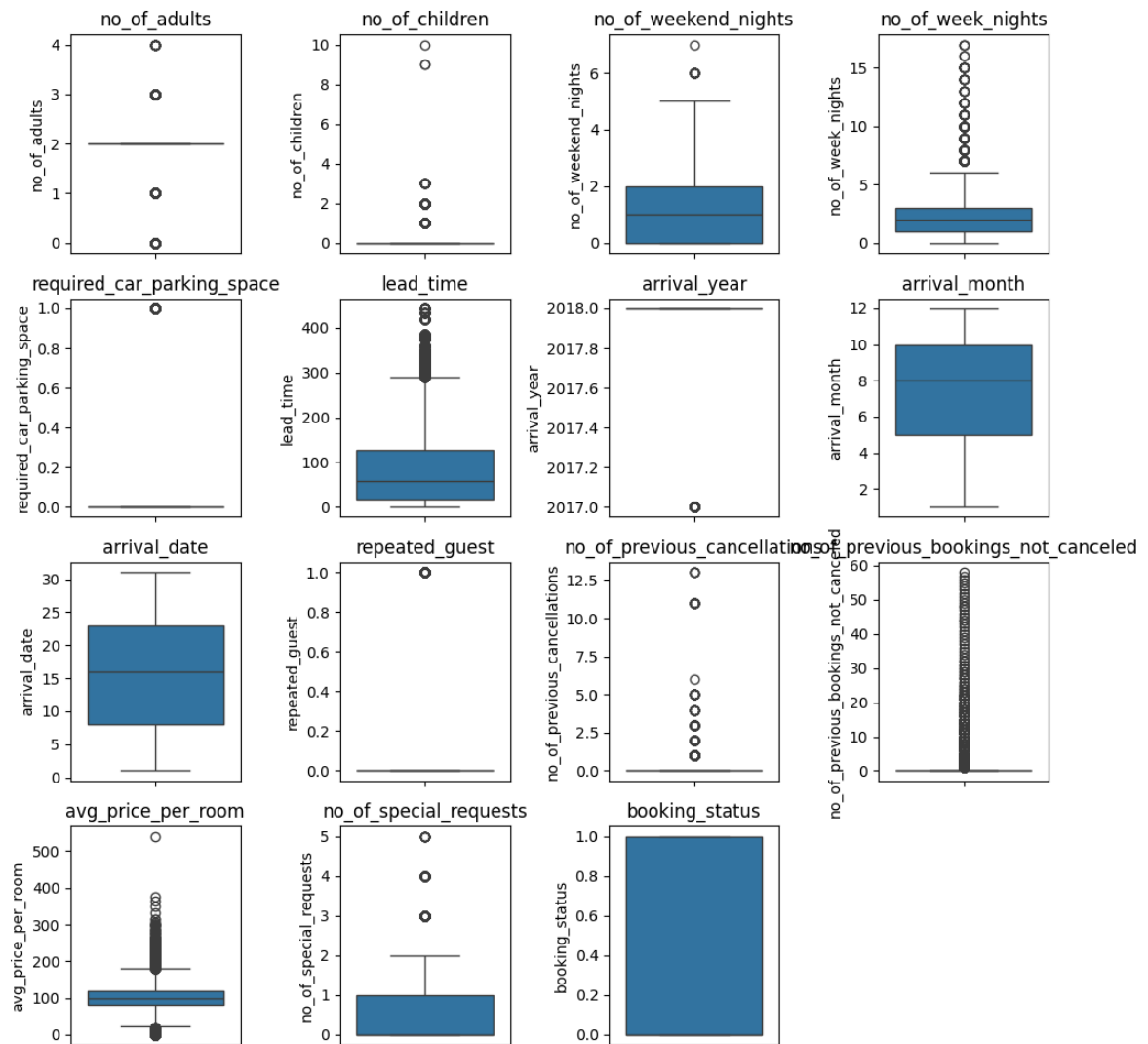
Data Preprocessing

Outlier check

```
In [ ]: numerical_col = inn_hotels.select_dtypes(include=['number']).columns.tolist()
plt.figure(figsize=(10, 10))

for i, variable in enumerate(numerical_col):
    plt.subplot(4, 4, i + 1)
    sns.boxplot(y=inn_hotels[variable], whis=1.5) # Use y for vertical boxplots
    plt.tight_layout()
    plt.title(variable)

plt.show()
```

- Although outliers exist so we will keep them as they may have a valuable input.

```
In [ ]: # Drop arrival date and arrival year
inn_hotels.drop(['arrival_date', 'arrival_year'], axis=1, inplace=True)
```

Data Preparation for Modelling

Creating Training and Test sets

```
In [ ]: X = inn_hotels.drop(["booking_status"], axis=1)
y = inn_hotels["booking_status"]
```

```
In [ ]: # Let's add the intercept to data
X = sm.add_constant(X)
```

```
In [ ]: # encoding the categorical variables
X= pd.get_dummies(X, columns=X.select_dtypes(include=['object', 'category']).columns)

# Converting values to float
X=X.astype(float)
X.head()
```

Out []:

	const	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights	requi
0	1.0	2.0	0.0	1.0	2.0	
1	1.0	2.0	0.0	2.0	3.0	
2	1.0	1.0	0.0	2.0	1.0	
3	1.0	2.0	0.0	0.0	2.0	
4	1.0	2.0	0.0	1.0	1.0	

In []: *#Creating Training and Test sets*
`X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=1)`

In []: `print('Number of Rows in train data',X_train.shape[0])`
`print('Number of Rows in test data',X_test.shape[0])`

Number of Rows in train data 25392
 Number of Rows in test data 10883

In []: `print("Percentage of classes in training set:")`
`print(y_train.value_counts(normalize=True))`
`print("Percentage of classes in test set:")`
`print(y_test.value_counts(normalize=True))`

Percentage of classes in training set:
 booking_status
 0 0.670644
 1 0.329356
 Name: proportion, dtype: float64
 Percentage of classes in test set:
 booking_status
 0 0.676376
 1 0.323624
 Name: proportion, dtype: float64

In []: `import statsmodels.api as sm`
`logit = sm.Logit(y_train, X_train.astype(float))`
`lg = logit.fit(dis= False) # setting disp=False will remove the information on`
`print(lg.summary())`

Logit Regression Results

```

=====
Dep. Variable:          booking_status    No. Observations:          25392
Model:                  Logit             Df Residuals:              25366
Method:                 MLE              Df Model:                  25
Date:                  Sun, 03 Aug 2025   Pseudo R-squ.:            0.3274
Time:                  14:53:41          Log-Likelihood:           -10822.
converged:              False            LL-Null:                  -16091.
Covariance Type:       nonrobust         LLR p-value:              0.000
=====

```

```

=====
                                coef    std err          z      P>|z|
-----
[0.025    0.975]
-----
const                        -2.4279    0.262     -9.251    0.000
-2.942    -1.913
no_of_adults                   0.1012    0.038      2.695    0.007
0.028     0.175
no_of_children                 0.1631    0.057      2.870    0.004
0.052     0.274
no_of_weekend_nights           0.1083    0.020      5.487    0.000
0.070     0.147
no_of_week_nights              0.0405    0.012      3.296    0.001
0.016     0.065
required_car_parking_space     -1.6213    0.138    -11.781    0.000
-1.891    -1.352
lead_time                      0.0163    0.000     63.035    0.000
0.016     0.017
arrival_month                  -0.0615    0.006    -10.280    0.000
-0.073    -0.050
repeated_guest                 -2.3702    0.609     -3.891    0.000
-3.564    -1.176
no_of_previous_cancellations    0.2665    0.086      3.105    0.002
0.098     0.435
no_of_previous_bookings_not_canceled -0.1624    0.147     -1.108    0.268
-0.450     0.125
avg_price_per_room             0.0201    0.001     28.015    0.000
0.019     0.022
no_of_special_requests         -1.4549    0.030    -48.654    0.000
-1.513    -1.396
type_of_meal_plan_Meal Plan 2    0.0471    0.064      0.735    0.462
-0.078     0.173
type_of_meal_plan_Meal Plan 3   18.6582   9200.802      0.002    0.998
-1.8e+04   1.81e+04
type_of_meal_plan_Not Selected    0.3442    0.052      6.572    0.000
0.242     0.447
room_type_reserved_Room_Type 2   -0.3747    0.130     -2.879    0.004
-0.630    -0.120
room_type_reserved_Room_Type 3    0.0494    1.299      0.038    0.970
-2.496     2.594
room_type_reserved_Room_Type 4   -0.2598    0.053     -4.901    0.000
-0.364    -0.156
room_type_reserved_Room_Type 5   -0.6819    0.210     -3.253    0.001
-1.093    -0.271
room_type_reserved_Room_Type 6   -1.0238    0.146     -7.009    0.000
-1.310    -0.737
room_type_reserved_Room_Type 7   -1.4600    0.294     -4.968    0.000
-2.036    -0.884
market_segment_type_Complementary -25.6305   9202.371     -0.003    0.998

```

-1.81e+04	1.8e+04				
market_segment_type_Corporate		-1.3068	0.266	-4.906	0.000
-1.829	-0.785				
market_segment_type_Offline		-2.3268	0.255	-9.127	0.000
-2.826	-1.827				
market_segment_type_Online		-0.4955	0.252	-1.968	0.049
-0.989	-0.002				
=====					
=====					

- Negative values of the coefficient show that the probability of customers cancelling the booking decreases with the increase of the corresponding attribute value.
- Positive values of the coefficient show that the probability of customer cancelling increases with the increase of corresponding attribute value.

Model evaluation criterion

Model can make wrong predictions as:

1. Predicting a customer will not cancel their booking but in reality, the customer will cancel their booking.
2. Predicting a customer will cancel their booking but in reality, the customer will not cancel their booking.

Which case is more important?

- Both the cases are important as:
- If we predict that a booking will not be cancelled and the bookings get cancelled then the hotel will lose resources and will have to bear additional costs which means the hotel will have to face revenue loss.
- If we predict that a booking will get cancelled and the booking doesn't get cancelled the hotel might not be able to provide satisfactory services to the customers. This might effect on the ratings of the hotel.

How to reduce the losses?

- We need to F1 score to be maximized, greater the F1 score higher are the chances of minimizing False Negatives and False Positives.
- f1_score is computed as:

$$f1_score = (2 * Precision * Recall) / (Precision + Recall)$$

```
In [ ]: # defining a function to compute different metrics to check performance of a cla
def model_performance_classification_statsmodels(
    model, predictors, target, threshold=0.5
):
    """
    Function to compute different metrics to check classification model performa
```

```

model: classifier
predictors: independent variables
target: dependent variable
threshold: threshold for classifying the observation as class 1
"""

# checking which probabilities are greater than threshold
pred_temp = model.predict(predictors) > threshold
# rounding off the above values to get classes
pred = np.round(pred_temp)

acc = accuracy_score(target, pred) # to compute Accuracy
recall = recall_score(target, pred) # to compute Recall
precision = precision_score(target, pred) # to compute Precision
f1 = f1_score(target, pred) # to compute F1-score

# creating a dataframe of metrics
df_perf = pd.DataFrame(
    {"Accuracy": acc, "Recall": recall, "Precision": precision, "F1": f1},
    index=[0],
)

return df_perf

```

In []: # defining a function to plot the confusion_matrix of a classification model

```

def confusion_matrix_statsmodels(model, predictors, target, threshold=0.5):
    """
    To plot the confusion_matrix with percentages

    model: classifier
    predictors: independent variables
    target: dependent variable
    threshold: threshold for classifying the observation as class 1
    """
    y_pred = model.predict(predictors) > threshold
    cm = confusion_matrix(target, y_pred)
    labels = np.asarray(
        [
            "{0:0.0f}".format(item) + "\n{0:.2%}".format(item / cm.flatten().sum())
            for item in cm.flatten()
        ]
    ).reshape(2, 2)

    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=labels, fmt="")
    plt.ylabel("True label")
    plt.xlabel("Predicted label")

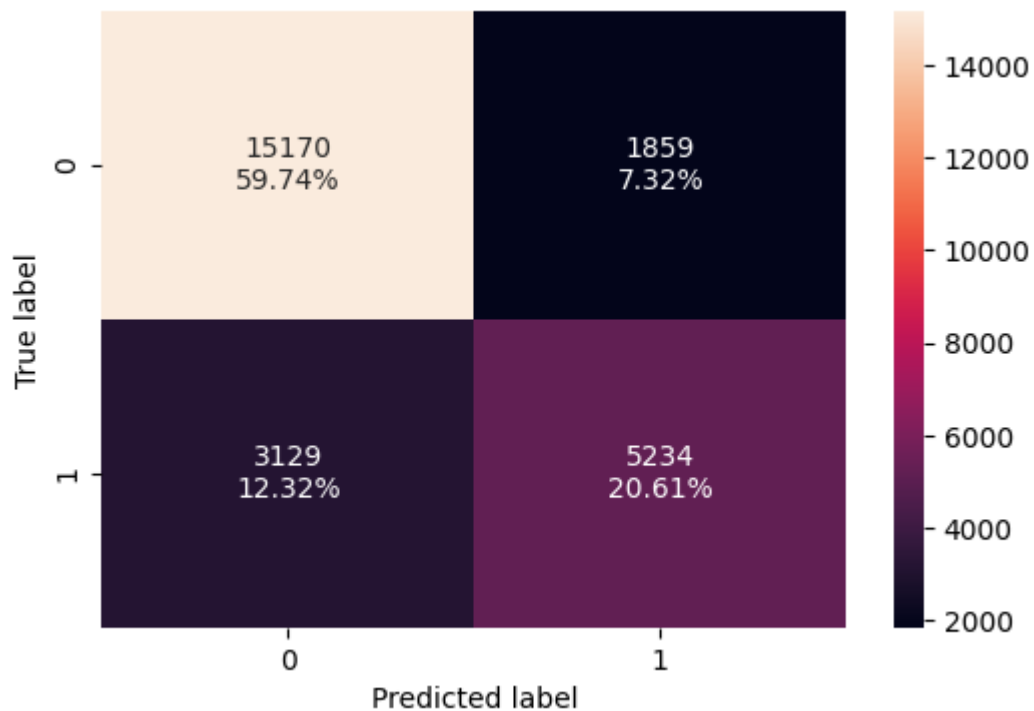
```

In []: print("Training performance:")
 model_performance_classification_statsmodels(lg, X_train, y_train)

Training performance:

Out []:	Accuracy	Recall	Precision	F1
0	0.80356	0.625852	0.737911	0.677277

```
In [ ]: confusion_matrix_statsmodels(lg, X_train, y_train)
```



- The F1 score of the model is ~ 0.68 and we will try to maximize it further.
- The variables used to build the model might contain multicollinearity, which will affect the p-values.
- We will have to remove multicollinearity from the data to get reliable coefficients and p-values.

Detecting and Dealing with Multicollinearity

```
In [ ]: from statsmodels.stats.outliers_influence import variance_inflation_factor
```

```
def checking_vif(predictors):
    vif = pd.DataFrame()
    vif["feature"] = predictors.columns

    # calculating VIF for each feature
    vif["VIF"] = [
        variance_inflation_factor(predictors.values, i)
        for i in range(len(predictors.columns))
    ]
    vif["VIF"] = vif["VIF"].round(2)
    return vif
```

```
In [ ]: checking_vif(X_train)
```

Out[]:

	feature	VIF
0	const	326.13
1	no_of_adults	1.35
2	no_of_children	1.98
3	no_of_weekend_nights	1.07
4	no_of_week_nights	1.10
5	required_car_parking_space	1.04
6	lead_time	1.25
7	arrival_month	1.05
8	repeated_guest	1.78
9	no_of_previous_cancellations	1.39
10	no_of_previous_bookings_not_canceled	1.65
11	avg_price_per_room	1.94
12	no_of_special_requests	1.25
13	type_of_meal_plan_Meal Plan 2	1.20
14	type_of_meal_plan_Meal Plan 3	1.03
15	type_of_meal_plan_Not Selected	1.24
16	room_type_reserved_Room_Type 2	1.10
17	room_type_reserved_Room_Type 3	1.00
18	room_type_reserved_Room_Type 4	1.36
19	room_type_reserved_Room_Type 5	1.03
20	room_type_reserved_Room_Type 6	1.96
21	room_type_reserved_Room_Type 7	1.11
22	market_segment_type_Complementary	4.50
23	market_segment_type_Corporate	16.90
24	market_segment_type_Offline	63.95
25	market_segment_type_Online	71.05

- None of the numerical variables show moderate or high multicollinearity.
- We will ignore the VIF for the dummy variables.

Dealing with high p-values

- Build a model, check the P-values of the variables, and drop the column with the highest p-value.

- Create a new model without the dropped feature, check the p-values of the variables, and drop the column with the highest p-value.
- Repeat the above two steps till there are no columns with p-value > 0.05.

Note: The above process can also be done manually by picking one variable at a time that has a high p-value, dropping it, and building a model again. But that might be a little tedious and using a loop will be more efficient.

```
In [ ]: # initial list of columns
cols = X_train.columns.tolist()

# setting an initial max p-value
max_p_value = 1

while len(cols) > 0:
    # defining the train set
    X_train_aux = X_train[cols]

    # fitting the model
    model = sm.Logit(y_train, X_train_aux).fit(dis=False)

    # getting the p-values and the maximum p-value
    p_values = model.pvalues
    max_p_value = max(p_values)

    # name of the variable with maximum p-value
    feature_with_p_max = p_values.idxmax()

    if max_p_value > 0.05:
        cols.remove(feature_with_p_max)
    else:
        break

selected_features = cols
print(selected_features)
```

```
['const', 'no_of_adults', 'no_of_children', 'no_of_weekend_nights', 'no_of_week_nights', 'required_car_parking_space', 'lead_time', 'arrival_month', 'repeated_guest', 'no_of_previous_cancellations', 'avg_price_per_room', 'no_of_special_requests', 'type_of_meal_plan_Not Selected', 'room_type_reserved_Room_Type 2', 'room_type_reserved_Room_Type 4', 'room_type_reserved_Room_Type 5', 'room_type_reserved_Room_Type 6', 'room_type_reserved_Room_Type 7', 'market_segment_type_Corporate', 'market_segment_type_Offline']
```

```
In [ ]: X_train2 = X_train[selected_features]
```

```
In [ ]: logit2 = sm.Logit(y_train, X_train2.astype(float))
lg2 = logit2.fit(dis=False)

print(lg2.summary())
```


Logit Regression Results

```

=====
Dep. Variable:          booking_status    No. Observations:          25392
Model:                  Logit             Df Residuals:              25372
Method:                 MLE              Df Model:                 19
Date:                  Sun, 03 Aug 2025    Pseudo R-squ.:            0.3264
Time:                  14:53:45           Log-Likelihood:           -10838.
converged:              True              LL-Null:                  -16091.
Covariance Type:        nonrobust         LLR p-value:              0.000
=====

```

```

=====
                                coef      std err          z      P>|z|
-----
[0.025      0.975]
-----
const                        -2.9597      0.107     -27.589      0.000      -
3.170      -2.749
no_of_adults                  0.0955      0.037      2.565      0.010
0.023      0.168
no_of_children                0.1581      0.057      2.783      0.005
0.047      0.269
no_of_weekend_nights          0.1103      0.020      5.594      0.000
0.072      0.149
no_of_week_nights             0.0423      0.012      3.453      0.001
0.018      0.066
required_car_parking_space    -1.6212      0.138     -11.777      0.000      -
1.891      -1.351
lead_time                     0.0163      0.000      63.684      0.000
0.016      0.017
arrival_month                 -0.0621      0.006     -10.394      0.000      -
0.074      -0.050
repeated_guest                -2.7033      0.559      -4.833      0.000      -
3.800      -1.607
no_of_previous_cancellations  0.2270      0.077      2.958      0.003
0.077      0.377
avg_price_per_room            0.0206      0.001      29.717      0.000
0.019      0.022
no_of_special_requests        -1.4562      0.030     -48.762      0.000      -
1.515      -1.398
type_of_meal_plan_Not Selected 0.3491      0.052      6.681      0.000
0.247      0.451
room_type_reserved_Room_Type 2 -0.3718      0.130      -2.859      0.004      -
0.627      -0.117
room_type_reserved_Room_Type 4 -0.2619      0.053      -4.975      0.000      -
0.365      -0.159
room_type_reserved_Room_Type 5 -0.7030      0.209      -3.368      0.001      -
1.112      -0.294
room_type_reserved_Room_Type 6 -1.0457      0.146      -7.180      0.000      -
1.331      -0.760
room_type_reserved_Room_Type 7 -1.5016      0.293      -5.125      0.000      -
2.076      -0.927
market_segment_type_Corporate -0.8119      0.103      -7.904      0.000      -
1.013      -0.611
market_segment_type_Offline    -1.8143      0.050     -36.177      0.000      -
1.913      -1.716
=====
=====

```

```

In [ ]: print("Training performance:")
        model_performance_classification_statsmodels(lg2, X_train2, y_train)

```

Training performance:

```
Out[ ]:

```

	Accuracy	Recall	Precision	F1
0	0.804033	0.626211	0.738959	0.677929

Training Performances after treatment of multicollinearity

- All the variables in the model have p-values < 0.05.
- So, we can say that this model (lg2) is the best model for making any inference.
- The performance on the training data is the same as before dropping the variables with the high p-value.

Converting coefficients to odds

- The coefficients of the logistic regression model are in terms of log(odd), to find the odds we have to take the exponential of the coefficients.
- Therefore, $\text{odds} = \exp(b)$ The percentage change in odds is given as $\text{odds} = (\exp(b) - 1) * 100$

```
In [ ]: # converting coefficients to odds
odds = np.exp(lg2.params)

# finding the percentage change
perc_change_odds = (np.exp(lg2.params) - 1) * 100

# removing limit from number of columns to display
pd.set_option("display.max_columns", None)

# adding the odds to a dataframe
pd.DataFrame({"Odds": odds, "Change_odd%": perc_change_odds}, index=X_train2.col
```

Out[]:

	Odds	Change_odd%
const	0.051833	-94.816709
no_of_adults	1.100186	10.018618
no_of_children	1.171286	17.128569
no_of_weekend_nights	1.116603	11.660334
no_of_week_nights	1.043191	4.319063
required_car_parking_space	0.197652	-80.234750
lead_time	1.016412	1.641159
arrival_month	0.939774	-6.022621
repeated_guest	0.066984	-93.301633
no_of_previous_cancellations	1.254807	25.480650
avg_price_per_room	1.020782	2.078193
no_of_special_requests	0.233125	-76.687458
type_of_meal_plan_Not Selected	1.417735	41.773497
room_type_reserved_Room_Type 2	0.689492	-31.050776
room_type_reserved_Room_Type 4	0.769581	-23.041857
room_type_reserved_Room_Type 5	0.495115	-50.488479
room_type_reserved_Room_Type 6	0.351460	-64.854036
room_type_reserved_Room_Type 7	0.222763	-77.723691
market_segment_type_Corporate	0.444008	-55.599205
market_segment_type_Offline	0.162953	-83.704741

Coefficient interpretations

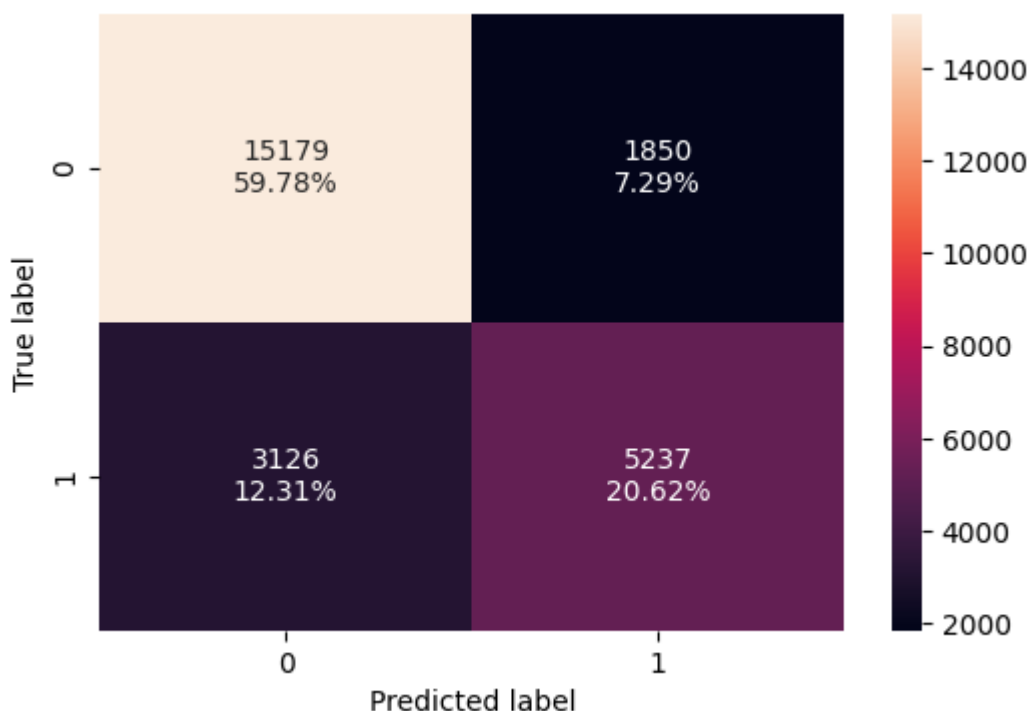
- **no_of_adults:** Holding all other features constant 1 unit change in number of adults increases the odds of booking getting cancelled by 1.10 times or a 10% increase of a booking getting cancelled.
- **no_of_children:** Holding all other features constant 1 unit change in number of children increase the odds of booking getting cancelled by 1.17 times or a 17% increase of a booking getting cancelled.
- **no_of_weekend_nights:** Holding all other features constant 1 unit change in number of weekend nights increase the odds of booking getting cancelled by 1.11 times or a 12% increase of a booking getting cancelled.
- **no_of_week_nights:** Holding all other features constant 1 unit change in number of week nights increases the odds of booking getting cancelled by 1.04 times or a 4.31% increase of a booking getting cancelled.
- **required_car_parking_space:** The odds of a customers who requires a car parking space are 0.19 times less than a customer who doesn't require car_parking_space or

a 80.23% fewer odds of a customer.

- **lead_time**: Holding all other features constant 1 unit change in number of lead_time increase the odds of booking getting cancelled by 1.01 times or a 1.64% increase of a booking getting cancelled.
- **avg_price_per_room**: Holding all other features constant 1 unit change in number of lead_time increase the odds of booking getting cancelled by 1.02 times or a 2.07% increase of a booking getting cancelled.
- **no_of_special_requests**: Holding all other features constant 1 unit change in number of lead_time increase the odds of booking getting cancelled by 0.23 times or a 77% decrease of a booking getting cancelled.

checking the model performance on the training set

```
In [ ]: confusion_matrix_statsmodels(lg2, X_train2, y_train)
```



```
In [ ]: log_reg_model_train_perf=model_performance_classification_statsmodels(lg2, X_train2, y_train)
print("Training performance:")
log_reg_model_train_perf
```

Training performance:

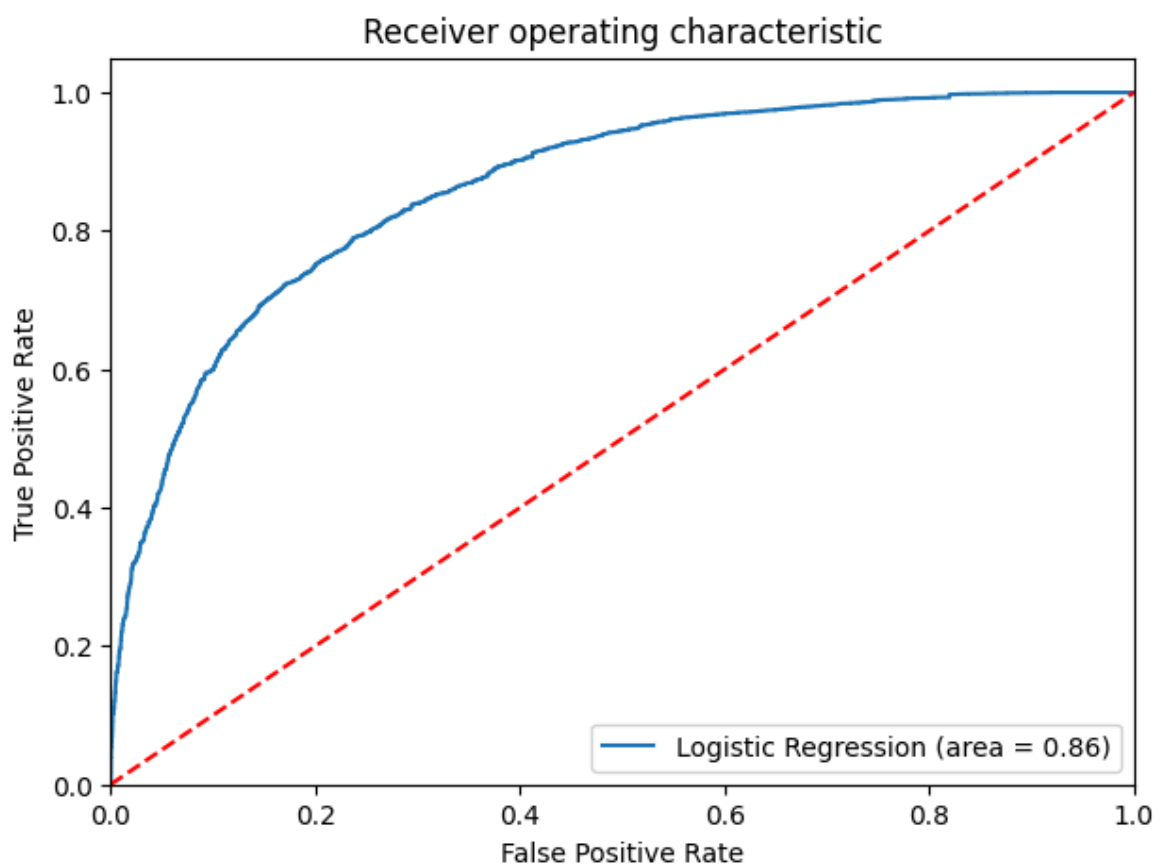
```
Out[ ]: Accuracy  Recall  Precision  F1
0 0.804033 0.626211 0.738959 0.677929
```

ROC-AUC

- ROC-AUC on training set

```
In [ ]: # To get diferent metric scores
from sklearn import metrics
from sklearn.metrics import accuracy_score, roc_curve, confusion_matrix, roc_auc

logit_roc_auc_train = roc_auc_score(y_train, lg2.predict(X_train2))
fpr, tpr, thresholds = roc_curve(y_train, lg2.predict(X_train2))
plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, label="Logistic Regression (area = %0.2f)" % logit_roc_auc_train)
plt.plot([0, 1], [0, 1], "r--")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver operating characteristic")
plt.legend(loc="lower right")
plt.show()
```



- Logistic Regression model is giving a generalized performance on training and test set.
- ROC-AUC score of 0.86 on training is quite good.

Model Performance Improvement

- Let's see if the recall score can be improved further, by changing the model threshold using AUC-ROC Curve.

Optimal threshold using AUC-ROC Curve

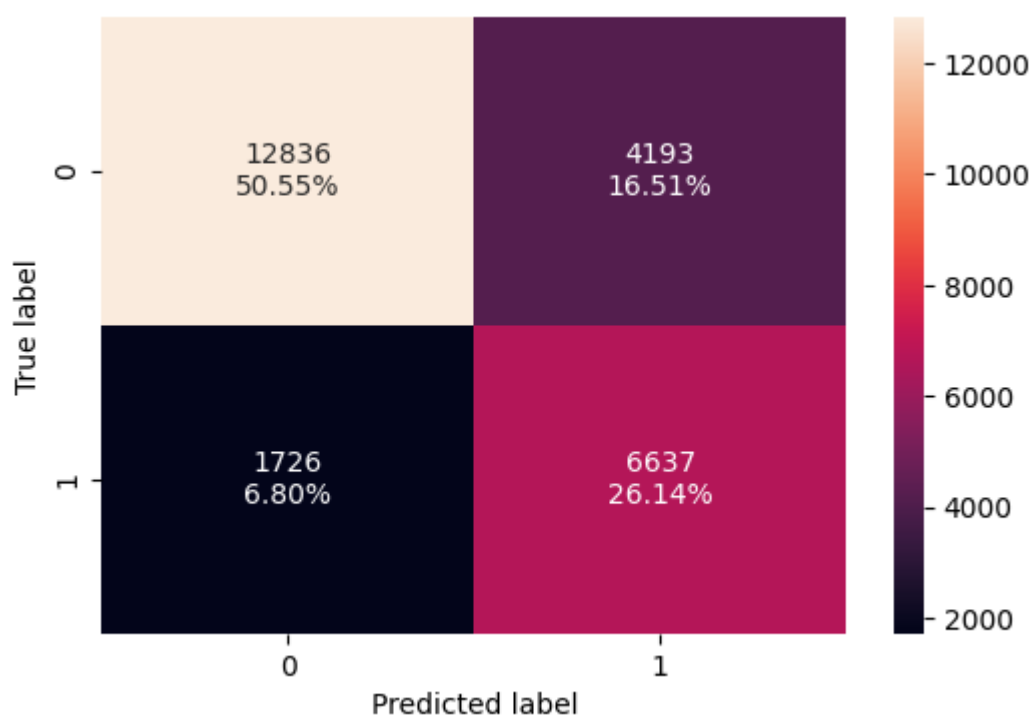
```
In [ ]: # Optimal threshold as per AUC-ROC curve
# The optimal cut off would be where tpr is high and fpr is low
fpr, tpr, thresholds = roc_curve(y_train, lg2.predict(X_train2))

optimal_idx = np.argmax(tpr - fpr)
optimal_threshold_auc_roc = thresholds[optimal_idx]
print(optimal_threshold_auc_roc)
```

0.30893774503410887

Checking model performance in training set

```
In [ ]: optimal_threshold_auc_roc = 0.30
# creating confusion matrix
confusion_matrix_statsmodels(
    lg2, X_train2, y_train, threshold=optimal_threshold_auc_roc
)
```



```
In [ ]: # checking model performance for this model
log_reg_model_train_perf_threshold_auc_roc = model_performance_classification_st
    lg2, X_train2, y_train, threshold=optimal_threshold_auc_roc
)
print("Training performance:")
log_reg_model_train_perf_threshold_auc_roc
```

Training performance:

```
Out[ ]:      Accuracy      Recall   Precision      F1
0   0.766895   0.793615   0.612835   0.691606
```

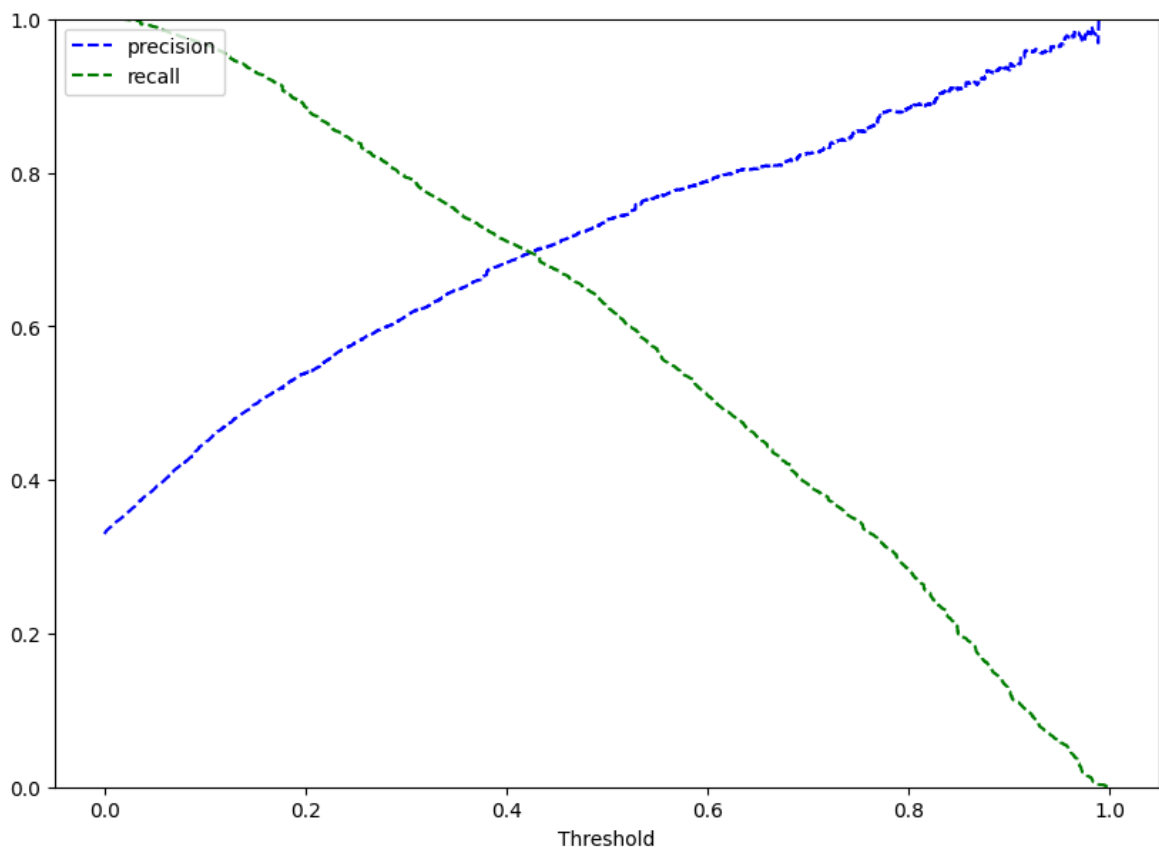
- Recall has increased significantly as compared to the previous model.
- As we decrease the threshold value, Recall will keep on increasing and precision will decrease, but this is not right, we need to choose an optimal balance between recall and precision.

Using Precision-Recall curve to find a better threshold

```
In [ ]: y_scores = lg2.predict(X_train2)
prec, rec, tre = precision_recall_curve(y_train, y_scores,)

def plot_prec_recall_vs_tresh(precisions, recalls, thresholds):
    plt.plot(thresholds, precisions[:-1], "b--", label="precision")
    plt.plot(thresholds, recalls[:-1], "g--", label="recall")
    plt.xlabel("Threshold")
    plt.legend(loc="upper left")
    plt.ylim([0, 1])

plt.figure(figsize=(10, 7))
plot_prec_recall_vs_tresh(prec, rec, tre)
plt.show()
```

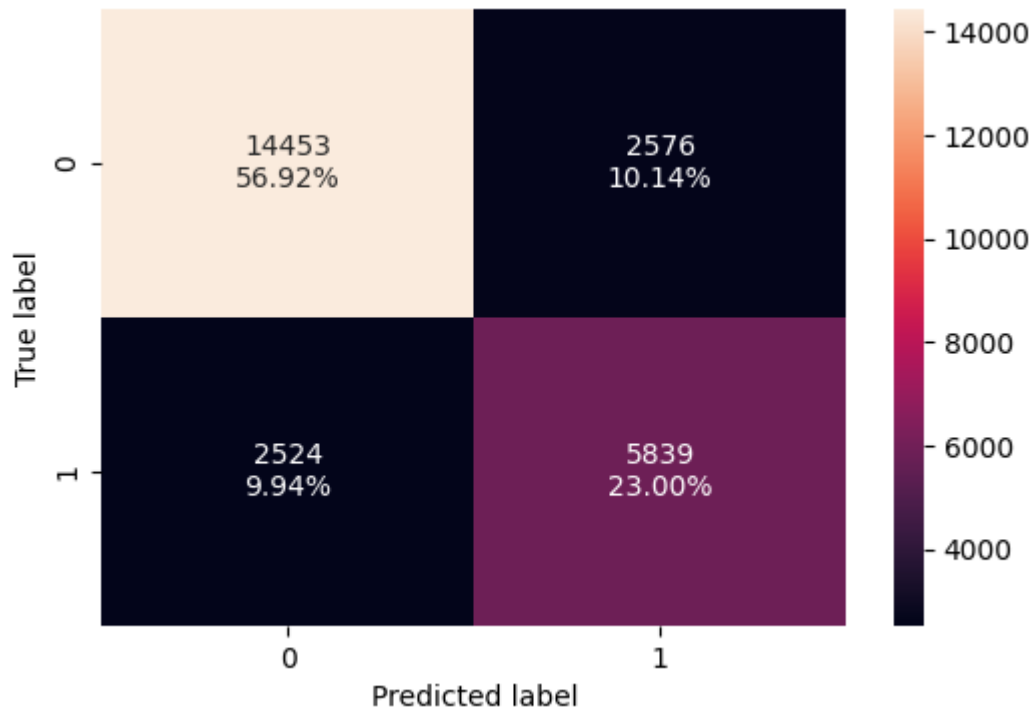


- We get a balanced precision-recall at 0.42 threshold.

Checking model performance on training set- with 0.42 as Threshold

```
In [ ]: optimal_threshold_curve = 0.42

# creating confusion matrix
confusion_matrix_statsmodels(lg2, X_train2, y_train, threshold=optimal_threshold)
```



```
In [ ]: # checking model performance for this model
log_reg_model_train_perf_threshold_curve = model_performance_classification_stat
lg2, X_train2, y_train, threshold=optimal_threshold_curve
)
print("Training performance:")
log_reg_model_train_perf_threshold_curve
```

Training performance:

```
Out[ ]:   Accuracy   Recall   Precision   F1
0  0.799149  0.698194   0.69388   0.696031
```

Model Performance Summary

```
In [ ]: # training performance comparison

models_train_comp_df = pd.concat(
    [
        log_reg_model_train_perf.T,
        log_reg_model_train_perf_threshold_auc_roc.T,
        log_reg_model_train_perf_threshold_curve.T,
    ],
    axis=1,
)
models_train_comp_df.columns = [
    "Logistic Regression statsmodel",
    "Logistic Regression-0.30 Threshold",
    "Logistic Regression-0.42 Threshold",
]

print("Training performance comparison:")
models_train_comp_df
```

Training performance comparison:

Out[]:

	Logistic Regression statsmodel	Logistic Regression-0.30 Threshold	Logistic Regression-0.42 Threshold
Accuracy	0.804033	0.766895	0.799149
Recall	0.626211	0.793615	0.698194
Precision	0.738959	0.612835	0.693880
F1	0.677929	0.691606	0.696031

Let's check the performance on the test set

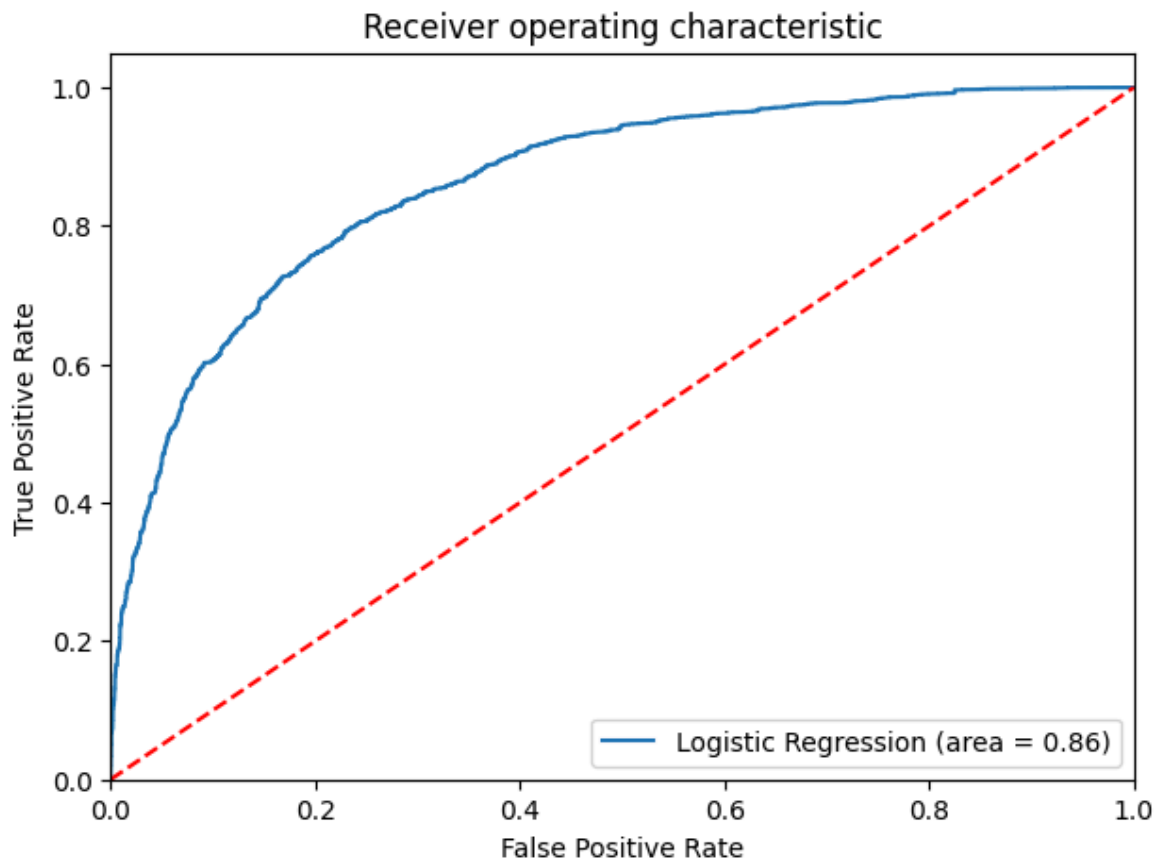
Dropping the columns from the test set that were dropped from the training set

```
In [ ]: X_test2=X_test[X_train2.columns].astype(float)
```

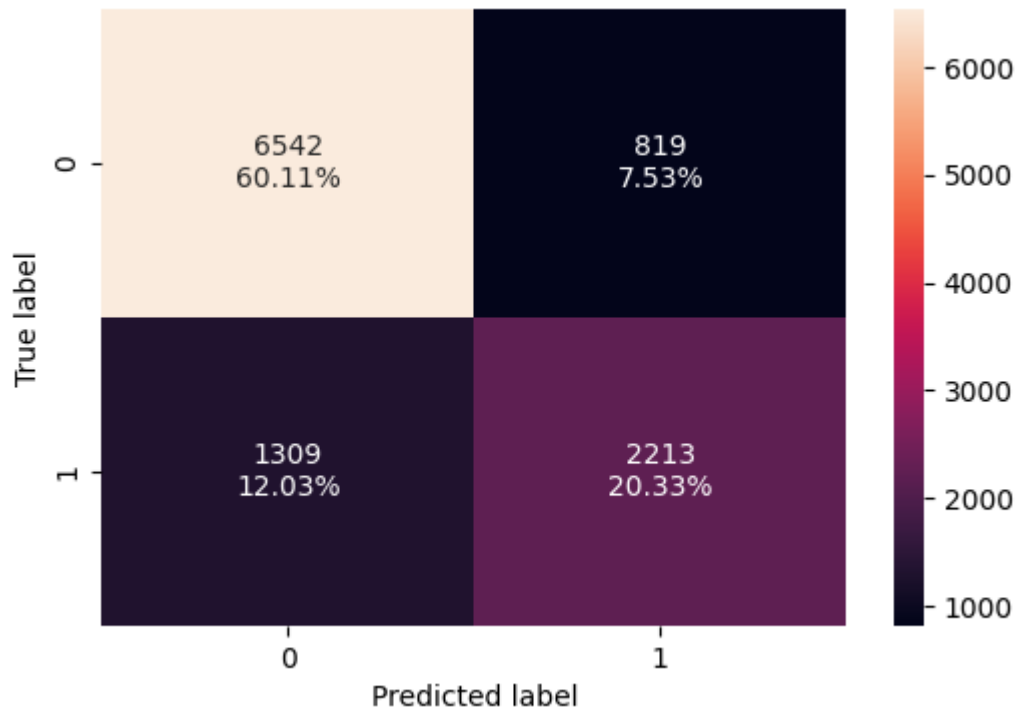
ROC-AUC on test set

```
In [ ]: # To get diferent metric scores
from sklearn import metrics
from sklearn.metrics import accuracy_score, roc_curve, confusion_matrix, roc_auc

logit_roc_auc_train = roc_auc_score(y_test, lg2.predict(X_test2))
fpr, tpr, thresholds = roc_curve(y_test, lg2.predict(X_test2))
plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, label="Logistic Regression (area = %0.2f)" % logit_roc_auc_train)
plt.plot([0, 1], [0, 1], "r--")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver operating characteristic")
plt.legend(loc="lower right")
plt.show()
```



```
In [ ]: #Confusion matrix
confusion_matrix_statsmodels(lg2, X_test2, y_test)
```



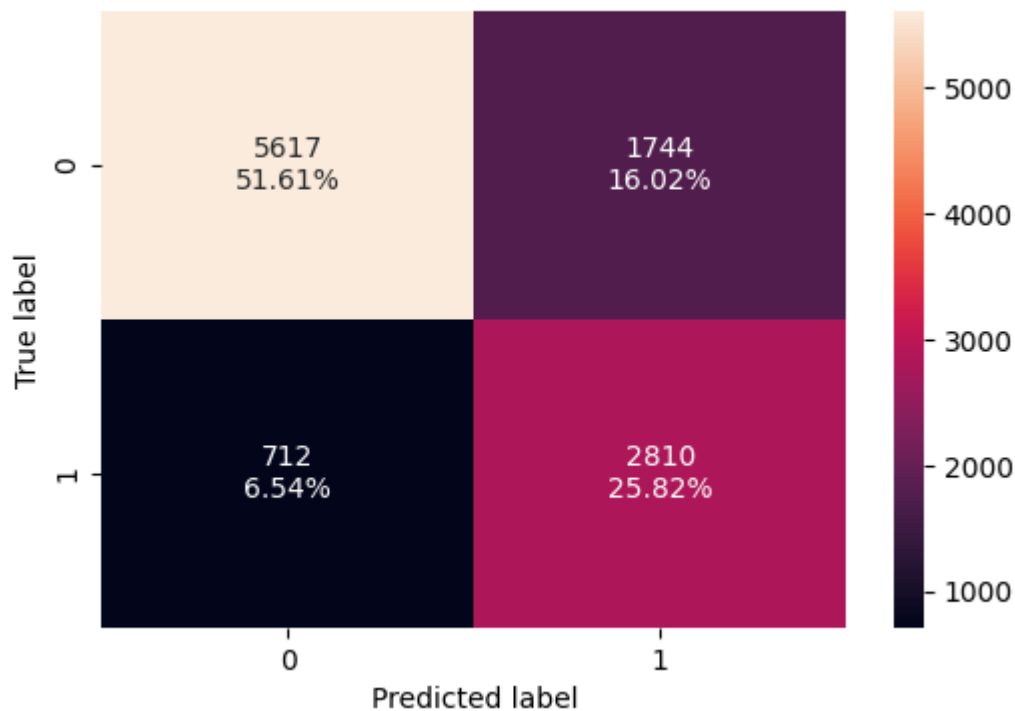
```
In [ ]: log_reg_model_test_perf=model_performance_classification_statsmodels(lg2, X_test2, y_test2)
print("Test performance:")
log_reg_model_test_perf
```

Test performance:

```
Out[ ]:      Accuracy    Recall    Precision    F1
0  0.804466  0.628336  0.729881  0.675313
```

Checking model with threshold=0.30

```
In [ ]: optimal_threshold_auc_roc = 0.30
# creating confusion matrix
confusion_matrix_statsmodels(
    lg2, X_test2, y_test, threshold=optimal_threshold_auc_roc
)
```



```
In [ ]: # checking model performance for this model
log_reg_model_test_perf_threshold_auc_roc = model_performance_classification_sta
    lg2, X_test2, y_test, threshold=optimal_threshold_auc_roc
)
print("Test performance:")
log_reg_model_test_perf_threshold_auc_roc
```

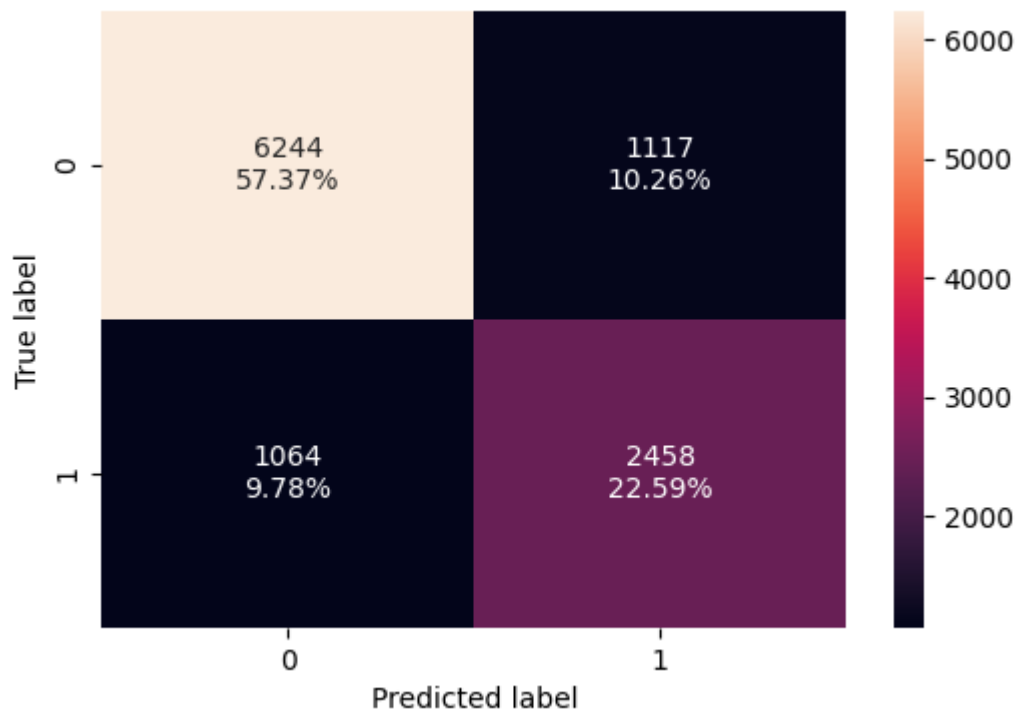
Test performance:

```
Out[ ]:      Accuracy    Recall    Precision    F1
0  0.774327  0.797842  0.61704  0.695889
```

Checking model with threshold = 0.42

```
In [ ]: optimal_threshold_curve = 0.42

# creating confusion matrix
confusion_matrix_statsmodels(lg2, X_test2, y_test, threshold=optimal_threshold_c
```



```
In [ ]: # checking model performance for this model
log_reg_model_test_perf_threshold_curve = model_performance_classification_stats
      lg2, X_test2, y_test, threshold=optimal_threshold_curve
    )
print("Test performance:")
log_reg_model_test_perf_threshold_curve
```

Test performance:

```
Out[ ]:   Accuracy   Recall   Precision   F1
0  0.799596  0.697899  0.687552  0.692687
```

Model performance summary

```
In [ ]: # test performance comparison

models_test_comp_df = pd.concat(
    [
        log_reg_model_test_perf.T,
        log_reg_model_test_perf_threshold_auc_roc.T,
        log_reg_model_test_perf_threshold_curve.T,
    ],
    axis=1,
)
models_test_comp_df.columns = [
    "Logistic Regression statsmodel",
    "Logistic Regression-0.30 Threshold",
    "Logistic Regression-0.42 Threshold",
]

print("Test performance comparison:")
models_test_comp_df
```

Test performance comparison:

Out[]:

	Logistic Regression statsmodel	Logistic Regression-0.30 Threshold	Logistic Regression-0.42 Threshold
Accuracy	0.804466	0.774327	0.799596
Recall	0.628336	0.797842	0.697899
Precision	0.729881	0.617040	0.687552
F1	0.675313	0.695889	0.692687

Observations from logistic regression model

- All the models are giving a generalized performance on training and test set.
- The highest F1 score is 69% on the training set.
- Using the model with default threshold the model will give a low recall but good precision score. The hotel will be able to predict which bookings will not be cancelled and will be able to provide satisfactory services to those customers which help in maintaining the brand equity but will lose on resources.
- Using the model with a 0.30 threshold the model will give a high recall but low precision score. The hotel will be able to save resources by correctly predicting the bookings which are likely to be cancelled but might damage brand quality.
- using the model with 0.42 threshold the model will give a balance recall and precision score. The hotel will be able to maintain a balance between resources and brand equity.

Model Building- Decision Tree

```
In [ ]: model0 = DecisionTreeClassifier(criterion="gini", random_state=1)
        model0.fit(X_train, y_train)
```

Out[]:

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=1)
```

```
In [ ]: print("Percentage of classes in training set:")
        print(y_train.value_counts(normalize=True))
        print("Percentage of classes in test set:")
        print(y_test.value_counts(normalize=True))
```

Percentage of classes in training set:

booking_status

0 0.670644

1 0.329356

Name: proportion, dtype: float64

Percentage of classes in test set:

booking_status

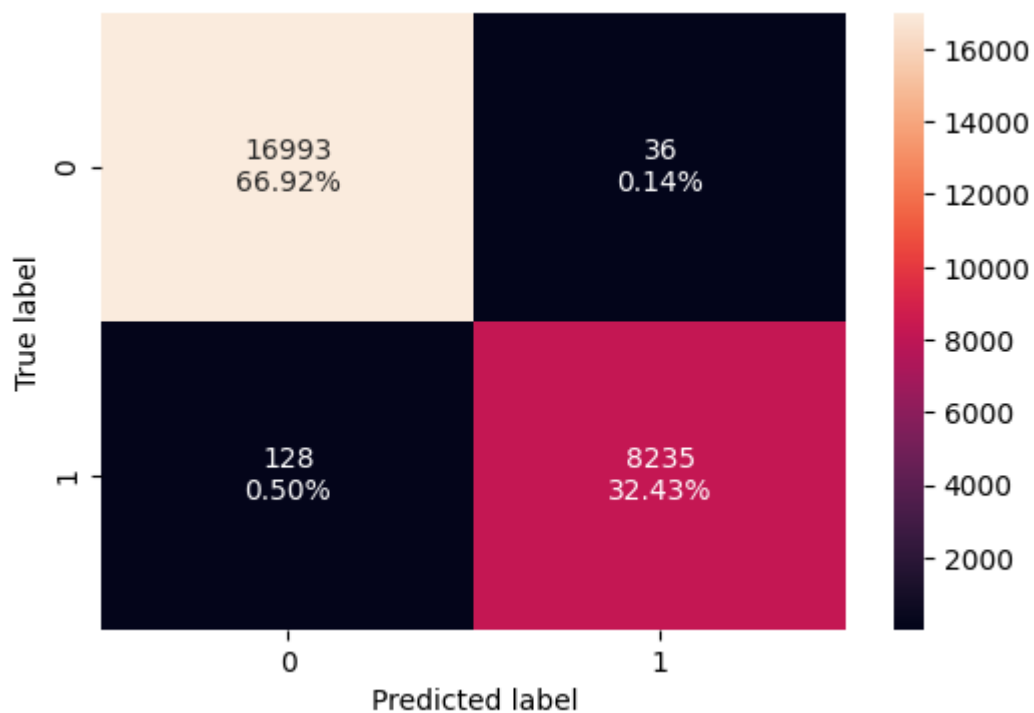
0 0.676376

1 0.323624

Name: proportion, dtype: float64

Checking model performance on Training set

```
In [ ]: confusion_matrix_statsmodels(model0, X_train, y_train)
```



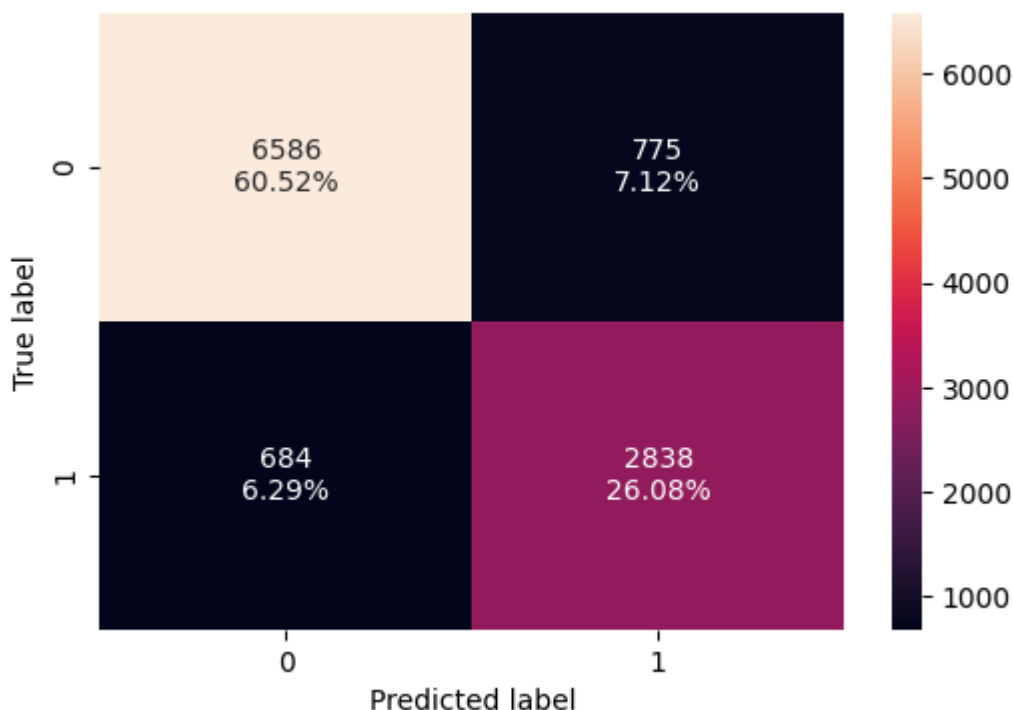
```
In [ ]: decision_tree_perf_train = model_performance_classification_statsmodels(
    model0, X_train, y_train
)
decision_tree_perf_train
```

```
Out[ ]:   Accuracy   Recall   Precision   F1
0  0.993541  0.984694  0.995647  0.990141
```

- There are almost 0 errors on the training set. It means each sample has been classified correctly.
- The model has performed very well on the training set.
- Let's check the performance on the test data to see if the model is overfitting.

Checking model performance on the Test set

```
In [ ]: confusion_matrix_statsmodels(model0, X_test, y_test)
```



```
In [ ]: decision_tree_perf_test = model_performance_classification_statsmodels(
        model0, X_test, y_test
    )
        decision_tree_perf_test
```

```
Out[ ]:   Accuracy   Recall   Precision   F1
0  0.865938  0.805792  0.785497  0.795515
```

- The decision tree model is overfitting the data as expected and not able to generalize well on the test set.
- We will have to prune the decision tree.

Before pruning tree let's check important features

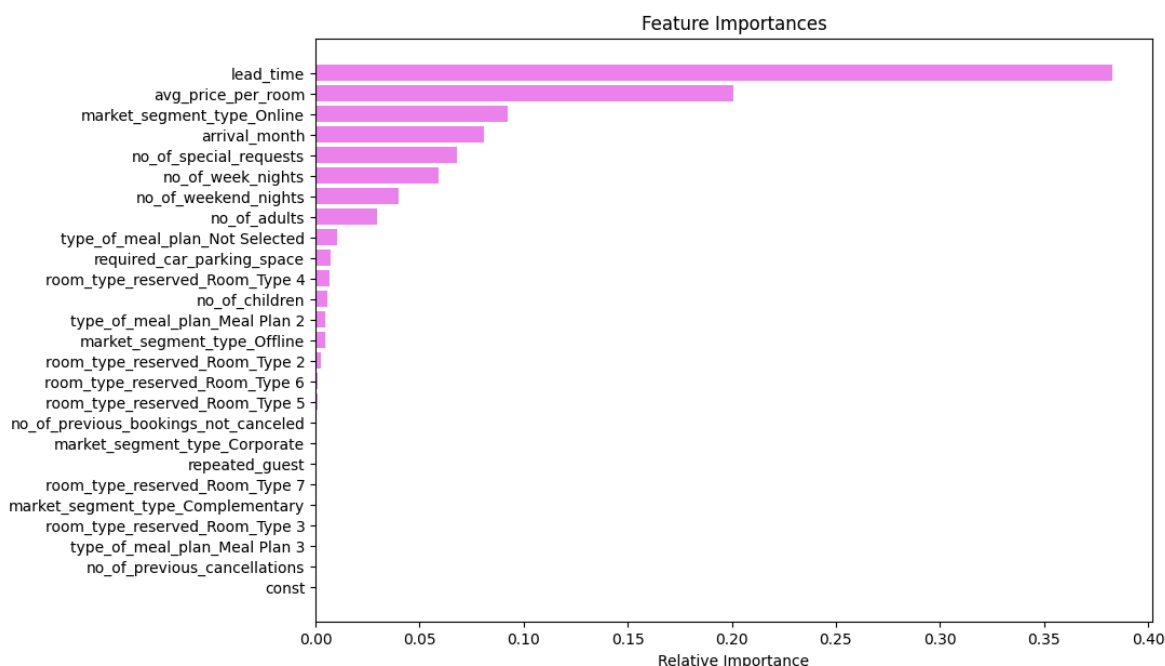
```
In [ ]: feature_columns=list(X.columns)
        feature_names=feature_columns
        print(feature_names)
```

```
['const', 'no_of_adults', 'no_of_children', 'no_of_weekend_nights', 'no_of_week_n
ights', 'required_car_parking_space', 'lead_time', 'arrival_month', 'repeated_gue
st', 'no_of_previous_cancellations', 'no_of_previous_bookings_not_canceled', 'avg
_price_per_room', 'no_of_special_requests', 'type_of_meal_plan_Meal Plan 2', 'typ
e_of_meal_plan_Meal Plan 3', 'type_of_meal_plan_Not Selected', 'room_type_reserve
d_Room_Type 2', 'room_type_reserved_Room_Type 3', 'room_type_reserved_Room_Type
4', 'room_type_reserved_Room_Type 5', 'room_type_reserved_Room_Type 6', 'room_typ
e_reserved_Room_Type 7', 'market_segment_type_Complementary', 'market_segment_typ
e_Corporate', 'market_segment_type_Offline', 'market_segment_type_Online']
```

```
In [ ]: importances = model0.feature_importances_
        indices = np.argsort(importances)

        plt.figure(figsize=(10, 7))
        plt.title("Feature Importances")
```

```
plt.barh(range(len(indices)), importances[indices], color="violet", align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.show()
```



- We can see that the lead time is the most important feature followed by average price per room.
- Now let's prune the tree to see if we can reduce the complexity.

Pruning the Tree

Pre_pruning

Using GridSearch for Hyperparameter tuning of our tree model

```
In [ ]: # Choose the type of classifier.
estimator = DecisionTreeClassifier(random_state=1)

# Grid of parameters to choose from

parameters = {
    "max_depth": np.arange(5, 13, 2),          ## Max Depth of t
    "max_leaf_nodes": [10, 20, 40, 50, 75, 100], ## Maximum number
    "min_samples_split": [2, 5, 7, 10, 20, 30], ## Minimum number
    "class_weight": ['balanced', None]        ## whether or not
}

# Type of scoring used to compare parameter combinations
acc_scorer = make_scorer(recall_score)

# Run the grid search
grid_obj = GridSearchCV(estimator, parameters, scoring=acc_scorer, cv=5)
grid_obj = grid_obj.fit(X_train, y_train)

# Set the clf to the best combination of parameters
```



```
estimator = grid_obj.best_estimator_

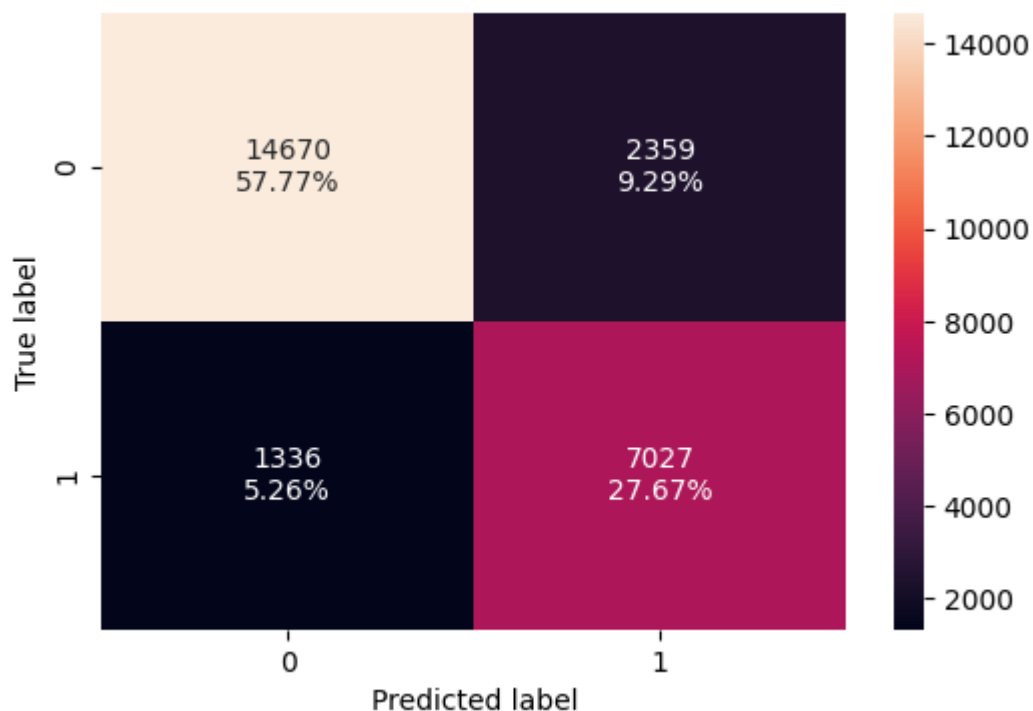
# Fit the best algorithm to the data.
estimator.fit(X_train, y_train)
```

Out[]: **DecisionTreeClassifier**

DecisionTreeClassifier(class_weight='balanced', max_depth=np.int64(9), max_leaf_nodes=100, min_samples_split=20, random_state=1)

Checking performance on Training set

```
In [ ]: confusion_matrix_statsmodels(estimator, X_train, y_train)
```



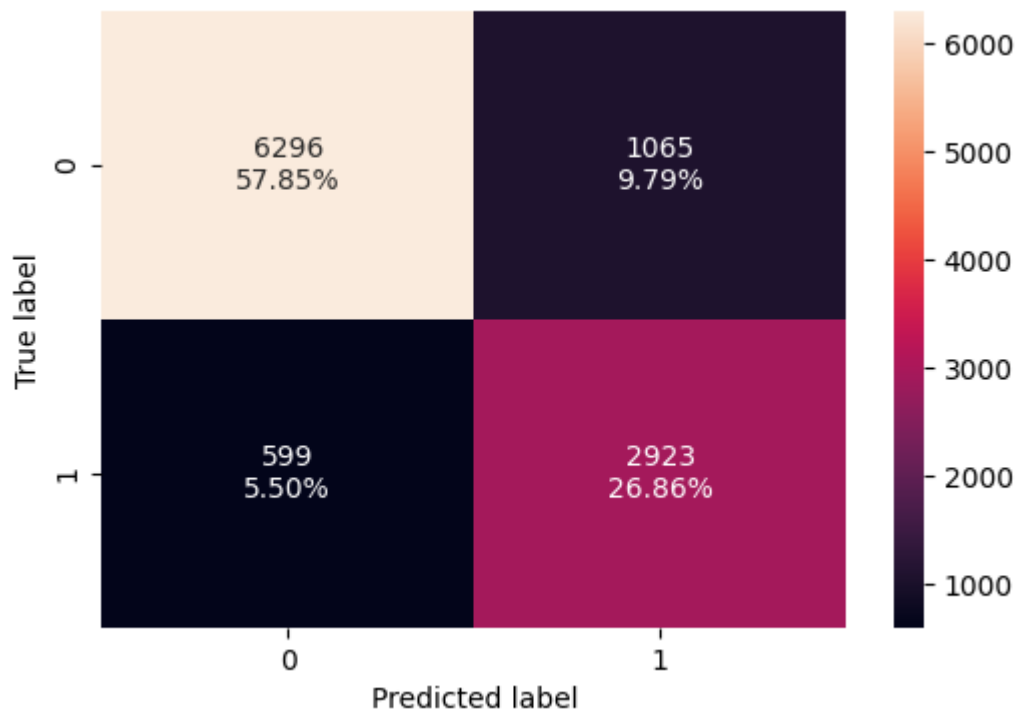
```
In [ ]: decision_tree_tune_perf_train = model_performance_classification_statsmodels(
        estimator, X_train, y_train
    )
print('Training Performance :')
decision_tree_tune_perf_train
```

Training Performance :

```
Out[ ]:   Accuracy  Recall  Precision  F1
0  0.854482  0.840249  0.748668  0.791819
```

Checking performance on Test set

```
In [ ]: confusion_matrix_statsmodels(estimator, X_test, y_test)
```



```
In [ ]: decision_tree_tune_perf_test = model_performance_classification_statsmodels(
        estimator, X_test, y_test
    )
    print('Test Performance')
    decision_tree_tune_perf_test
```

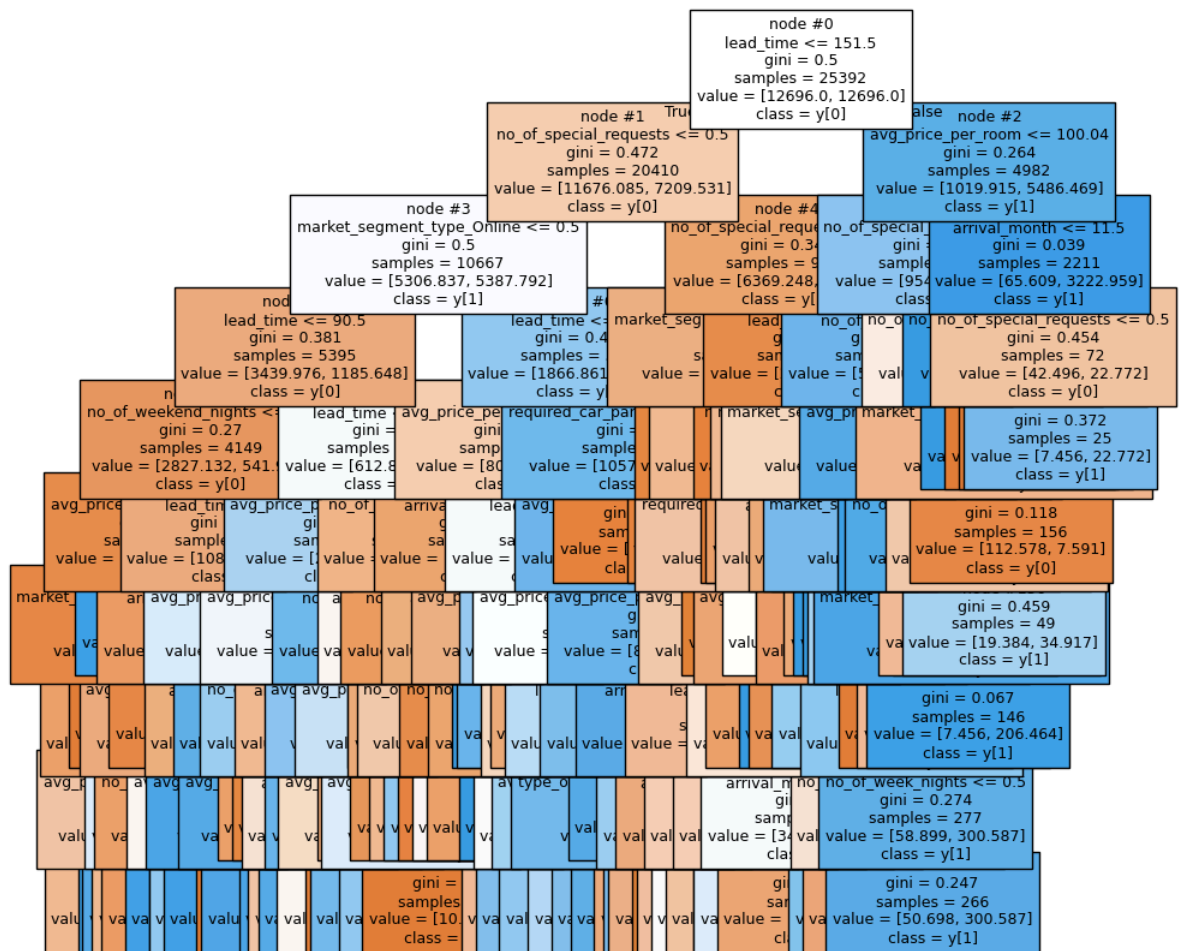
Test Performance

```
Out[ ]: Accuracy  Recall  Precision  F1
0 0.847101 0.829926 0.732949 0.778429
```

Let's visualizing the Decision Tree

```
In [ ]: plt.figure(figsize=(10, 10))

out = tree.plot_tree(
    estimator,
    feature_names=feature_names,
    filled=True,
    fontsize=9,
    node_ids=True,
    class_names=True,
)
for o in out:
    arrow = o.arrow_patch
    if arrow is not None:
        arrow.set_edgecolor("black")
        arrow.set_linewidth(1)
plt.show()
```



```
In [ ]: print(tree.export_text(estimator, feature_names=feature_names, show_weights=True))
```

```

|--- lead_time <= 151.50
|   |--- no_of_special_requests <= 0.50
|       |--- market_segment_type_Online <= 0.50
|           |--- lead_time <= 90.50
|               |--- no_of_weekend_nights <= 0.50
|                   |--- avg_price_per_room <= 196.50
|                       |--- weights: [1736.39, 132.08] class: 0
|                           |--- avg_price_per_room > 196.50
|                               |--- weights: [0.75, 25.81] class: 1
|                                   |--- no_of_weekend_nights > 0.50
|                                       |--- lead_time <= 68.50
|                                           |--- weights: [960.27, 223.16] class: 0
|                                               |--- lead_time > 68.50
|                                                   |--- weights: [129.73, 160.92] class: 1
|                                                       |--- lead_time > 90.50
|                                                           |--- lead_time <= 117.50
|                                                               |--- avg_price_per_room <= 93.58
|                                                                   |--- weights: [214.72, 227.72] class: 1
|                                                                       |--- avg_price_per_room > 93.58
|                                                                           |--- weights: [82.76, 285.41] class: 1
|                                                                               |--- lead_time > 117.50
|                                                                                   |--- no_of_week_nights <= 1.50
|                                                                                       |--- weights: [87.23, 81.98] class: 0
|                                                                                           |--- no_of_week_nights > 1.50
|                                                                                               |--- weights: [228.14, 48.58] class: 0
|                                                                                                   |--- market_segment_type_Online > 0.50
|                                                                                                       |--- lead_time <= 13.50
|                                                                                                           |--- avg_price_per_room <= 99.44
|                                                                                                               |--- arrival_month <= 1.50
|                                                                                                                   |--- weights: [92.45, 0.00] class: 0
|                                                                                                                       |--- arrival_month > 1.50
|                                                                                                                           |--- weights: [363.83, 132.08] class: 0
|                                                                                                       |--- avg_price_per_room > 99.44
|                                                                                                           |--- lead_time <= 3.50
|                                                                                                               |--- weights: [219.94, 85.01] class: 0
|                                                                                                                   |--- lead_time > 3.50
|                                                                                                                       |--- weights: [132.71, 280.85] class: 1
|                                                                                                       |--- lead_time > 13.50
|                                                                                                           |--- required_car_parking_space <= 0.50
|                                                                                                               |--- avg_price_per_room <= 71.92
|                                                                                                                   |--- weights: [158.80, 159.40] class: 1
|                                                                                                                       |--- avg_price_per_room > 71.92
|                                                                                                                           |--- weights: [850.67, 3543.28] class: 1
|                                                                                                       |--- required_car_parking_space > 0.50
|                                                                                                           |--- no_of_weekend_nights <= 3.00
|                                                                                                               |--- weights: [48.46, 0.00] class: 0
|                                                                                                                   |--- no_of_weekend_nights > 3.00
|                                                                                                                       |--- weights: [0.00, 1.52] class: 1
|                                                                                                       |--- no_of_special_requests > 0.50
|                                                                                                           |--- no_of_special_requests <= 1.50
|                                                                                                               |--- market_segment_type_Online <= 0.50
|                                                                                                                   |--- lead_time <= 102.50
|                                                                                                                       |--- type_of_meal_plan_Not Selected <= 0.50
|                                                                                                                           |--- weights: [697.09, 9.11] class: 0
|                                                                                                       |--- type_of_meal_plan_Not Selected > 0.50
|                                                                                                           |--- weights: [15.66, 9.11] class: 0
|                                                                                                               |--- lead_time > 102.50
|                                                                                                                   |--- no_of_week_nights <= 2.50
|                                                                                                                       |--- weights: [32.06, 19.74] class: 0
|                                                                                                       |--- no_of_week_nights > 2.50

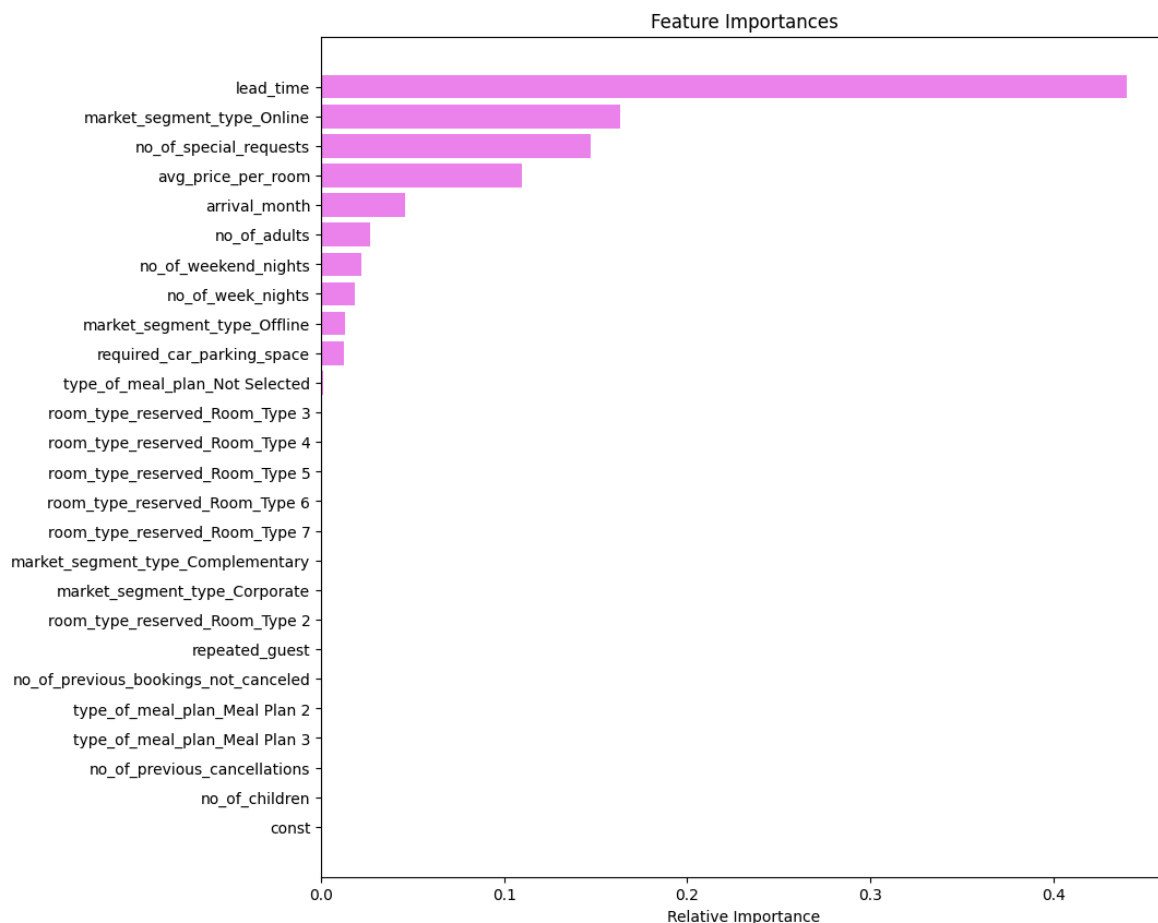
```

61/81

[illegible]

```
In [ ]: importances = estimator.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize=(10, 10))
plt.title("Feature Importances")
plt.barh(range(len(indices)), importances[indices], color="violet", align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.show()
```



- Lead time is the most important feature followed by market segment-online, number of special requests, average price per room.
- The model performance has been generalized.
- We can see that lead time plays a key role in identifying if a booking will be cancelled or not. 151 days has been considered as a threshold value by the model to make the first split.
- If the average price per room is greater than 100 euros and the arrival month is December, then the booking is less likely to be cancelled.
- If the average price per room is less than or equal to 100 euros and the number of special requests is 0, then the booking is likely to get cancelled.
- If the customer has at least 1 request the booking is less likely to be cancelled.
- If the customer didn't make any special requests and the booking was done online it is more likely to get cancelled. If the booking was not done online, it is less likely to get cancelled.

Cost Complexity Pruning

```
In [ ]: clf = DecisionTreeClassifier(random_state=1)
path = clf.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities
```

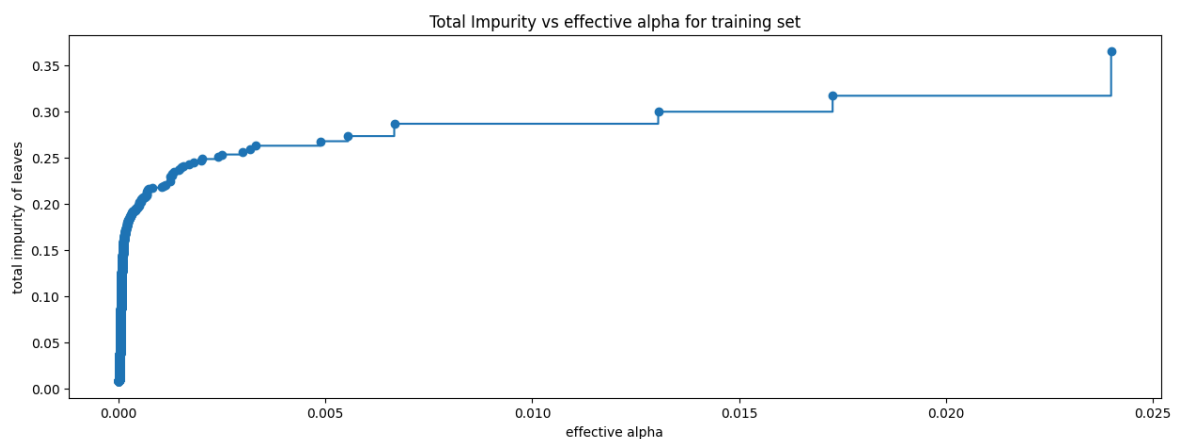
```
In [ ]: pd.DataFrame(path)
```

Out[]:

	ccp_alphas	impurities
0	0.000000e+00	0.008309
1	4.327745e-07	0.008309
2	4.688391e-07	0.008310
3	5.329960e-07	0.008310
4	5.967043e-07	0.008311
...	...	...
1470	6.665684e-03	0.286897
1471	1.304480e-02	0.299942
1472	1.725993e-02	0.317202
1473	2.399048e-02	0.365183
1474	7.657789e-02	0.441761

1475 rows × 2 columns

```
In [ ]: fig, ax = plt.subplots(figsize=(15, 5))
ax.plot(ccp_alphas[:-1], impurities[:-1], marker="o", drawstyle="steps-post")
ax.set_xlabel("effective alpha")
ax.set_ylabel("total impurity of leaves")
ax.set_title("Total Impurity vs effective alpha for training set")
plt.show()
```



- Next we train a decision tree using effective alphas. The last value in the `ccp_alphas` is the alpha value that prunes the whole tree, leaving the tree `clfs[-1]`, with one node.

```
In [ ]: clfs = []
for ccp_alpha in ccp_alphas:
    clf = DecisionTreeClassifier(random_state=1, ccp_alpha=ccp_alpha)
    clf.fit(X_train, y_train)
    clfs.append(clf)
print(
    "Number of nodes in the last tree is: {} with ccp_alpha: {}".format(
        clfs[-1].tree_.node_count, ccp_alphas[-1]
    )
)
```



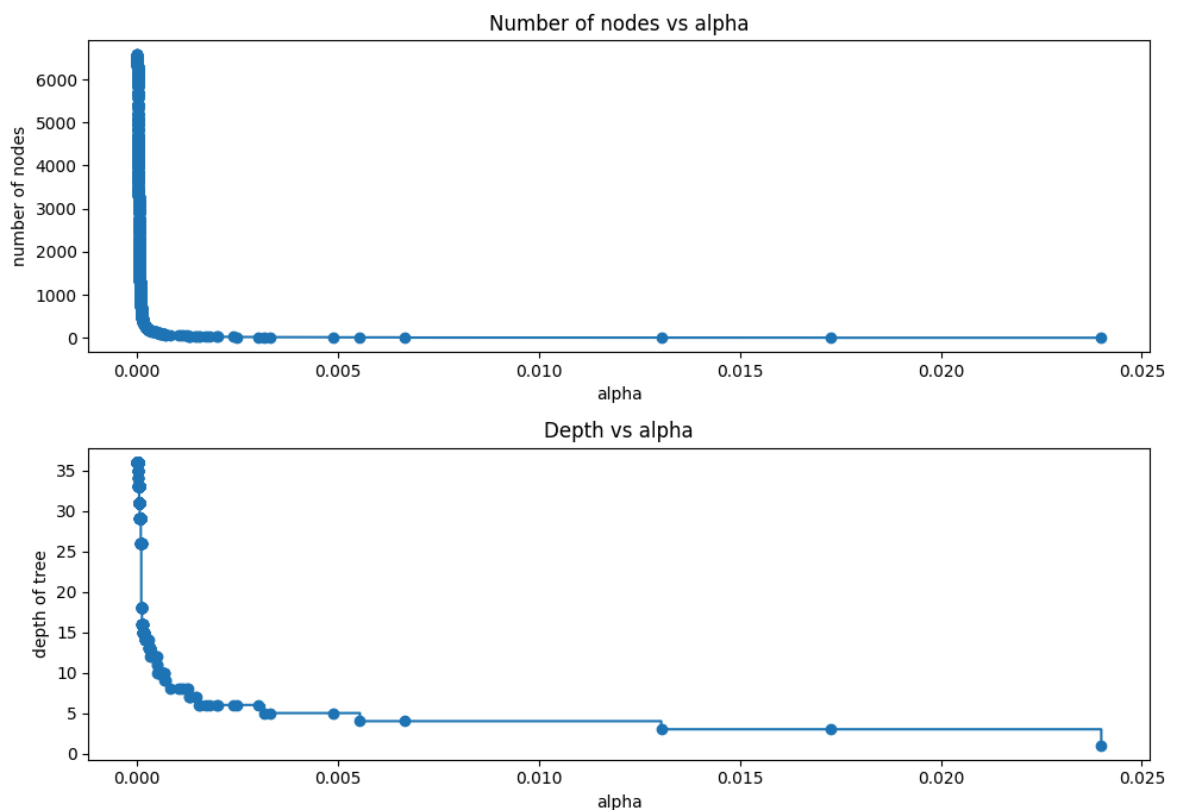
```
)
)
```

Number of nodes in the last tree is: 3 with ccp_alpha: 0.023990479445095594

- For the remainder, we remove the last element in clfs and ccp_alphas, because it is a trivial tree with only one node. Here we show the number of nodes and Tree depth decreases as alpha increases.

```
In [ ]: clfs = clfs[:-1]
        ccp_alphas = ccp_alphas[:-1]

        node_counts = [clf.tree_.node_count for clf in clfs]
        depth = [clf.tree_.max_depth for clf in clfs]
        fig, ax = plt.subplots(2, 1, figsize=(10, 7))
        ax[0].plot(ccp_alphas, node_counts, marker="o", drawstyle="steps-post")
        ax[0].set_xlabel("alpha")
        ax[0].set_ylabel("number of nodes")
        ax[0].set_title("Number of nodes vs alpha")
        ax[1].plot(ccp_alphas, depth, marker="o", drawstyle="steps-post")
        ax[1].set_xlabel("alpha")
        ax[1].set_ylabel("depth of tree")
        ax[1].set_title("Depth vs alpha")
        fig.tight_layout()
```

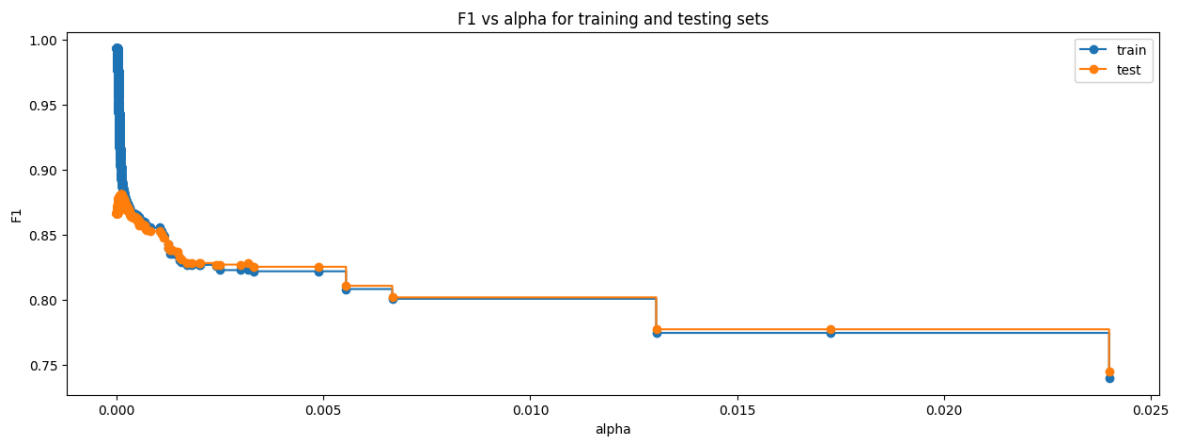


F1 score vs alpha for training and test sets

```
In [ ]: train_f1 = [f1_score(y_train, clf.predict(X_train), average='weighted') for clf in clfs]
        test_f1 = [f1_score(y_test, clf.predict(X_test), average='weighted') for clf in clfs]
```

```
In [ ]: fig, ax = plt.subplots(figsize=(15, 5))
        ax.set_xlabel("alpha")
        ax.set_ylabel("F1")
```

```
ax.set_title("F1 vs alpha for training and testing sets")
ax.plot(ccp_alphas, train_f1, marker="o", label="train", drawstyle="steps-post")
ax.plot(ccp_alphas, test_f1, marker="o", label="test", drawstyle="steps-post")
ax.legend()
plt.show()
```

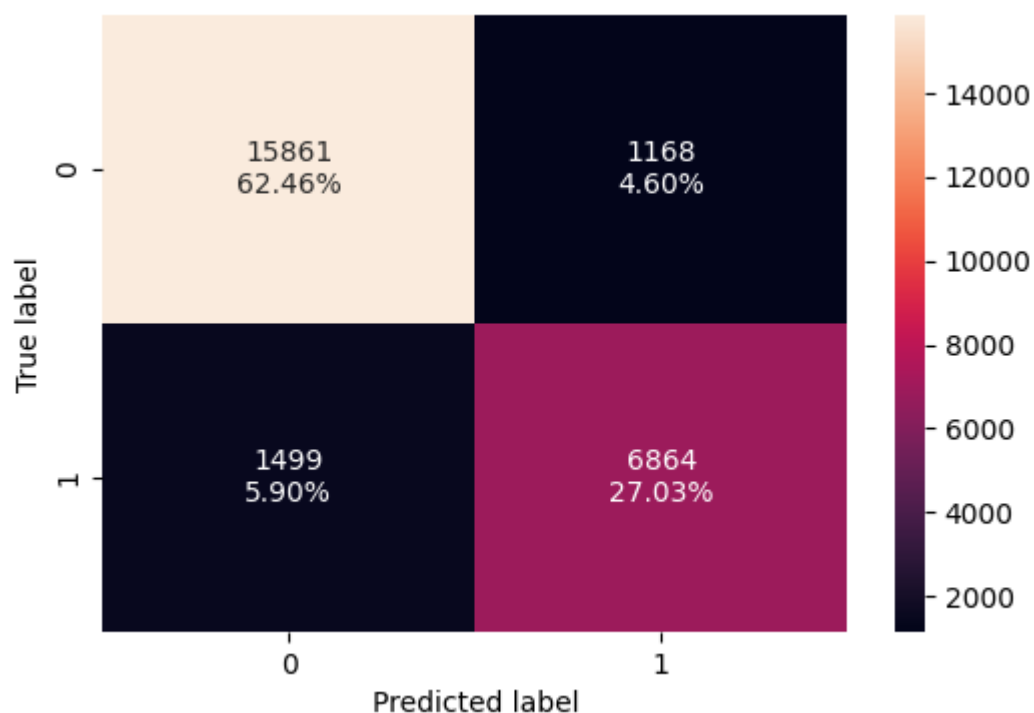


```
In [ ]: # creating the model where we get highest train and test recall
index_best_model = np.argmax(test_f1)
best_model = clfs[index_best_model]
print(best_model)
```

```
DecisionTreeClassifier(ccp_alpha=np.float64(0.00012252327942309038),
                      random_state=1)
```

Checking performance on Training set

```
In [ ]: confusion_matrix_statsmodels(best_model, X_train, y_train)
```



```
In [ ]: decision_tree_postpruned_perf_train = model_performance_classification_statsmode
        best_model, X_train, y_train
    )
print('Training Performance')
decision_tree_postpruned_perf_train
```

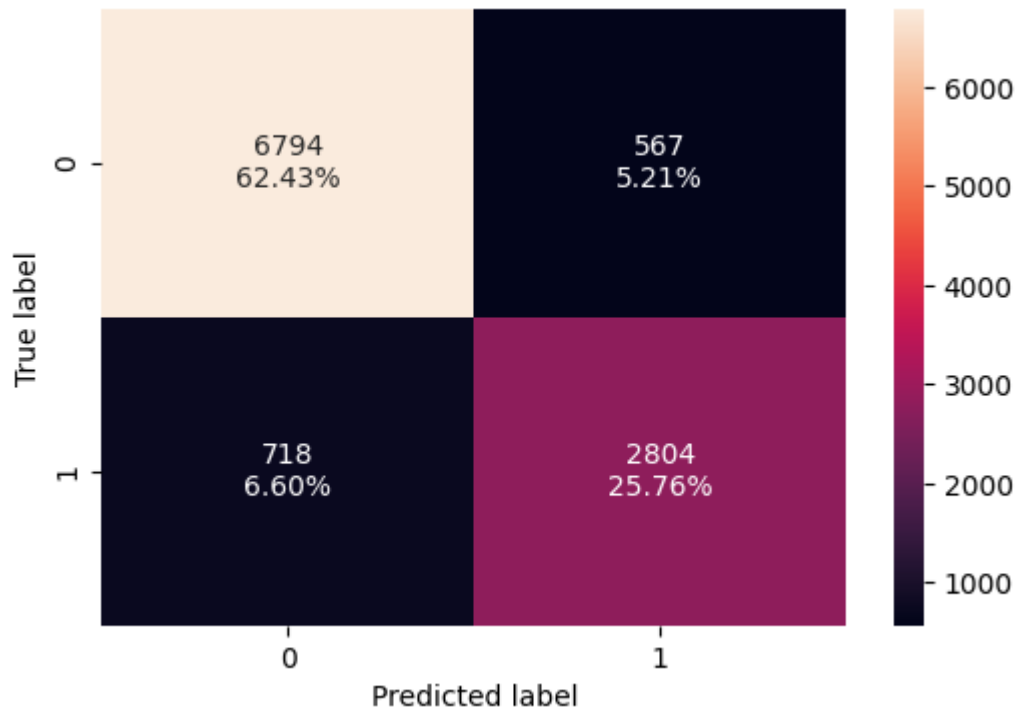
Training Performance

```
Out[ ]:
```

	Accuracy	Recall	Precision	F1
0	0.894967	0.820758	0.854582	0.837328

Checking Test performance

```
In [ ]: confusion_matrix_statsmodels(best_model, X_test, y_test)
```



```
In [ ]: decision_tree_postpruned_perf_test = model_performance_classification_statsmodel
        best_model, X_test, y_test
        )
        print('Test Performance')
        decision_tree_postpruned_perf_test
```

Test Performance

```
Out[ ]:
```

	Accuracy	Recall	Precision	F1
0	0.881926	0.796139	0.831801	0.813579

- After post pruning the decision tree the performance has generalized on training and test set.

Decision Tree- Post Pruning

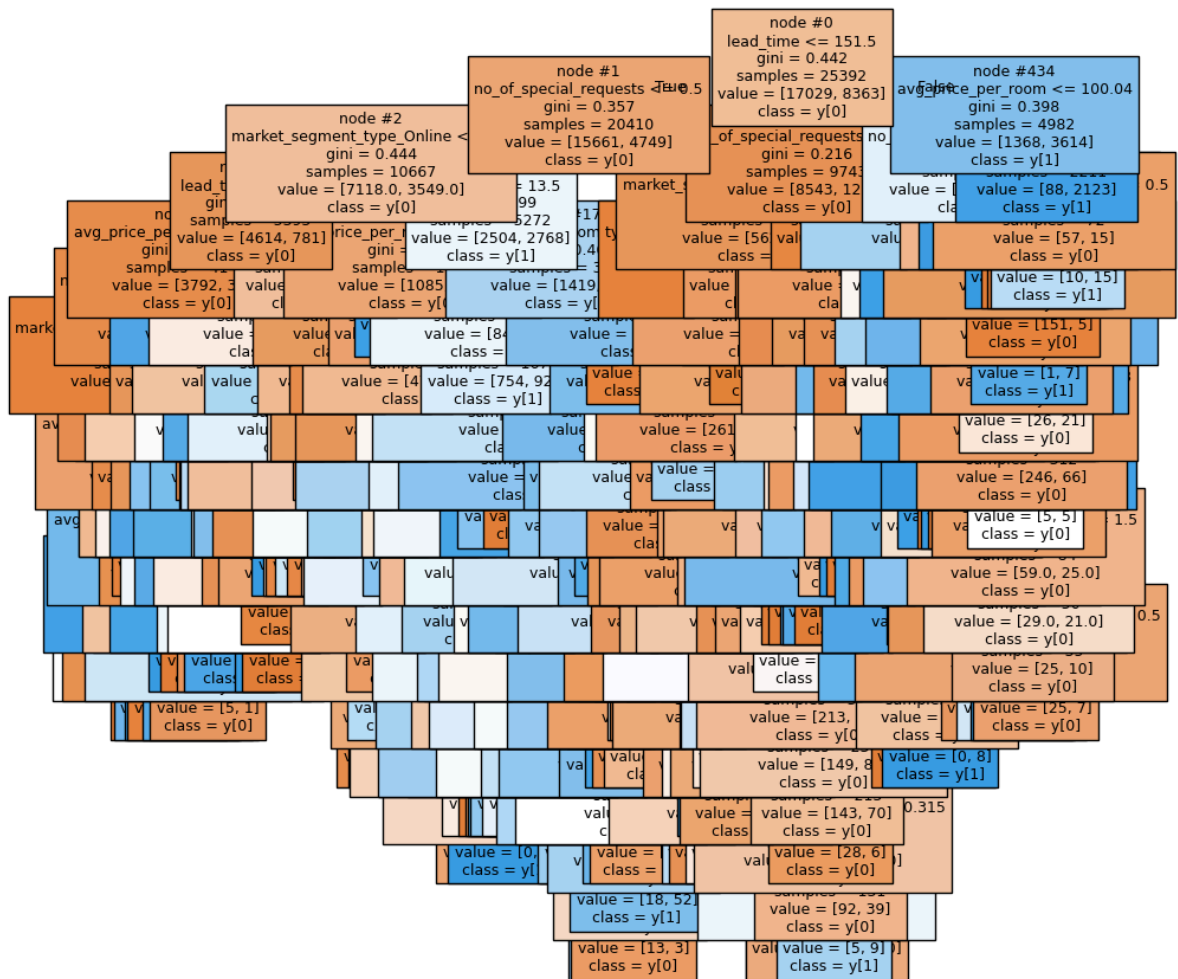
```
In [ ]: plt.figure(figsize=(10, 10))

out = tree.plot_tree(
    best_model,
    feature_names=feature_names,
```

```

filled=True,
fontsize=9,
node_ids=True,
class_names=True,
)
for o in out:
    arrow = o.arrow_patch
    if arrow is not None:
        arrow.set_edgecolor("black")
        arrow.set_linewidth(1)
plt.show()
plt.show()

```



In []: # Text report showing the rules of a decision tree -

```
print(tree.export_text(best_model, feature_names=feature_names, show_weights=True))
```

```

|--- lead_time <= 151.50
|   |--- no_of_special_requests <= 0.50
|       |--- market_segment_type_Online <= 0.50
|           |--- lead_time <= 90.50
|               |--- avg_price_per_room <= 201.50
|                   |--- no_of_weekend_nights <= 0.50
|                       |--- market_segment_type_Offline <= 0.50
|                           |--- lead_time <= 16.50
|                               |--- weights: [558.00, 43.00] class: 0
|                                   |--- lead_time > 16.50
|                                       |--- avg_price_per_room <= 135.00
|                                           |--- weights: [162.00, 36.00] class: 0
|                                               |--- avg_price_per_room > 135.00
|                                                   |--- weights: [0.00, 8.00] class: 1
|                                                       |--- market_segment_type_Offline > 0.50
|                                                           |--- weights: [1609.00, 0.00] class: 0
|                                                               |--- no_of_weekend_nights > 0.50
|                                                                   |--- lead_time <= 68.50
|                                                                       |--- no_of_weekend_nights <= 4.50
|                                                                           |--- lead_time <= 1.50
|                                                                               |--- arrival_month <= 2.50
|                                                                                   |--- avg_price_per_room <= 62.50
|                                                                                       |--- truncated branch of depth 2
|                                                                                           |--- avg_price_per_room > 62.50
|                                                                                               |--- weights: [8.00, 0.00] class: 0
|                                                                                                   |--- arrival_month > 2.50
|                                                                                                       |--- weights: [97.00, 7.00] class: 0
|                                                                                                           |--- lead_time > 1.50
|                                                                                                               |--- arrival_month <= 9.50
|                                                                                                                   |--- lead_time <= 59.50
|                                                                                                                       |--- weights: [616.00, 68.00] class:
0
|                                                                                                       |--- lead_time > 59.50
|                                                                                                           |--- truncated branch of depth 3
|                                                                                                               |--- arrival_month > 9.50
|                                                                                                                   |--- weights: [524.00, 16.00] class: 0
|                                                                                                                       |--- no_of_weekend_nights > 4.50
|                                                                                                                           |--- weights: [0.00, 8.00] class: 1
|                                                                                                                               |--- lead_time > 68.50
|                                                                                                                                   |--- avg_price_per_room <= 99.98
|                                                                                                                                       |--- arrival_month <= 3.50
|                                                                                                                                           |--- avg_price_per_room <= 62.50
|                                                                                                                                               |--- weights: [21.00, 0.00] class: 0
|                                                                                                       |--- avg_price_per_room > 62.50
|                                                                                                           |--- lead_time <= 77.00
|                                                                                                               |--- weights: [1.00, 9.00] class: 1
|                                                                                                                   |--- lead_time > 77.00
|                                                                                                                       |--- truncated branch of depth 3
|                                                                                                       |--- arrival_month > 3.50
|                                                                                                           |--- weights: [103.00, 8.00] class: 0
|                                                                                                               |--- avg_price_per_room > 99.98
|                                                                                                                   |--- no_of_adults <= 1.50
|                                                                                                                       |--- weights: [1.00, 52.00] class: 1
|                                                                                                       |--- no_of_adults > 1.50
|                                                                                                           |--- avg_price_per_room <= 105.20
|                                                                                                               |--- lead_time <= 74.50
|                                                                                                                   |--- weights: [2.00, 0.00] class: 0
|                                                                                                                       |--- lead_time > 74.50
|                                                                                                       |--- truncated branch of depth 2
|                                                                                                           |--- avg_price_per_room > 105.20

```

[illegible]

71/81

72/81

50

[illegible]

[illegible]

```

|--- avg_price_per_room <= 97.50
|   |--- no_of_weekend_nights <= 0.50
|       |--- weights: [0.00, 9.00] class: 1
|   |--- no_of_weekend_nights > 0.50
|       |--- weights: [4.00, 0.00] class: 0
|   |--- avg_price_per_room > 97.50
|       |--- weights: [62.00, 6.00] class: 0
|   |--- lead_time > 173.00
|       |--- weights: [262.00, 8.00] class: 0
|   |--- lead_time > 341.00
|       |--- no_of_week_nights <= 4.00
|           |--- weights: [17.00, 10.00] class: 0
|       |--- no_of_week_nights > 4.00
|           |--- weights: [1.00, 8.00] class: 1
|--- no_of_adults > 1.50
|   |--- avg_price_per_room <= 84.58
|       |--- lead_time <= 244.00
|           |--- no_of_week_nights <= 1.50
|               |--- no_of_weekend_nights <= 1.50
|                   |--- lead_time <= 166.50
|                       |--- weights: [3.00, 0.00] class: 0
|                   |--- lead_time > 166.50
|                       |--- weights: [3.00, 38.00] class: 1
|               |--- no_of_weekend_nights > 1.50
|                   |--- weights: [24.00, 0.00] class: 0
|           |--- no_of_week_nights > 1.50
|               |--- avg_price_per_room <= 66.50
|                   |--- no_of_weekend_nights <= 0.50
|                       |--- weights: [2.00, 7.00] class: 1
|                   |--- no_of_weekend_nights > 0.50
|                       |--- weights: [17.00, 2.00] class: 0
|               |--- avg_price_per_room > 66.50
|                   |--- weights: [123.00, 9.00] class: 0
|           |--- lead_time > 244.00
|               |--- arrival_month <= 11.50
|                   |--- avg_price_per_room <= 75.83
|                       |--- no_of_week_nights <= 0.50
|                           |--- weights: [11.00, 0.00] class: 0
|                       |--- no_of_week_nights > 0.50
|                           |--- no_of_week_nights <= 4.50
|                               |--- truncated branch of depth 4
|                                   |--- no_of_week_nights > 4.50
|                                       |--- weights: [2.00, 0.00] class: 0
|                   |--- avg_price_per_room > 75.83
|                       |--- lead_time <= 292.50
|                           |--- no_of_week_nights <= 4.50
|                               |--- weights: [34.00, 4.00] class: 0
|                           |--- no_of_week_nights > 4.50
|                               |--- weights: [1.00, 4.00] class: 1
|                       |--- lead_time > 292.50
|                           |--- no_of_week_nights <= 3.50
|                               |--- weights: [1.00, 28.00] class: 1
|                           |--- no_of_week_nights > 3.50
|                               |--- weights: [11.00, 0.00] class: 0
|                   |--- arrival_month > 11.50
|                       |--- weights: [37.00, 0.00] class: 0
|   |--- avg_price_per_room > 84.58
|       |--- arrival_month <= 11.50
|           |--- no_of_weekend_nights <= 1.50
|               |--- room type reserved Room Type 4 <= 0.50

```

[illegible]

```

| | | --- arrival_month <= 11.50
| | | | --- no_of_special_requests <= 2.50
| | | | | --- weights: [0.00, 2108.00] class: 1
| | | | --- no_of_special_requests > 2.50
| | | | | --- weights: [31.00, 0.00] class: 0
| | | --- arrival_month > 11.50
| | | | --- no_of_special_requests <= 0.50
| | | | | --- weights: [47.00, 0.00] class: 0
| | | | --- no_of_special_requests > 0.50
| | | | | --- weights: [10.00, 15.00] class: 1

```

In []: *# importance of features in the tree building (The importance of a feature is c
(normalized) total reduction of the 'criterion' brought by that feature. It is*

```

print(
    pd.DataFrame(
        best_model.feature_importances_, columns=["Imp"], index=X_train.columns
    ).sort_values(by="Imp", ascending=False)
)

```

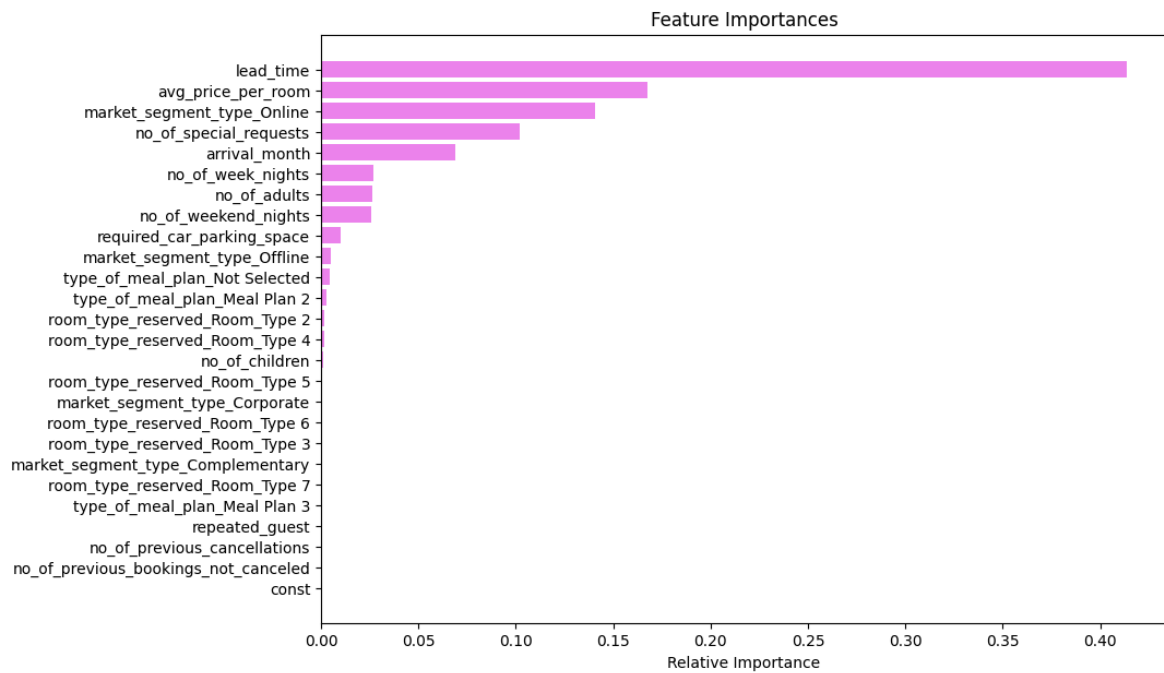
	Imp
lead_time	0.413746
avg_price_per_room	0.167364
market_segment_type_Online	0.140725
no_of_special_requests	0.101904
arrival_month	0.068705
no_of_week_nights	0.026865
no_of_adults	0.026611
no_of_weekend_nights	0.025831
required_car_parking_space	0.010276
market_segment_type_Offline	0.005175
type_of_meal_plan_Not Selected	0.004586
type_of_meal_plan_Meal Plan 2	0.002554
room_type_reserved_Room_Type 2	0.001931
room_type_reserved_Room_Type 4	0.001896
no_of_children	0.001398
room_type_reserved_Room_Type 5	0.000435
const	0.000000
repeated_guest	0.000000
no_of_previous_cancellations	0.000000
room_type_reserved_Room_Type 3	0.000000
no_of_previous_bookings_not_canceled	0.000000
type_of_meal_plan_Meal Plan 3	0.000000
room_type_reserved_Room_Type 7	0.000000
room_type_reserved_Room_Type 6	0.000000
market_segment_type_Corporate	0.000000
market_segment_type_Complementary	0.000000

```

In [ ]: importances = best_model.feature_importances_
indices = np.argsort(importances)

plt.figure(figsize=(10, 7))
plt.title("Feature Importances")
plt.barh(range(len(indices)), importances[indices], color="violet", align="center")
plt.yticks(range(len(indices)), [feature_names[i] for i in indices])
plt.xlabel("Relative Importance")
plt.show()

```



Comparison of Models and Final Model Selection

```
In [ ]: # training performance comparison

models_train_comp_df = pd.concat(
    [
        decision_tree_perf_train.T,
        decision_tree_tune_perf_train.T,
        decision_tree_postpruned_perf_train.T,
    ],
    axis=1,
)
models_train_comp_df.columns = [
    "Decision Tree sklearn",
    "Decision Tree (Pre-Pruning)",
    "Decision Tree (Post-Pruning)",
]
print("Training performance comparison:")
models_train_comp_df
```

Training performance comparison:

Out []:	Decision Tree sklearn	Decision Tree (Pre- Pruning)	Decision Tree (Post- Pruning)
Accuracy	0.993541	0.854482	0.894967
Recall	0.984694	0.840249	0.820758
Precision	0.995647	0.748668	0.854582
F1	0.990141	0.791819	0.837328

```
In [ ]: # test performance comparison
```

```
models_train_comp_df = pd.concat(  
    [  
        decision_tree_perf_test.T,  
        decision_tree_tune_perf_test.T,  
        decision_tree_postpruned_perf_test.T,  
    ],  
    axis=1,  
)  
models_train_comp_df.columns = [  
    "Decision Tree sklearn",  
    "Decision Tree (Pre-Pruning)",  
    "Decision Tree (Post-Pruning)",  
)  
print("Test set performance comparison:")  
models_train_comp_df
```

Test set performance comparison:

Out[]:	Decision Tree sklearn	Decision Tree (Pre- Pruning)	Decision Tree (Post- Pruning)
Accuracy	0.865938	0.847101	0.881926
Recall	0.805792	0.829926	0.796139
Precision	0.785497	0.732949	0.831801
F1	0.795515	0.778429	0.813579

- Decision Tree model with default parameters overfitting the training data and is not able to generalize well.
- Decision Tree after post-Pruning gives us the best result on both training and test set.

Actionable Insights and Recommendations:

Insights

- We can see that the Decision Tree model performs better on the dataset.
- Looking at the important variables based on p-values in logistic regression and feature importance in the Decision Tree model.
- Lead time, Number of special requests, Average price per room are important in both models.
- From the Logistic Regression model we observe that Lead time and Average price per room have a positive relation with bookings getting cancelled. And the number of special requests has a negative relation with bookings getting cancelled.

Recommendations

- The hotel can take actions like setting up a system where the customers get notifications via email before the arrival date and asking for re-confirmation of their booking and any changes they would like to make in their bookings.
- Better cancellation policies can be adopted by the hotel and the refunds, cancellation fee, etc should be highlighted on the website/app before a customer confirms their booking.
- The length of stay can be restricted. Hotel can allow bookings up to 5 days only and then they have to allow customers to re-book if they wish to stay longer.
- October and September have the highest number of bookings but also a high number of cancellations. This should be investigated further by the hotel.
- Post booking interactions will show the guests the level of attention and care that they would receive at the hotel.
- Improving the experience of repeated customers. The cancellation among them is very less which is a good indication as repeat customers are important to improve the hotel's business infrastructure.